

DEEP LEARNING, TOLD SIMPLY

Practice Set • Modules 1–6

WRITTEN BY

Prof. Saurabh

60 Exam Questions

Ten exam-style problems per module, each with sub-parts, a plan of attack, a full step-by-step solution, and the places where marks are lost

Sixty questions in the shapes an exam actually uses: compute by hand, recover a hidden value from a trajectory, derive then apply, diagnose a recorded run, write the matrix form, design and justify, find the slip. Every solution follows its question. Built to match Modules 1–6 of the Deep Neural Network course, and the six companion books.



What is inside

Six sections, one per module. Each section opens with a table of its ten questions (topic, shape, slides, marks) so you can see at a glance what the deck covers and what can be asked.

1	Module 1: Introduction to Deep Learning	3
Ten mid-sem-style questions: a concrete scenario with all its numbers, a numeric core to compute completely, and one or two short pointed sub-parts with a specific expected answer.		
2	Module 2: Artificial Neural Network and the Perceptron	21
Ten scenario questions on one artificial cell: weighted sums and thresholds, the perceptron learning rule traced to convergence, lines and half-planes, logged runs read backwards, the vector form, brain-scale arithmetic, linear separability, the perceptron design question for a four-row table, and the two output conventions with two learning rates.		
3	Module 3: Linear Neural Networks for Regression	39
Ten mid-sem-style questions on one neuron that predicts "how much": weight updates with two learning rates, full gradient-descent iterations in matrix form, MSE/RMSE/MAE/ R^2 , feature scaling, reading loss curves, recovering η , identity versus sigmoid output, stopping rules, and a Keras regression snippet.		
4	Module 4: Linear Neural Networks for Classification	58
Ten mid-sem-style questions on one neuron that answers "which one": softmax and cross-entropy with the gradient, a logistic-regression update with two learning rates, two SGD steps with a trace table, confusion matrices for two and three classes, parameter counts and mini-batch arithmetic, one mini-batch step in matrix form, a Keras classifier, the softmax properties with numbers, and top- K accuracy.		
5	Module 5: Deep Feed-Forward Neural Networks	78
Ten mid-sem-style questions on networks with hidden layers: an XOR-like alarm that one neuron cannot learn, parameter counts, a full forward pass, backward pass and update on a 2-3-2 network, an architecture spec to examine, gradient flow through five layers, activation and loss derivatives, softmax with cross-entropy, and the deck's sizing rules.		
6	Module 6: Convolutional Neural Networks	98
Ten exam-style questions on the CNN deck: shapes through a whole pipeline, parameters per layer, convolution and pooling by hand, multi-channel filters, a Keras CNN snippet, receptive fields, padding-stride-dilation arithmetic, the 1×1 bottleneck, an engineer's flawed spec, and ResNet with transfer learning.		

How to use this set

Cover the solution. Read the question and the **What this tests** box. Write your own plan, then compare with **How to attempt**. Only then work through the **Solution**, one STEP at a time — every step is tagged with the part it answers. Check your final line against **Answers at a glance**, and read **Where marks are lost** before moving on. Questions marked **by hand** should be done with a pen; if you cannot finish one in about 12 minutes, go back to the companion book chapter named in the question header.

Module 1: Introduction to Deep Learning

Ten mid-sem-style questions: a concrete scenario with all its numbers, a numeric core to compute completely, and one or two short pointed sub-parts with a specific expected answer.

On your slides: **Module 1, slides 4–34** · depth · why now (scale, timeline) · applications table · data, model, objective, learning algorithm · wake-word example · kinds of learning · supervised tasks

Module 1 says what deep learning is, why it took off (data, compute, scale), what it is used for, and the four parts of every learning problem: **data, model, objective function, learning algorithm**. It ends with the kinds of learning and the supervised tasks. Each question below follows the mid-sem form: a scenario with its numbers in one place, parts i)–v) that build on each other, at least 60 % of the marks on exact computation, and the rest on pointed questions whose answer is a specific fact or number ("which is larger and by how many", "what does 0.73 mean", "is a better model feasible — use the baseline number").

Q	Scenario	Numeric core	Pointed sub-part	Slides	Marks
1.1	A 7-row property table	N, d , one-hot, 7×9 , 9 parameters, memory	why one-hot, not 1/2/3 — in terms of $\mathbf{w}^\top \mathbf{x}$	23–24	10
1.2	Three candidate price models	squared vs absolute loss, move the slope	which loss if $y = 12$ is a bad record; two stopping criteria	25–27	10
1.3	12,500 clips, 80/10/10, batch 64, 15 epochs	split sizes, iterations, updates, time	what an epoch is; epochs for 5,000 updates	28–30	10
1.4	Five applications from the slide table	input lengths, output units, parameters	interpret an output of 0.73	19–20	10
1.5	Three teams at 60/62/58 % on balanced data; scale table	baseline, gains per doubling, crossover, prediction	is a superior model feasible — with the baseline number; which architecture	9–10	10
1.6	$\hat{y} = w_1 x_1 + w_2 x_2 + b$ on 3 examples	forward pass, loss, gradient, one update	the same step with $\eta = 20$ — what goes wrong, with the number	22–27	10
1.7	Timeline with parameter counts	years, ratios, $\log_2$ doublings, memory	how many 80 GB GPUs hold GPT-3	11	8
1.8	Four datasets' stored labels	K , task, one-hot, 2^K , label-matrix shape	softmax or sigmoid for a multi-hot label — in terms of the sum	32–34	10
1.9	3,000 wake-word clips, 10 % positive	counts, accuracy, balancing, weighted accuracy	the majority-class baseline; which model wins under which metric	28–30	10
1.10	A Keras snippet: 784 → 128 → 64 → 10, softmax, 15 epochs, batch 64	parameters per layer, iterations, epochs, cross-entropy	problem type; softmax vs sigmoid; interpret the output vector	7, 25	10

Q1.1

A dataset table: N, d , one-hot, design matrix, parameters, memory

10 marks Slides 23–24 · Book 1, Ch. 4

A property site trains a linear model $\hat{y} = \mathbf{w}^\top \mathbf{x}$ to predict Price (lakh rupees) from this table.

ID	Bedrooms	Location	Type	Sq ft	Age (yr)	Price
1	2	Urban	Flat	900	4	62
2	3	Suburban	House	1400	12	75
3	1	Urban	Flat	600	2	48
4	4	Rural	House	2100	20	70
5	3	Urban	Flat	1250	8	96
6	2	Rural	House	1000	15	41
7	3	Suburban	Flat	1150	6	68

- Write N and d as the table stands, and name the label column. [1]
- Replace Location and Type by one-hot columns (order: Urban, Suburban, Rural; Flat, House). How many columns does each become, and what is the new d ? [2]
- A column of ones is added for the bias. Write the shape of the design matrix $\mathbf{X}$ and write row 3 as a vector (bias, Bedrooms, Sq ft, Age, then the one-hot columns). [3]
- How many parameters does the linear model have? How many bytes does $\mathbf{X}$ take at 4 bytes per entry, and how many MB for $N = 2,000,000$ rows? [2]
- Location could instead be coded as a single number Urban = 1, Suburban = 2, Rural = 3. Justify, in terms of $\hat{y} = \mathbf{w}^\top \mathbf{x}$, why one-hot coding is used instead. [2]

by hand

data → numbers

justify

🔗 WHAT THIS TESTS

Whether you can turn a raw table into the numbers a model actually sees — N, d , one-hot widths, the bias column, shapes and memory — and say what a single-number code would wrongly force on a linear model.

▶ HOW TO ATTEMPT

- Rows are examples; every column except ID and the label is a feature.
- A categorical column with k values becomes k one-hot columns; $\mathbf{X}$ is $N \times (d + 1)$ with the bias column.
- For v) write what $\mathbf{w}^\top \mathbf{x}$ does with a code of 1, 2, 3: one weight multiplies the code.

Σ SOLUTION

STEP 1 Rows 1 to 7 are the examples, so $N = 7$. Features: Bedrooms, Location, Type, Sq ft, Age, so $d = 5$. Price is the label t ; ID is only a name.

Data = $\{\mathbf{X}, \mathbf{t}\}$, slide 24

STEP 2 Location has 3 values → 3 columns: Urban = (1, 0, 0), Suburban = (0, 1, 0), Rural = (0, 0, 1). Type has 2 values → 2 columns: Flat = (1, 0), House = (0, 1).

STEP 3 Numeric columns kept: 3. New $d = 3 + 3 + 2 = 8$.

STEP 4 With the bias column $\mathbf{X}$ is $N \times (d + 1) = 7 \times 9$: 63 entries.

STEP 5 Row 3 is (1 bedroom, Urban, Flat, 600, 2): $\mathbf{x}^{(3)} = (1, 1, 600, 2, 1, 0, 0, 1, 0)$, length 9.

STEP 6 One weight per column of $\mathbf{X}$: $8 + 1 = 9$ parameters. Memory: $63 \times 4 = 252$ bytes.

iv)

STEP 7 $N = 2,000,000$: entries = $2,000,000 \times 9 = 18,000,000$; bytes = $18,000,000 \times 4 =$

iv)

$72,000,000 = 72$ MB.

STEP 8 With a single code $c \in \{1, 2, 3\}$ the model adds one term $w_{\text{loc}} c$: Rural contributes exactly 3 times what Urban does and Suburban sits exactly halfway – an order and equal spacing the data never stated. With three one-hot columns each location gets its own weight

v)

w_U, w_S, w_R , so the three effects are free.

one weight per category, no invented order

i) $N = 7, d = 5$, label = Price

ii) Location $\rightarrow$ 3 columns, Type $\rightarrow$ 2 columns, new $d = 8$

iii) $\mathbf{X}$ is 7×9 ; $\mathbf{x}^{(3)} = (1, 1, 600, 2, 1, 0, 0, 1, 0)$

iv) 9 parameters; 252 bytes; 72 MB

v) A code 1/2/3 forces Rural = $3 \times$ Urban through the single weight w_{loc} ; one-hot gives each location its own weight.

✓ ANSWERS AT A GLANCE

i) 7, 5, Price ii) 3, 2, $d = 8$ iii) 7×9 ; $(1, 1, 600, 2, 1, 0, 0, 1, 0)$ iv) 9; 252 B; 72 MB v) 1/2/3 imposes order and spacing

! WHERE MARKS ARE LOST

- Counting ID or Price as a feature (gives $d = 6$ or 7).
- Forgetting the bias column, so $\mathbf{X}$ comes out 7×8 and the model has 8 parameters.
- In v) saying "words are not numbers" – the point is the false order $1 < 2 < 3$ with equal gaps.

Q1.2

Three candidate models, two losses, and which way to move the slope

10 marks Slides 25–27 · Book 1, Ch. 5

A shop records size x (hundreds of sq ft) and monthly rent y (thousands of rupees) for four shops, and considers three models from the family $\hat{y} = wx + b$:

x	1	2	3	4
y	2	4	6	12

Model A: $\hat{y} = 2x$. Model B: $\hat{y} = 2.5x$. Model C: $\hat{y} = x + 4$.

Squared loss $J_2 = \frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$; absolute loss $J_1 = \frac{1}{N} \sum_i |\hat{y}^{(i)} - y^{(i)}|$.

- Tabulate the four predictions and errors $\hat{y} - y$ for each model. [2]
- Compute J_2 for A, B, C. [2]
- Compute J_1 for A, B, C. Which model is best under J_2 and which under J_1 ? [2]
- Keep $b = 0$ and compute J_2 at $w = 2.4$ and $w = 2.6$. Should the slope of model B be moved up or down? Estimate $\Delta J_2 / \Delta w$ from the two values. [2]

- v) The fourth record ($y = 12$) is later found to be a typing error. Which loss should be preferred while it remains in the data — justify in terms of how each loss weighs the error of -4 . Then state two criteria for stopping the repeated slope moves of iv). [2]

by hand

objective function

justify

WHAT THIS TESTS

Whether you can score a model family by a loss, see that two losses can pick different winners, use "lower is better" to choose a direction for a parameter, and name the stopping rules of that search.

HOW TO ATTEMPT

- Fill a table: one row per model, columns for $\hat{y}$, error, error², |error|; sum and divide by $N = 4$.
- For iv) redo the squared row for $w = 2.4$ and 2.6 ; the lower loss gives the direction; divide the change in loss by 0.2 .
- For v) compare $4^2 = 16$ with $|-4| = 4$.

SOLUTION

model	$\hat{y}$ at $x = 1, 2, 3, 4$	errors $\hat{y} - y$	squared	$\sum$	J_2	errors	$\sum$	J_1
A	2, 4, 6, 8	0, 0, 0, -4	0, 0, 0, 16	16	4.0	0, 0, 0, 4	4	1.0
B	2.5, 5, 7.5, 10	0.5, 1, 1.5, -2	0.25, 1, 2.25, 4	7.5	1.875	0.5, 1, 1.5, 2	5	1.25
C	5, 6, 7, 8	3, 2, 1, -4	9, 4, 1, 16	30	7.5	3, 2, 1, 4	10	2.5

STEP 1 A: $\hat{y} = 2, 4, 6, 8$; errors 0, 0, 0, -4 . B: $\hat{y} = 2.5, 5, 7.5, 10$; errors 0.5, 1, 1.5, -2 . C: $\hat{y} = 5, 6, 7, 8$; errors 3, 2, 1, -4 .

STEP 2 A: $J_2 = \frac{1}{4}(0 + 0 + 0 + 16) = 4.0$. B: $J_2 = \frac{1}{4}(0.25 + 1 + 2.25 + 4) = \frac{7.5}{4} = 1.875$. C: $J_2 = \frac{1}{4}(9 + 4 + 1 + 16) = 7.5$.

STEP 3 A: $J_1 = \frac{4}{4} = 1.0$; B: $J_1 = \frac{5}{4} = 1.25$; C: $J_1 = \frac{10}{4} = 2.5$.

STEP 4 Lowest J_2 is B ($1.875 < 4.0 < 7.5$). Lowest J_1 is A ($1.0 < 1.25 < 2.5$).

STEP 5 $w = 2.4$: $\hat{y} = 2.4, 4.8, 7.2, 9.6$; errors 0.4, 0.8, 1.2, -2.4 ; squares 0.16, 0.64, 1.44, 5.76; $J_2 = \frac{8.0}{4} = 2.0$.

STEP 6 $w = 2.6$: $\hat{y} = 2.6, 5.2, 7.8, 10.4$; errors 0.6, 1.2, 1.8, -1.6 ; squares 0.36, 1.44, 3.24, 2.56; $J_2 = \frac{7.6}{4} = 1.9$.

STEP 7 Loss falls from 2.0 to 1.9 as w rises, so move the slope **up**, but by less than 0.1 (both 2.0 and 1.9 are above 1.875). $\frac{\Delta J_2}{\Delta w} = \frac{1.9 - 2.0}{0.2} = -0.5$.
the best w is $\sum xy / \sum x^2 = 76/30 = 2.533$

STEP 8 Prefer J_1 . The bad record's error of -4 costs $|-4| = 4$ under J_1 but $4^2 = 16$ under J_2 — four times as much — so J_2 lets one wrong record pull the winner from A to B.

STEP 9 Stopping criteria: (1) the change in loss between moves is tiny, $|J^{(t)} - J^{(t-1)}| < \epsilon$; (2) the slope of the loss is tiny, $|\Delta J / \Delta w| < \epsilon$ (gradient near zero); also a maximum number of moves.

i) A: 2, 4, 6, 8 (0, 0, 0, -4); B: 2.5, 5, 7.5, 10 (0.5, 1, 1.5, -2); C: 5, 6, 7, 8 (3, 2, 1, -4)

ii) J_2 : A = 4.0, B = 1.875, C = 7.5

iii) J_1 : A = 1.0, B = 1.25, C = 2.5; best under J_2 is B, under J_1 is A

iv) $J_2(2.4) = 2.0, J_2(2.6) = 1.9$; move up; $\Delta J_2 / \Delta w = -0.5$

v) J_1 : it charges 4, not 16, for the -4 error. Stop when $|J^{(t)} - J^{(t-1)}| < \varepsilon$ or $|\Delta J / \Delta w| < \varepsilon$ (or max iterations).

✓ ANSWERS AT A GLANCE

ii) 4.0, 1.875, 7.5 iii) 1.0, 1.25, 2.5; B / A iv) 2.0, 1.9, up, -0.5 v) J_1 (4 vs 16); ΔJ small, gradient small

! WHERE MARKS ARE LOST

- Forgetting to divide by $N = 4$ (writing 16, 7.5, 30 as J_2).
- Sign slips in $\hat{y} - y$ – harmless for the loss, fatal for $\Delta J_2 / \Delta w$.
- In v) naming only "run for 100 steps" – the two loss-based criteria are what is asked.

Q1.3

Split, batches per epoch, total updates, training time

10 marks Slides 28–30 · Book 1, Ch. 6

Asha has 12,500 labelled audio clips for a wake-word detector. She splits them 80 % / 10 % / 10 % into training, validation and test sets and trains with mini-batches of 64 clips for 15 epochs. One parameter update happens per mini-batch, every training clip is used once per epoch, and the last (smaller) batch is still used. One update takes 40 ms.

- i) How many clips are in each set? Which is larger, the training set or the validation set, and by how many clips? [2]
- ii) How many updates (iterations) per epoch, and how many clips in the last batch? [2]
- iii) How many updates in 15 epochs, and how many clip-presentations does the model see? [1]
- iv) How many seconds does training take? With batch size 128 each update takes 60 ms: repeat ii)–iv) and give the speed-up factor. [3]
- v) What does one epoch represent in this run (state the number of clips and of updates it contains)? How many epochs are needed to make at least 5,000 updates at batch size 64? [2]

by hand

train the model

epoch

🎯 WHAT THIS TESTS

Whether you can do the bookkeeping of a training run – split sizes, the ceiling in "iterations per epoch", updates = iterations $\times$ epochs, time = updates $\times$ time per update – and say exactly what an epoch is.

▶ HOW TO ATTEMPT

- Split sizes are percentages of 12,500.
- Iterations per epoch = $\lceil N_{\text{train}} / B \rceil$; last batch = $N_{\text{train}} - (\text{iterations} - 1) \times B$.
- Epochs for a target number of updates = $\lceil \text{updates} / \text{iterations per epoch} \rceil$.

Σ SOLUTION

STEP 1 Training = $0.8 \times 12,500 = 10,000$; validation = $0.1 \times 12,500 = 1,250$; test = 1,250. Check:

i) $10,000 + 1,250 + 1,250 = 12,500$.

STEP 2 Training is larger by $10,000 - 1,250 = 8,750$ clips.

i)

- STEP 3** $10,000/64 = 156.25 \Rightarrow \lceil 156.25 \rceil = 157$ iterations: 156 full batches and one short one of
 ii) $10,000 - 156 \times 64 = 10,000 - 9,984 = 16$ clips.
- STEP 4** Updates = $157 \times 15 = 2,355$. Clip-presentations = $10,000 \times 15 = 150,000$.
 iii)
- STEP 5** Time = $2,355 \times 40 \text{ ms} = 94,200 \text{ ms} = 94.2 \text{ s}$.
 iv)
- STEP 6** Batch 128: $10,000/128 = 78.125 \Rightarrow 79$ iterations; last batch = $10,000 - 78 \times 128 = 16$;
 iv) updates = $79 \times 15 = 1,185$; time = $1,185 \times 60 \text{ ms} = 71.1 \text{ s}$.
- STEP 7** Speed-up = $94.2/71.1 = 1.325$.
 iv)
- STEP 8** One epoch = one full pass over the training set: all 10,000 clips seen once, in 157 mini-batches, so 157 updates.
 v)
- STEP 9** Epochs for 5,000 updates: $5,000/157 = 31.85 \Rightarrow 32$ epochs (31 epochs give only $31 \times 157 = 4,867$; 32 give 5,024).
 v)

i) 10,000 / 1,250 / 1,250; training larger by 8,750

ii) 157 iterations; last batch 16 clips

iii) 2,355 updates; 150,000 presentations

iv) 94.2 s; batch 128: 79 iterations, last batch 16, 1,185 updates, 71.1 s; speed-up 1.325×

v) one pass over all 10,000 training clips = 157 updates; 32 epochs for 5,000 updates

✓ ANSWERS AT A GLANCE

i)	10,000 / 1,250 /	ii)	157;	iii)	2,355;	iv)	94.2 s; 79, 16, 1,185,	v)	10,000 clips / 157
	1,250; +8,750		16		150,000		71.1 s; 1.325×		updates; 32

! WHERE MARKS ARE LOST

- Rounding 156.25 down to 156 — the 16 leftover clips still need an update.
- Using 12,500 instead of 10,000 as the training size.
- Answering 31.85 or 31 epochs in v): epochs are whole, and 31 falls short of 5,000.

Q1.4

The applications table as input lengths and output units

10 marks Slides 19–20 · Book 1, Ch. 3

Your slide lists applications as input → output. A network only sees a vector of numbers in and gives a vector of numbers out. Five systems are specified below.

system	input	output
Photo tagging	colour image 64×64 pixels, 3 channels	which of 20 tags apply (any number may)
Wake word	2-second clip at 16,000 samples/s; then cut into 20 ms frames, 13 numbers per frame	probability that the wake word is present
Real estate	12 numeric features	the price
Object detection	(an image)	up to $M = 5$ boxes, 4 numbers each; then also a score for each of 10 classes per box
Translation	40 English words, each one-hot over 8,000 words	one French word at a time from 12,000 words

- i) Photo tagging: input length, output units, and the number of parameters (weights + biases) of a linear model from input to output. [2]
- ii) Wake word: number of raw samples; number of frames and the length of the summarised input; output units. [2]
- iii) Real estate: input length, output units, weights of a linear model including the bias. [1]
- iv) Object detection: output units for boxes only, then with class scores. Translation: number of input numbers and output units per step. [3]
- v) The wake-word output is 0.73 for a clip. State what the number means and the decision at threshold 0.5. For a photo whose true tags are "beach, sea, sky", how many of the 20 tagging outputs should be near 1? [2]

by hand

applications

interpret an output

🎯 WHAT THIS TESTS

Whether you can convert "image", "audio", "text" into a vector length and "tags", "price", "boxes", "next word" into a number of output units — and read a single output number the right way.

▶ HOW TO ATTEMPT

- Input length = product of the dimensions (height $\times$ width $\times$ channels; seconds $\times$ samples per second; words $\times$ vocabulary).
- Output units = one per number the answer needs; linear model parameters = inputs $\times$ outputs + outputs.
- A single output between 0 and 1 is a probability; compare it with the threshold.

Σ SOLUTION

STEP 1 Input length = $64 \times 64 \times 3 = 4,096 \times 3 = 12,288$. Output: one unit per tag = 20.

i)

STEP 2 Parameters: weights = $12,288 \times 20 = 245,760$; biases = 20; total = 245,780.

i)

STEP 3 Raw samples = $2 \times 16,000 = 32,000$. Frames = $2,000 \text{ ms} / 20 \text{ ms} = 100$; summarised input = $100 \times 13 = 1,300$. Output: 1 unit.

ii)

STEP 4 Input 12; output 1; weights = $12 + 1 = 13$.

iii)

STEP 5 Boxes: $4 \times 5 = 20$ units. With 10 class scores per box: $5 \times (4 + 10) = 70$.

iv)

STEP 6 Translation input = $40 \times 8,000 = 320,000$ numbers; output per step = 12,000 units (one per French word).

iv)

STEP 7 0.73 is the model's probability that the wake word is present (so 0.27 that it is absent).

v) $0.73 > 0.5 \rightarrow$ decision "yes, wake word".

STEP 8 Three tags are true, so 3 of the 20 outputs should be near 1 and the other 17 near 0 — this is why the outputs are independent yes/no units, not a single choice.

i) 12,288 inputs; 20 outputs; 245,780 parameters

ii) 32,000 samples; 100 frames; 1,300 inputs; 1 output unit

iii) 12; 1; 13

iv) 20 units, then 70; 320,000 input numbers; 12,000 output units

v) $0.73 =$ probability of the wake word; $0.73 > 0.5 \rightarrow$ yes; 3 outputs near 1

ANSWERS AT A GLANCE

i) 12,288; 20; 245,780 ii) 32,000; 100; 1,300; 1 iii) 12; 1; 13 iv) 20; 70; 320,000; 12,000 v) $P(\text{wake}) = 0.73 \rightarrow$ yes; 3

WHERE MARKS ARE LOST

- Forgetting the 3 colour channels (4,096 instead of 12,288) or the 20 biases.
- Mixing seconds and milliseconds: 2 s is 2,000 ms, so 100 frames, not 10.
- Reading 0.73 as "73 % of the clip contains the word" — it is a probability of the class.

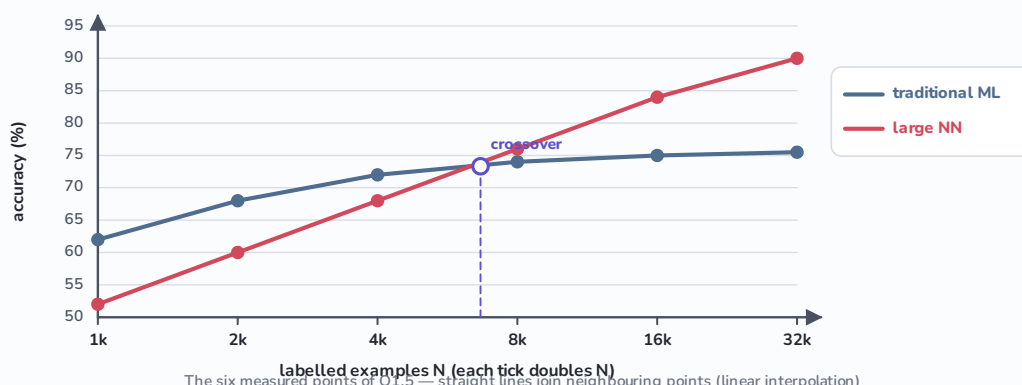
Q1.5

Three modest baselines, and "scale drives progress" as a table

10 marks Slides 9–10 · Book 1, Ch. 2

A healthcare company classifies patient records as Positive / Negative. Its dataset is balanced: 2,000 records, 1,000 of each class. Three teams report test accuracy: Naïve Bayes 60 %, Random Forest 62 %, SVM 58 %. A fourth team then measures a large neural network and the best traditional method as the amount of labelled data doubles:

N (thousands)	1	2	4	8	16	32
traditional ML	62	68	72	74	75	75.5
large NN	52	60	68	76	84	90



- i) Compute the majority-class baseline accuracy on the balanced set. Is it feasible to build a model that outperforms all three teams? Justify with the baseline number and the best team's margin over it. [2]

- ii) From the table, list the gain per doubling of N for each method (five numbers each). [2]
- iii) By linear interpolation in N , find the crossover data size at which the two methods tie, and the accuracy there. [3]
- iv) Assume the NN keeps its last gain per doubling and the traditional method stays at 75.5. Predict both at $N = 64,000$ and the gap; how many doublings from 32,000 until the NN first reaches 99 % or more? [2]
- v) Recommend which of the two methods the company should build at $N = 3,000$ and at $N = 32,000$. Justify each choice with the interpolated or tabulated accuracies. [1]

by hand

why now

justify with the baseline

Ⓢ WHAT THIS TESTS

Whether you can judge "modest accuracy" against the chance baseline, and read the scale story off numbers: the traditional method's gain per doubling shrinks to nothing while the network's does not, so there is a crossover size above which the network wins.

▶ HOW TO ATTEMPT

- Majority-class baseline = largest class ÷ total. Compare each team with it.
- Gains are differences between neighbouring columns; work with the gap NN – traditional and interpolate it to zero.
- For v) interpolate at 3,000 (halfway between the 2k and 4k columns) and read 32k off the table.

Σ SOLUTION

doubling	1 → 2	2 → 4	4 → 8	8 → 16	16 → 32
traditional gain	6	4	2	1	0.5
NN gain	8	8	8	8	6
gap NN – trad.	-10, -8, -4, +2, +9, +14.5 (at 1, 2, 4, 8, 16, 32 k)				

STEP 1 Majority-class baseline = $1,000/2,000 = 50\%$ (always answer the bigger class; here both are equal).

STEP 2 The best team is only $62 - 50 = 12$ points above chance (SVM only 8). A ceiling of 100 leaves 38 points of room, and the table shows the same kind of task reaching 90 % – so yes, a superior model is feasible; the modest scores point to a data or feature limit, not a task limit.

STEP 3 Traditional: $68 - 62, 72 - 68, 74 - 72, 75 - 74, 75.5 - 75 = 6, 4, 2, 1, 0.5$ – halving each time. NN: 8, 8, 8, 8, 6.

STEP 4 Gap NN – traditional at 4k: $68 - 72 = -4$; at 8k: $76 - 74 = +2$. It changes sign between 4k and 8k.

STEP 5 The gap moves by $2 - (-4) = 6$ over 4k, reaching 0 after $\frac{4}{6}$ of the way: $N^* = 4 + 4 \times \frac{4}{6} = 6.667k \approx 6,667$ records.

STEP 6 Accuracy there: traditional = $72 + \frac{2}{3} \times 2 = 73.33$; NN = $68 + \frac{2}{3} \times 8 = 73.33$. Equal, as it must be.

STEP 7 NN at 64k = $90 + 6 = 96$; traditional = 75.5; gap = 20.5 points. For 99: need 9 points at 6 per doubling, $9/6 = 1.5 \Rightarrow 2$ doublings, i.e. $N = 128,000$.

STEP 8 At 3,000 (halfway between 2k and 4k): traditional = $68 + 0.5 \times 4 = 70$, NN = $60 + 0.5 \times 8 = 64 \rightarrow$ build the traditional model (better by 6 points). At 32,000: NN 90 vs 75.5 $\rightarrow$ build

the large NN (better by 14.5 points).

i) baseline 50 %; feasible — best team is only 12 points above chance, and the table reaches 90 %

ii) traditional 6, 4, 2, 1, 0.5; NN 8, 8, 8, 8, 6

iii) $N^* \approx 6,667$; 73.33 %

iv) 96 vs 75.5, gap 20.5; 2 doublings → 128,000

v) 3,000: traditional (70 vs 64); 32,000: large NN (90 vs 75.5)

✓ ANSWERS AT A GLANCE

i) 50 %; yes, +12 only ii) 6,4,2,1,0.5 / 8,8,8,8,6 iii) 6,667; 73.33 iv) 96, 75.5, 20.5; 2 → 128,000 v) trad. at 3k; NN at 32k

! WHERE MARKS ARE LOST

- Answering "feasible" without the 50 % baseline and the 12-point margin.
- Using the 4k → 8k fraction the wrong way round ($\frac{2}{6}$ instead of $\frac{4}{6}$), giving 5,333.
- Recommending the NN at 3,000 — at that size it is 6 points worse.

Q1.6

The four components with numbers: predict, score, improve — and a learning rate too large

10 marks Slides 22–27 · Book 1, Ch. 4–6

A model predicts a shop's daily profit (thousand rupees) from footfall x_1 (hundreds) and staff x_2 .

Data: three examples below. **Model:** $\hat{y} = w_1x_1 + w_2x_2 + b$ with $w_1 = 0.5$, $w_2 = -1.0$, $b = 2.0$.

Objective: $J = \frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$. **Learning algorithm:** gradient descent on b , $b \leftarrow b - \eta \partial J / \partial b$.

example	x_1	x_2	y
1	2	1	2.6
2	4	3	0.4
3	1	2	0.2

- i) Write N , d , the number of parameters, and compute the three predictions. [2]
- ii) Compute the three squared errors and J . [2]
- iii) Write $\partial J / \partial b$ as a formula and evaluate it. [2]
- iv) Perform one update with $\eta = 0.5$: give the new b and the new J . Did the loss fall? [2]
- v) Repeat the update from $b = 2.0$ with $\eta = 20$: give the new b and the new J . Which learning rate gives the larger change in b , and what problem does the large one show — state it with the two J values? [2]

by hand

four components

large learning rate

🎯 WHAT THIS TESTS

Whether you can run the whole loop once by hand — data in, model predicts, objective scores, algorithm moves a parameter — and show with numbers what a too-large step does to the loss.

▶ HOW TO ATTEMPT

- Substitute each row into $0.5x_1 - 1.0x_2 + 2.0$; error = $\hat{y} - y$; square; average over $N = 3$.
- $\partial J / \partial b = \frac{2}{N} \sum (\hat{y} - y)$ because $\partial \hat{y} / \partial b = 1$.
- Every prediction shifts by the same amount as b , so recomputing J after an update is quick.

Σ SOLUTION

ex.	$\hat{y}$ ($b = 2.0$)	$\hat{y} - y$	sq.	$\hat{y}$ ($b = 1.9$)	$\hat{y} - y$	sq.	$\hat{y}$ ($b = -2.0$)	$\hat{y} - y$	sq.
1	2.0	-0.6	0.36	1.9	-0.7	0.49	-2.0	-4.6	21.16
2	1.0	0.6	0.36	0.9	0.5	0.25	-3.0	-3.4	11.56
3	0.5	0.3	0.09	0.4	0.2	0.04	-3.5	-3.7	13.69
sum		0.3	0.81		0.0	0.78		-11.7	46.41

STEP 1 $N = 3, d = 2$, parameters = $2 + 1 = 3$.

i)

STEP 2 $\hat{y}^{(1)} = 0.5 \times 2 - 1.0 \times 1 + 2.0 = 2.0$; $\hat{y}^{(2)} = 0.5 \times 4 - 1.0 \times 3 + 2.0 = 1.0$; $\hat{y}^{(3)} = 0.5 \times 1 -$

ii)

$1.0 \times 2 + 2.0 = 0.5$.

STEP 3 Errors $2.0 - 2.6, 1.0 - 0.4, 0.5 - 0.2 = -0.6, 0.6, 0.3$; squares $0.36, 0.36, 0.09$; $J = \frac{0.81}{3} =$

iii)

0.27 .

STEP 4 $\frac{\partial J}{\partial b} = \frac{2}{N} \sum_i (\hat{y}^{(i)} - y^{(i)}) = \frac{2}{3} (-0.6 + 0.6 + 0.3) = \frac{2}{3} \times 0.3 = 0.2$.

iv)

$\partial \hat{y} / \partial b = 1$

STEP 5 $b \leftarrow 2.0 - 0.5 \times 0.2 = 1.9$. Predictions all drop by 0.1: 1.9, 0.9, 0.4; errors $-0.7, 0.5, 0.2$;

v)

squares $0.49, 0.25, 0.04$; $J = \frac{0.78}{3} = 0.26$. Yes: $0.26 < 0.27$.

STEP 6 $b \leftarrow 2.0 - 20 \times 0.2 = 2.0 - 4.0 = -2.0$. Predictions drop by 4: $-2.0, -3.0, -3.5$; errors

v)

$-4.6, -3.4, -3.7$; squares $21.16, 11.56, 13.69$; $J = \frac{46.41}{3} = 15.47$.

STEP 7 $\eta = 20$ changes b by 4.0 against 0.1 for $\eta = 0.5$ — 40 times larger. It overshoots the

v)

minimum: the loss goes **up** from 0.27 to 15.47 (57 times worse) instead of down, and repeated steps of this size diverge.

i) $N = 3, d = 2, 3$ parameters; $\hat{y} = 2.0, 1.0, 0.5$

ii) $0.36, 0.36, 0.09$; $J = 0.27$

iii) $\partial J / \partial b = \frac{2}{N} \sum (\hat{y} - y) = 0.2$

iv) $b = 1.9$; $J = 0.26$; yes, fell

v) $b = -2.0$; $J = 15.47$; $\eta = 20$ moves b by 4.0 vs 0.1 — overshoot, loss rises $0.27 \rightarrow 15.47$ (divergence)

✓ ANSWERS AT A GLANCE

i) 3, 2, 3; 2.0, 1.0, 0.5 ii) 0.27 iii) 0.2 iv) 1.9; 0.26 v) -2.0; 15.47; overshoot

! WHERE MARKS ARE LOST

- Dropping the factor 2 in $\partial J / \partial b$ (gives 0.1 and a step to 1.95).
- Writing $y - \hat{y}$ in the gradient — the step then goes to 2.1 and the loss rises.
- In v) saying "too fast" without the numbers $0.27 \rightarrow 15.47$.

Q1.7

Timeline arithmetic: years, parameter ratios, doublings, memory

8 marks Slide 11 · Book 1, Ch. 2

Milestones from the timeline, with the parameter count of the famous network of each year:

year	milestone	parameters
1958	Perceptron (Rosenblatt)	—
1986	Backpropagation popularised	—
1998	LeNet-5	60 thousand (60 K)
2012	AlexNet	60 million (60 M)
2014	VGG-16	138 million (138 M)
2020	GPT-3	175 billion (175,000 M)

- Give the years between consecutive milestones 1958, 1986, 1998, 2012, 2020. [1]
- Compute the ratios AlexNet : LeNet-5, VGG-16 : AlexNet, GPT-3 : VGG-16. [2]
- How many doublings take 60 K to 60 M, and 60 M to 175 B? ($\log_2 1000 = 9.97$, $\log_2 2916.7 = 11.51$.) [2]
- Give the average time per doubling over 1998–2012 and over 2012–2020, in years. Which period doubled faster, and by what factor? [2]
- At 4 bytes per parameter, how much memory do LeNet-5's and GPT-3's parameters take? How many 80 GB GPUs are needed just to hold GPT-3's parameters? [1]

by hand

timeline

ratios and $\log_2$

🔗 WHAT THIS TESTS

Whether you can put numbers on "scale": model size grew a thousandfold and then almost three-thousandfold, with the doubling time shrinking below a year, and memory that no single GPU holds.

▶ HOW TO ATTEMPT

- Subtract years; divide parameter counts after matching units (K, M, B).
- Doublings = $\log_2(\text{ratio})$; time per doubling = years $\div$ doublings.
- Bytes = parameters $\times$ 4; GPUs = ceiling of GB $\div$ 80.

Σ SOLUTION

STEP 1 1986 – 1958 = 28; 1998 – 1986 = 12; 2012 – 1998 = 14; 2020 – 2012 = 8 years.

i)

STEP 2 AlexNet : LeNet = $60,000,000/60,000 = 1,000$. VGG-16 : AlexNet = $138/60 = 2.3$. GPT-3 :

ii)

VGG-16 = $175,000/138 = 1,268.1$.

STEP 3 60 K $\rightarrow$ 60 M: ratio 1,000, $\log_2 1000 = 9.97 \approx 10$ doublings ($2^{10} = 1024$). 60 M $\rightarrow$ 175 B: ratio

iii)

$175,000/60 = 2,916.7$, $\log_2 2916.7 = 11.51$ doublings.

STEP 4 1998–2012: $14/9.97 = 1.40$ years per doubling. 2012–2020: $8/11.51 = 0.695$ years (about 8

iv)

months). The second period doubled faster, by $1.40/0.695 = 2.0$ times.

STEP 5 LeNet-5: $60,000 \times 4 = 240,000$ bytes = 240 KB. GPT-3: $175 \times 10^9 \times 4 = 700 \times 10^9$ bytes =

v)

700 GB. GPUs: $700/80 = 8.75 \Rightarrow 9$ GPUs.

i) 28, 12, 14, 8 years

ii) 1,000; 2.3; 1,268.1

iii) 9.97 (≈ 10) doublings; 11.51 doublings

iv) 1.40 years; 0.695 years; 2012–2020 faster by 2.0×

v) 240 KB; 700 GB; 9 GPUs

✓ ANSWERS AT A GLANCE

i) 28, 12, 14, 8 ii) 1,000; 2.3; 1,268.1 iii) 9.97; 11.51 iv) 1.40 y; 0.695 y; 2.0× v) 240 KB; 700 GB; 9

! WHERE MARKS ARE LOST

- Mixing units: 175 B ÷ 138 M is 1,268, not 1.27.
- Using $\log_{10}$ (3 and 3.46) instead of $\log_2$ for doublings.
- Answering 8.75 GPUs — hardware comes in whole units, so 9.

Q1.8

Decode the label: K , multi-class or multi-label, 2^K , and the matching output

10 marks Slides 32–34 · Book 1, Ch. 7

Four supervised datasets store their labels as follows. One example label from each:

dataset	one stored label t	note
(i)	(0, 0, 1, 0, 0)	every label has exactly one 1
(ii)	(1, 0, 1, 1, 0, 0)	labels may have any number of 1s
(iii)	47.5	a real number
(iv)	3	an integer from $\{0, 1, \dots, 9\}$

- i) For each dataset give K (number of classes, or 1 for a real value) and the task — regression, multi-class or multi-label — deciding (i) vs (ii) by counting the ones. [3]
- ii) Write the label of (iv) as a one-hot vector and give its length. [1]
- iii) For dataset (ii): how many different label vectors are possible, and how many have exactly 3 ones? [2]
- iv) Dataset (ii) has $N = 500$ examples with 2.4 ones per label on average. Give the shape of the label matrix $\mathbf{T}$, its number of entries and its number of ones. Give the output-layer size for each dataset and their sum. [2]
- v) For dataset (ii) the network's third output is 0.73. Is tag 3 assigned at threshold 0.5? Should the output layer use softmax or one sigmoid per unit — justify in terms of the sum of the outputs against the sum of the stored label. [2]

by hand

supervised tasks

justify

🎯 WHAT THIS TESTS

Whether you can tell the supervised tasks apart from the shape of the label alone, count what follows from K , and match the output activation to the label's shape.

▶ HOW TO ATTEMPT

- K = length of the vector or size of the integer set. One 1 $\rightarrow$ multi-class; several 1s allowed $\rightarrow$ multi-label; a real number $\rightarrow$ regression.
- Multi-label with K tags has 2^K label vectors; exactly m ones gives $\binom{K}{m}$.
- Softmax outputs sum to 1; a multi-hot label sums to the number of tags.

Σ SOLUTION

STEP 1 (i): length 5, one 1 $\rightarrow K = 5$, multi-class. (ii): length 6, three 1s $\rightarrow K = 6$, multi-label. (iii): real $\rightarrow K = 1$, regression. (iv): one of 10 integers $\rightarrow K = 10$, multi-class.

STEP 2 Class 3 of $\{0, \dots, 9\}$: $(0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$, length 10 — the 1 is in position 4 because counting starts at 0.

STEP 3 $2^6 = 64$ label vectors. Exactly 3 ones: $\binom{6}{3} = \frac{6 \times 5 \times 4}{3 \times 2 \times 1} = 20$.

STEP 4 $\mathbf{T}$ is 500×6 ; entries = 3,000; ones = $500 \times 2.4 = 1,200$ (1,800 zeros).

STEP 5 Output units: (i) 5, (ii) 6, (iii) 1, (iv) 10; sum = 22.

STEP 6 $0.73 > 0.5 \rightarrow$ tag 3 is assigned. The stored label $(1, 0, 1, 1, 0, 0)$ sums to 3, but softmax outputs always sum to 1, so softmax cannot match it; one sigmoid per unit gives six independent probabilities whose sum can be 3.

i) (i) $K = 5$ multi-class; (ii) $K = 6$ multi-label; (iii) $K = 1$ regression; (iv) $K = 10$ multi-class

ii) $(0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$, length 10

iii) 64; 20

iv) 500×6 ; 3,000 entries; 1,200 ones; units 5, 6, 1, 10 $\rightarrow 22$

v) yes ($0.73 > 0.5$); sigmoid per unit — label sums to 3, softmax sums to 1

✓ ANSWERS AT A GLANCE

i) 5 MC; 6 ML; 1 reg.; 10 MC ii) $(0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$; 10 iii) 64; 20 iv) 500×6 ; 3,000; 1,200; 22 v) yes; sigmoid (3 vs 1)

! WHERE MARKS ARE LOST

- Putting the 1 in position 3 instead of 4 for class 3 (classes start at 0).
- Answering 6 or 64 for "exactly 3 ones" — that is $\binom{6}{3} = 20$.
- Choosing softmax "because there are 6 classes" — the sum-to-1 constraint is the point.

Q1.9

Wake-word data: imbalance, the majority baseline, balancing, class weights

10 marks Slides 28–30 · Book 1, Ch. 6

Asha collects 3,000 one-second clips at 16,000 samples per second for the wake word "Nova". Only 10 % contain the word (label yes). Model B catches 240 of the yes clips and wrongly says yes on 270 of the no clips.

- i) How many yes and no clips? Give the shape of the data matrix $\mathbf{X}$ (raw samples as features) and its size in MB at 4 bytes per number. [2]
- ii) State the majority-class baseline: the accuracy of a model that always answers "no", and the number of wake words it catches. [1]
- iii) For a 50/50 balance, how many more yes clips must be recorded (keeping all no clips)? Alternatively, if no clips are discarded instead, how many clips remain? [2]
- iv) Instead each yes clip is weighted $w_+ = \#no / \#yes$ and each no clip 1. Give w_+ , the total weight, and the **weighted accuracy** (weight of correct clips $\div$ total weight) of the always-"no" model. [2]
- v) Compute model B's plain accuracy and weighted accuracy. Which model wins under each metric, and by how many points? [3]

by hand

data and objective

class imbalance

🔗 WHAT THIS TESTS

Whether you can see, with numbers, why the objective must match the task: plain accuracy rewards a model that never fires, and class weights fix that.

▶ HOW TO ATTEMPT

- Counts first: 10 % of 3,000. $\mathbf{X}$ is clips $\times$ samples.
- Accuracy = correct $\div$ total; the always-"no" model is correct on every no clip only.
- Lay out TP, FN, FP, TN for model B; weighted accuracy multiplies each correct clip by its class weight and divides by the total weight.

Σ SOLUTION

model	TP	FN	FP	TN	correct	weighted correct
always "no"	0	300	0	2,700	2,700	$0 \times 9 + 2,700 = 2,700$
B	240	60	270	2,430	2,670	$240 \times 9 + 2,430 = 4,590$

STEP 1 Yes = $0.1 \times 3,000 = 300$; no = 2,700. $\mathbf{X}$ is $3,000 \times 16,000$: 48,000,000 numbers $\times 4 = 192,000,000$ bytes = 192 MB.

i)

STEP 2 Always "no" is right on all 2,700 no clips: accuracy = $2,700/3,000 = 90\%$. Wake words caught: 0. This 90 % is the baseline any useful model must beat.

ii)

STEP 3 50/50 with 2,700 no clips needs 2,700 yes clips: record $2,700 - 300 = 2,400$ more (5,400 total). Discarding no clips down to 300 leaves $300 + 300 = 600$.

iii)

STEP 4 $w_+ = 2,700/300 = 9$. Total weight = $300 \times 9 + 2,700 = 5,400$. Always "no": $2,700/5,400 = 50\%$.

iv)

both classes now carry equal weight

STEP 5 Model B: TP = 240, FN = 60, FP = 270, TN = $2,700 - 270 = 2,430$. Plain accuracy = $(240 + 2,430)/3,000 = 2,670/3,000 = 89\%$.

v)

STEP 6 Weighted accuracy = $(240 \times 9 + 2,430)/5,400 = (2,160 + 2,430)/5,400 = 4,590/5,400 = 85\%$.

v)

STEP 7 Plain: always-"no" wins by $90 - 89 = 1$ point. Weighted: B wins by $85 - 50 = 35$ points.

v)

The metric decides which model looks better.

i) 300 yes, 2,700 no; $\mathbf{X}$ is $3,000 \times 16,000$; 192 MB

ii) baseline 90 %; catches 0

iii) record 2,400 more; or keep 600 clips

iv) $w_+ = 9$; total weight 5,400; 50 %

v) B: 89 % plain, 85 % weighted; plain $\rightarrow$ always-"no" by 1 point; weighted $\rightarrow$ B by 35 points

✓ ANSWERS AT A GLANCE

i) 300 / 2,700; 3000 $\times$ 16000; 192 MB ii) 90 %; 0 iii) 2,400; 600 iv) 9; 5,400; 50 % v) 89 %, 85 %; +1 / +35

! WHERE MARKS ARE LOST

- Answering 300 in iii): the question asks how many *more*.
- Dividing weighted correct by 3,000 instead of the total weight 5,400.
- Forgetting $TN = 2,700 - 270$ for model B.

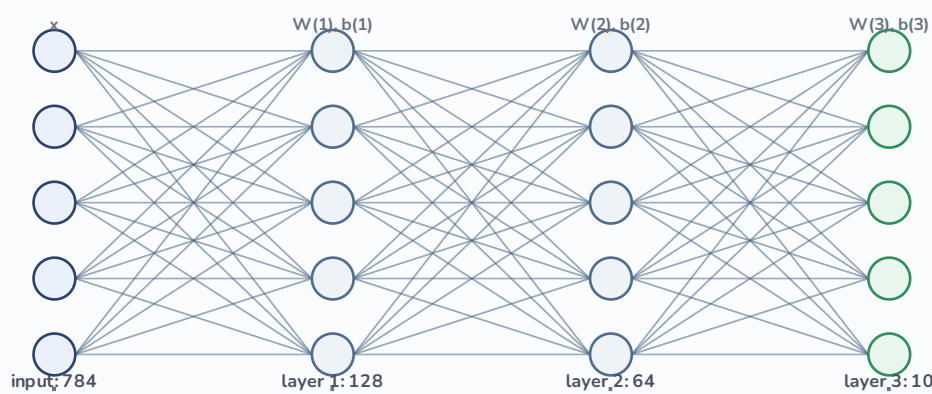
Q1.10

Read a Keras snippet: architecture, parameters, epochs, and the output vector

10 marks Slides 7, 25 · Book 1, Ch. 1, 5

A digit classifier is trained on 10,000 flattened 28×28 grey images with this code:

```
model = Sequential()
model.add(Dense(128, input_dim=784, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='SGD', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=15, batch_size=64)
```



- Give the depth L (layers with weights) and the units per layer. State the nature of the problem (binary / multi-class / regression) and K . [1]
- Calculate the trainable parameters (weights + biases) of each layer and the total. What percentage sits in layer 1? [3]
- Why softmax in the output layer rather than one sigmoid per unit – in terms of the sum of the outputs? Under what change of label would sigmoid be used instead? [1]
- What does one epoch represent here? How many updates per epoch, how many in the 15 epochs, and how many epochs would be needed for at least 5,000 updates? [2]

- v) For one image the output is (0.02, 0.01, 0.05, 0.03, 0.04, 0.01, 0.02, 0.73, 0.05, 0.04). Check the sum, give the predicted class and its confidence, and compute the cross-entropy loss $-\ln p_{\text{true}}$ if the true digit is 7, and if it is 2 ($\ln 0.73 = -0.315$, $\ln 0.05 = -2.996$). [3]

by hand

read the code

interpret the output

🔗 WHAT THIS TESTS

Whether you can read a model definition as numbers — layers, parameters, epochs, updates — and read its output vector as a probability distribution with a loss.

▶ HOW TO ATTEMPT

- Weights of a Dense layer = (units in) $\times$ (units out); biases = units out.
- Updates per epoch = $\lceil 10,000/64 \rceil$; epochs for a target = $\lceil \text{target}/\text{per epoch} \rceil$.
- The predicted class is the largest output; cross-entropy uses only the true class's probability.

Σ SOLUTION

layer	in $\rightarrow$ out	weights	biases	parameters
1	784 $\rightarrow$ 128	100,352	128	100,480
2	128 $\rightarrow$ 64	8,192	64	8,256
3	64 $\rightarrow$ 10	640	10	650
total		109,184	202	109,386

STEP 1 Three Dense layers carry weights, so $L = 3$; units 128, 64, 10 (the 784 inputs are not a layer). Ten softmax outputs with categorical cross-entropy $\rightarrow$ multi-class, $K = 10$.

STEP 2 Layer 1: $784 \times 128 + 128 = 100,352 + 128 = 100,480$. Layer 2: $128 \times 64 + 64 = 8,192 + 64 = 8,256$. Layer 3: $64 \times 10 + 10 = 650$.

STEP 3 Total = $100,480 + 8,256 + 650 = 109,386$ (weights 109,184 + biases 202). Layer 1 share = $100,480/109,386 = 91.86\%$.

STEP 4 Softmax makes the 10 outputs sum to 1, so they are the probabilities of 10 mutually exclusive digits and the largest is the answer; ten sigmoids would sum to anything. Use sigmoids only if an image may carry several labels at once (multi-label, multi-hot target).

STEP 5 One epoch = one pass over all 10,000 training images. Updates per epoch = $\lceil 10,000/64 \rceil = \lceil 156.25 \rceil = 157$; in 15 epochs = $157 \times 15 = 2,355$.

STEP 6 Epochs for 5,000 updates: $5,000/157 = 31.85 \Rightarrow 32$ epochs ($32 \times 157 = 5,024$).

STEP 7 Sum = $0.02 + 0.01 + 0.05 + 0.03 + 0.04 + 0.01 + 0.02 + 0.73 + 0.05 + 0.04 = 1.00 \checkmark$.

Largest output is 0.73 at index 7 $\rightarrow$ predicted digit 7 with confidence 73%; the other nine share 27%.

STEP 8 True digit 7: loss = $-\ln 0.73 = 0.315$. True digit 2: loss = $-\ln 0.05 = 2.996$ — about 9.5 times larger, because the model gave the right answer only 5%.

i) $L = 3$; 128, 64, 10; multi-class, $K = 10$

ii) $100,480 + 8,256 + 650 = 109,386$; layer 1 holds 91.86 %

iii) softmax outputs sum to 1 (one digit per image); switch to sigmoids for multi-label targets

iv) one pass over 10,000 images; 157 updates per epoch; 2,355 in 15 epochs; 32 epochs for 5,000

v) sum 1.00; digit 7 at 73 %; loss 0.315 if true 7, 2.996 if true 2

✓ ANSWERS AT A GLANCE

i) 3; 128, 64, 10; MC, $K = 10$	ii) 109,386; 91.86 %	iii) sum to 1; multi-label → sigmoid	iv) 157; 2,355; 32	v) 7 @ 0.73; 0.315 / 2.996
------------------------------------	-------------------------	---	-----------------------	-------------------------------

! WHERE MARKS ARE LOST

- Counting the input as a layer ($L = 4$) or forgetting the 202 biases (109,184).
- Rounding 156.25 down to 156 updates per epoch.
- Computing the loss from the predicted class (0.73) when the true class is 2 — cross-entropy uses p_{true} only.

Module 2: Artificial Neural Network and the Perceptron

Ten scenario questions on one artificial cell: weighted sums and thresholds, the perceptron learning rule traced to convergence, lines and half-planes, logged runs read backwards, the vector form, brain-scale arithmetic, linear separability, the perceptron design question for a four-row table, and the two output conventions with two learning rates.

On your slides: **Module 2, slides 5–51** · the brain in numbers · the artificial neuron $z = \sum_i w_i x_i + b$ · perceptron and logic gates · the learning rule · XOR · linearly separable data · vector form $h = \mathbf{w}^T \tilde{\mathbf{x}}$

Module 2 starts with the brain (10^{10} neurons, 10^4 connections each, 1 ms switching) and builds one artificial neuron: a weighted sum followed by an activation. It then spends most of its time on the **perceptron**: the threshold rule $\hat{y} = 1$ if $h \geq 0$, AND and OR gates by choosing weights, the learning rule $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$ traced epoch by epoch, the convergence theorem, the XOR wall, and the vector form. Every question below is an engineering scenario with parts that build on each other. Each is 7–8 marks in the mid-semester paper's form: a numeric core you compute completely, then one to three short pointed sub-parts (how many epochs and updates, which learning rate gives the larger change, the condition of the convergence theorem, why one layer cannot) whose answer is a specific number or fact from the deck.

Q	Scenario	Computation / justify	Slides	Marks
2.1	Smart lock: weighted sums, threshold, the boundary reading, the bias	5 / 3	11–12, 22–23	8
2.2	Machine alarm (AND): full learning trace with $\eta = 1$ from $\mathbf{w} = \mathbf{0}$; updates, epochs, convergence condition	6 / 2	33–39, 51	8
2.3	Fail-safe relay (NOR): trace with $\eta = 0.5$, verify, line and half-planes, epochs and updates	8 / 0	25–39, 51	8
2.4	A deployed perceptron: its line, distances, a new point, mis-classified rows	8 / 0	27–31, 45–48	8
2.5	A logged run read backwards: recover η , the examples, the sign flip	7 / 0	33, 36	7
2.6	Quality gate in vector form: $\mathbf{h} = \tilde{\mathbf{X}}\mathbf{w}$, accuracy, parameter counts	7 / 0	47–48	7
2.7	Brain versus machine: synapses, serial steps, time per pass	6 / 2	5–6, 13	8
2.8	Six drone-image points: test a line, build a line, the XOR contradiction	7 / 1	41–46	8
2.9	A four-row dataset: propose the architecture, why one layer fails, draw the MLP, count parameters	5 / 2	41–42, 50	7
2.10	One perceptron under $\{0, 1\}$ and $\{-1, +1\}$ targets; $\eta = 0.5$ vs $\eta = 2$; inputs recoded to ± 1	6 / 2	25, 33	8

Q2.1

A smart lock decides "unlock" from three sensor readings

8 marks Slides 11–12, 22–23 · Book 2, Ch. 2 and 4

A smart door lock uses one artificial neuron. Its three inputs are x_1 = keycard match (0 or 1), x_2 = PIN correct (0 or 1) and x_3 = tamper-sensor level (0, 1 or 2). The weights are $\mathbf{w} = (w_1, w_2, w_3) = (2, 3, -1)$ and the bias is $b = -4$. The neuron computes $z = w_1x_1 + w_2x_2 + w_3x_3 + b$ and unlocks ($\hat{y} = 1$) when $z \geq 0$; otherwise it stays locked ($\hat{y} = 0$). Four readings arrive in one minute:

Reading	x_1	x_2	x_3
R1	1	1	0
R2	0	1	2
R3	1	1	1
R4	1	0	0

- (a) Compute the weighted sum z for each of the four readings. [2]
 (b) Give the lock's decision (unlock / locked) for each reading. [1]
 (c) With $x_1 = x_2 = 1$, find the tamper level x_3 that puts the reading exactly on the decision boundary $z = 0$. Which of R1–R4 lies on the boundary? [2]
 (d) The installer changes the bias to $b = -5$. Recompute z for all four readings and state which decisions change. Justify the effect of the bias in terms of the threshold on the weighted sum $w_1x_1 + w_2x_2 + w_3x_3$. [3] **justify**

by hand

artificial neuron

justify

⊙ WHAT THIS TESTS

Whether you can run the neuron equation $z = \sum_i w_i x_i + b$, $\hat{y} = f(z)$ with a threshold, read a boundary case, and say exactly what the bias does: $z \geq 0$ is the same as $\sum_i w_i x_i \geq -b$, so the bias sets the threshold.

▶ HOW TO ATTEMPT

- Write $z = 2x_1 + 3x_2 - x_3 - 4$ once, then substitute each row.
- Threshold rule: $z = 0$ counts as unlock.
- For (c) set $z = 0$ and solve for x_3 . For (d) only the constant changes, so every z drops by 1.

Σ SOLUTION

STEP 1 $z = 2x_1 + 3x_2 + (-1)x_3 + (-4)$

(a) weights on the inputs, bias added once

STEP 2 R1: $z = 2(1) + 3(1) - 1(0) - 4 = 2 + 3 - 0 - 4 = 1$

(a)

STEP 3 R2: $z = 2(0) + 3(1) - 1(2) - 4 = 0 + 3 - 2 - 4 = -3$

(a)

STEP 4 R3: $z = 2(1) + 3(1) - 1(1) - 4 = 2 + 3 - 1 - 4 = 0$

(a)

STEP 5 R4: $z = 2(1) + 3(0) - 1(0) - 4 = 2 + 0 - 0 - 4 = -2$

(a)

STEP 6 R1: $1 \geq 0 \rightarrow$ unlock. R2: $-3 < 0 \rightarrow$ locked. R3: $0 \geq 0 \rightarrow$ unlock. R4: $-2 < 0 \rightarrow$ locked.

(b)

STEP 7 $2(1) + 3(1) - x_3 - 4 = 0 \Rightarrow 1 - x_3 = 0 \Rightarrow x_3 = 1$

(c)

STEP 8 R3 = (1, 1, 1) is that reading: $z = 0$, exactly on the boundary.

STEP 9 With $b = -5$ every z falls by 1: R1: $1 - 1 = 0 \rightarrow$ unlock; R2: $-3 - 1 = -4 \rightarrow$ locked; R3: $0 - 1 = -1 \rightarrow$ locked; R4: $-2 - 1 = -3 \rightarrow$ locked.

STEP 10 Only R3 changes (unlock $\rightarrow$ locked). R1 now sits on the boundary and still unlocks.

(d)

STEP 11 Key fact: $z \geq 0 \iff w_1x_1 + w_2x_2 + w_3x_3 \geq -b$. The bias is the threshold the weighted sum must reach. It was $-b = 4$; it is now 5. A more negative bias makes the lock stricter without touching any weight.

(a) $z = 1, -3, 0, -2$ for R1-R4

(b) unlock, locked, unlock, locked

(c) $x_3 = 1$; R3 lies on the boundary

(d) $z = 0, -4, -1, -3$; only R3 flips to locked. The threshold on the weighted sum is $-b$: it moves from 4 to 5.

✓ ANSWERS AT A GLANCE

(a) 1, -3, 0, -2 (b) U, L, U, L (c) $x_3 = 1$, R3 (d) 0, -4, -1, -3; R3 flips; threshold $-b$: 4 $\rightarrow$ 5

! WHERE MARKS ARE LOST

- Treating $z = 0$ as "locked". The deck's rule is $\hat{y} = 1$ when $z \geq 0$.
- Adding +4 instead of -4, or forgetting the $-1 \times x_3$ term for R2 and R3.
- In (d) saying "the bias shifts the line" without the number: the threshold on $\sum w_i x_i$ is $-b$, 4 before and 5 after.

Q2.2

A machine alarm learned by the perceptron rule, traced to convergence

8 marks Slides 33–39, 51 · Book 2, Ch. 5

A pump must raise an alarm ($t = 1$) only when *both* the vibration flag x_1 and the temperature flag x_2 are 1. A perceptron with weights $\mathbf{w} = (w_0, w_1, w_2)$ computes $h = w_0 + w_1x_1 + w_2x_2$ and outputs $\hat{y} = 1$ if $h \geq 0$, else 0. It is trained by the deck's rule $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$ (with $x_0 = 1$ for the bias), $\eta = 1$, starting from $\mathbf{w} = (0, 0, 0)$. In every epoch the four rows are presented in this order:

Row	x_1	x_2	t
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

- (a) Trace epoch 1 in a table: for each row give $h, \hat{y}$, whether an update happens, and the weights after the row. [2]
- (b) Continue the trace, epoch by epoch, until one full epoch passes with no update. Report the final weights. [3]
- (c) How many weight updates were made in total, and how many epochs were run (counting the final clean pass)? [1]

(d) State the condition of the perceptron convergence theorem and show, with one separating line, that these four rows satisfy it. [2] **justify**

by hand perceptron learning trace **justify**

⊙ WHAT THIS TESTS

Whether you can run the learning rule exactly as slides 35–39 do for NOT, but for four rows and several epochs: compute h , threshold, compare with t , add $\eta(t - \hat{y})\tilde{\mathbf{x}}$ only on a miss, and know when to stop.

▶ HOW TO ATTEMPT

- Columns: epoch, row, $\mathbf{w}$ before, h , $\hat{y}$, t , update?, $\mathbf{w}$ after. Carry the weights straight down.
- On a miss with $t = 1, \hat{y} = 0$ add $\tilde{\mathbf{x}} = (1, x_1, x_2)$; with $t = 0, \hat{y} = 1$ subtract it. With $\eta = 1$ nothing else is multiplied.
- Stop only after an epoch with four ticks in a row.

Σ SOLUTION

STEP 1 Rule with $\eta = 1$: on a miss, $\mathbf{w} \leftarrow \mathbf{w} + (t - \hat{y})(1, x_1, x_2)$. Start $(0, 0, 0)$.

(a)

Epoch	Row (x_1, x_2)	t	$\mathbf{w}$ before	$h = w_0 + w_1x_1 + w_2x_2$	$\hat{y}$	Update?	$\mathbf{w}$ after
1	1 $(0, 0)$	0	$(0, 0, 0)$	$0 + 0 + 0 = 0$	1	$-(1, 0, 0)$	$(-1, 0, 0)$
1	2 $(0, 1)$	0	$(-1, 0, 0)$	$-1 + 0 + 0 = -1$	0	no	$(-1, 0, 0)$
1	3 $(1, 0)$	0	$(-1, 0, 0)$	$-1 + 0 + 0 = -1$	0	no	$(-1, 0, 0)$
1	4 $(1, 1)$	1	$(-1, 0, 0)$	$-1 + 0 + 0 = -1$	0	$+(1, 1, 1)$	$(0, 1, 1)$
2	1 $(0, 0)$	0	$(0, 1, 1)$	$0 + 0 + 0 = 0$	1	$-(1, 0, 0)$	$(-1, 1, 1)$
2	2 $(0, 1)$	0	$(-1, 1, 1)$	$-1 + 0 + 1 = 0$	1	$-(1, 0, 1)$	$(-2, 1, 0)$
2	3 $(1, 0)$	0	$(-2, 1, 0)$	$-2 + 1 + 0 = -1$	0	no	$(-2, 1, 0)$
2	4 $(1, 1)$	1	$(-2, 1, 0)$	$-2 + 1 + 0 = -1$	0	$+(1, 1, 1)$	$(-1, 2, 1)$
3	1 $(0, 0)$	0	$(-1, 2, 1)$	-1	0	no	$(-1, 2, 1)$
3	2 $(0, 1)$	0	$(-1, 2, 1)$	$-1 + 0 + 1 = 0$	1	$-(1, 0, 1)$	$(-2, 2, 0)$
3	3 $(1, 0)$	0	$(-2, 2, 0)$	$-2 + 2 + 0 = 0$	1	$-(1, 1, 0)$	$(-3, 1, 0)$
3	4 $(1, 1)$	1	$(-3, 1, 0)$	$-3 + 1 + 0 = -2$	0	$+(1, 1, 1)$	$(-2, 2, 1)$
4	1 $(0, 0)$	0	$(-2, 2, 1)$	-2	0	no	$(-2, 2, 1)$
4	2 $(0, 1)$	0	$(-2, 2, 1)$	$-2 + 0 + 1 = -1$	0	no	$(-2, 2, 1)$
4	3 $(1, 0)$	0	$(-2, 2, 1)$	$-2 + 2 + 0 = 0$	1	$-(1, 1, 0)$	$(-3, 1, 1)$
4	4 $(1, 1)$	1	$(-3, 1, 1)$	$-3 + 1 + 1 = -1$	0	$+(1, 1, 1)$	$(-2, 2, 2)$
5	1 $(0, 0)$	0	$(-2, 2, 2)$	-2	0	no	$(-2, 2, 2)$
5	2 $(0, 1)$	0	$(-2, 2, 2)$	$-2 + 0 + 2 = 0$	1	$-(1, 0, 1)$	$(-3, 2, 1)$
5	3 $(1, 0)$	0	$(-3, 2, 1)$	$-3 + 2 + 0 = -1$	0	no	$(-3, 2, 1)$
5	4 $(1, 1)$	1	$(-3, 2, 1)$	$-3 + 2 + 1 = 0$	1	no	$(-3, 2, 1)$
6	1 $(0, 0)$	0	$(-3, 2, 1)$	-3	0	no	$(-3, 2, 1)$
6	2 $(0, 1)$	0	$(-3, 2, 1)$	$-3 + 0 + 1 = -2$	0	no	$(-3, 2, 1)$
6	3 $(1, 0)$	0	$(-3, 2, 1)$	$-3 + 2 + 0 = -1$	0	no	$(-3, 2, 1)$
6	4 $(1, 1)$	1	$(-3, 2, 1)$	$-3 + 2 + 1 = 0$	1	no	$(-3, 2, 1)$

STEP 1 Epoch 1: two updates (rows 1 and 4); weights after epoch 1 are $(0, 1, 1)$.

(a)

STEP 2 Updates per epoch: 2, 3, 3, 2, 1, 0. Epoch 6 has no update, so the algorithm stops there.

(b)

STEP 3 Final $\mathbf{w} = (-3, 2, 1)$. Check: $h = -3, -2, -1, 0$ on rows 1–4, so $\hat{y} = 0, 0, 0, 1 = \text{AND}$.

(b)

STEP 4 Total updates = $2 + 3 + 3 + 2 + 1 = 11$. Epochs run = 6 (five with updates plus the clean pass).

(c)

STEP 5 Condition (slide 33): *if the data are linearly separable, the perceptron learning algorithm*

(d)

converges in a finite number of steps. The four AND rows are linearly separable: the line $x_1 + x_2 = 1.5$, i.e. $\mathbf{w} = (-1.5, 1, 1)$, gives $h = -1.5, -0.5, -0.5, 0.5 \rightarrow 0, 0, 0, 1$. So a finite number of updates (here 11) was guaranteed.

(a) Epoch 1: updates on rows 1 and 4; $\mathbf{w} = (0, 1, 1)$ after epoch 1

(b) Table above; updates per epoch 2, 3, 3, 2, 1, 0; final $\mathbf{w} = (-3, 2, 1)$

(c) 11 updates; 6 epochs

(d) Condition: linearly separable data $\Rightarrow$ finite steps. AND is separated by $x_1 + x_2 = 1.5$.

✓ ANSWERS AT A GLANCE

(a) $(0, 1, 1)$, 2 updates (b) $(-3, 2, 1)$ (c) 11 updates; 6 epochs (d) linearly separable $\Rightarrow$ finite

! WHERE MARKS ARE LOST

- $h = 0$ gives $\hat{y} = 1$. Rows with $h = 0$ and $t = 0$ are misses and must be updated (epochs 2, 3, 4, 5 all have one).
- Forgetting to update w_0 (the bias input is $x_0 = 1$, so w_0 changes on every miss).
- Stopping after the last update instead of after a clean epoch: 5 epochs is wrong, 6 is right.
- In (d) quoting the theorem without naming the property (linear separability) that AND satisfies.

Q2.3

A fail-safe relay (NOR) learned with $\eta = 0.5$ from non-zero weights

8 marks Slides 25–39, 51 · Book 2, Ch. 4 and 5

A relay must close ($t = 1$) only when *neither* fault flag is set – a NOR gate of x_1 and x_2 . The perceptron is $\hat{y} = 1$ if $h = w_0 + w_1x_1 + w_2x_2 \geq 0$, else 0. Training uses $\eta = 0.5$, starting weights $\mathbf{w} = (w_0, w_1, w_2) = (1, 0.5, 0.5)$, and the rows in the order $(0, 0), (0, 1), (1, 0), (1, 1)$ with targets $t = 1, 0, 0, 0$.

- (a) Trace the learning algorithm epoch by epoch until an epoch passes with no update. Give $h, \hat{y}$, the update and the weights after every row. [3]
- (b) Verify the final weights on all four rows by computing h and $\hat{y}$. [1]
- (c) Write the equation of the decision line and the inequality for each half-plane (the $\hat{y} = 1$ side and the $\hat{y} = 0$ side). [1]
- (d) Compute the distance of the point $(1, 1)$ from that line using $|h|/\sqrt{w_1^2 + w_2^2}$, to 3 decimals. [1]
- (e) How many epochs and how many updates did the run take? State the condition under which the perceptron convergence theorem guarantees such a stop, and the number of parameters that were learned. [2]

by hand

perceptron learning

line and half-planes

WHAT THIS TESTS

The learning rule with a learning rate that is not 1 (each update adds $\pm 0.5 \tilde{\mathbf{x}}$), a start away from zero, and reading the geometry — line, sides, distance — off the finished weights.

HOW TO ATTEMPT

- On a miss add $0.5(t - \hat{y})(1, x_1, x_2)$: that is $\pm(0.5, 0.5x_1, 0.5x_2)$.
- The line is $h = 0$. Divide through by a common factor to make it clean. The $\hat{y} = 1$ side is $h \geq 0$.
- Distance = $|h(\text{point})|$ divided by the length of (w_1, w_2) .

SOLUTION

Epoch	Row (x_1, x_2)	t	w before	h	$\hat{y}$	Update	w after
1	(0,0)	1	(1, 0.5, 0.5)	$1 + 0 + 0 = 1$	1	no	(1, 0.5, 0.5)
1	(0,1)	0	(1, 0.5, 0.5)	$1 + 0 + 0.5 = 1.5$	1	$-(0.5, 0, 0.5)$	(0.5, 0.5, 0)
1	(1,0)	0	(0.5, 0.5, 0)	$0.5 + 0.5 + 0 = 1$	1	$-(0.5, 0.5, 0)$	(0, 0, 0)
1	(1,1)	0	(0, 0, 0)	$0 + 0 + 0 = 0$	1	$-(0.5, 0.5, 0.5)$	(-0.5, -0.5, -0.5)
2	(0,0)	1	(-0.5, -0.5, -0.5)	-0.5	0	$+(0.5, 0, 0)$	(0, -0.5, -0.5)
2	(0,1)	0	(0, -0.5, -0.5)	$0 + 0 - 0.5 = -0.5$	0	no	(0, -0.5, -0.5)
2	(1,0)	0	(0, -0.5, -0.5)	$0 - 0.5 + 0 = -0.5$	0	no	(0, -0.5, -0.5)
2	(1,1)	0	(0, -0.5, -0.5)	$0 - 0.5 - 0.5 = -1$	0	no	(0, -0.5, -0.5)
3	(0,0)	1	(0, -0.5, -0.5)	0	1	no	(0, -0.5, -0.5)
3	(0,1)	0	(0, -0.5, -0.5)	-0.5	0	no	(0, -0.5, -0.5)
3	(1,0)	0	(0, -0.5, -0.5)	-0.5	0	no	(0, -0.5, -0.5)
3	(1,1)	0	(0, -0.5, -0.5)	-1	0	no	(0, -0.5, -0.5)

STEP 1 Epoch 1 has three updates, epoch 2 has one, epoch 3 has none: converged after 4 updates
(a) in 3 epochs. Final $\mathbf{w} = (0, -0.5, -0.5)$.

STEP 2 (0,0): $h = 0 - 0 - 0 = 0 \Rightarrow \hat{y} = 1 \checkmark$. (0,1): $h = 0 - 0 - 0.5 = -0.5 \Rightarrow 0 \checkmark$.

(b)

STEP 3 (1,0): $h = 0 - 0.5 - 0 = -0.5 \Rightarrow 0 \checkmark$. (1,1): $h = 0 - 0.5 - 0.5 = -1 \Rightarrow 0 \checkmark$. All four rows are NOR.

(b)

STEP 4 Line: $0 - 0.5x_1 - 0.5x_2 = 0$. Multiply by -2 : $x_1 + x_2 = 0$.

(c)

STEP 5 $\hat{y} = 1$ half-plane: $h \geq 0 \iff x_1 + x_2 \leq 0$. $\hat{y} = 0$ half-plane: $x_1 + x_2 > 0$. Only (0,0) is on the closing side, and it sits exactly on the line.

(c)

STEP 6 $h(1,1) = -1$; $\sqrt{w_1^2 + w_2^2} = \sqrt{0.25 + 0.25} = \sqrt{0.5} = 0.707$

(d)

STEP 7 distance = $|-1|/0.707 = 1.414$

(d)

the diagonal of the unit square, as the line through the origin at 45° suggests

STEP 8 3 epochs (2 with updates + 1 clean pass), 4 updates. Condition: the data must be linearly separable — then the algorithm converges in a finite number of steps (slide 33); NOR is separable by $x_1 + x_2 = 0$ as shown in (c). Parameters learned: $n + 1 = 2 + 1 = 3$ (w_0, w_1, w_2).

(e)

(a) 4 updates, 3 epochs; $\mathbf{w} = (0, -0.5, -0.5)$

(b) $h = 0, -0.5, -0.5, -1$; $\hat{y} = 1, 0, 0, 0$ — all correct

(c) $x_1 + x_2 = 0; \hat{y} = 1$ where $x_1 + x_2 \leq 0, \hat{y} = 0$ where $x_1 + x_2 > 0$

(d) $1/0.707 = 1.414$

(e) 3 epochs, 4 updates; condition: linearly separable data; 3 parameters

✓ ANSWERS AT A GLANCE

(a) $(0, -0.5, -0.5)$, 4 updates, 3 epochs (b) 1, 0, 0, 0 (c) $x_1 + x_2 = 0$ (d) 1.414 (e) 3 ep, 4 upd; separable; 3

! WHERE MARKS ARE LOST

- Adding a full $\tilde{\mathbf{x}}$ instead of $0.5 \tilde{\mathbf{x}}$ – the whole trace then differs.
- Row (1, 1) in epoch 1: $h = 0$ gives $\hat{y} = 1$, which is a miss and must be updated.
- Dividing by $\|\mathbf{w}\|$ with w_0 included; the distance formula uses only (w_1, w_2) .

A deployed perceptron: its line,

Q2.4 distances, a new point, and the rows it gets wrong

8 marks Slides 27–31, 45–48 · Book 2, Ch. 4 and 6

A firmware team ships a perceptron with final weights $\mathbf{w} = (w_0, w_1, w_2) = (-3, 4, 3)$, rule $\hat{y} = 1$ if $h = w_0 + w_1 x_1 + w_2 x_2 \geq 0$, else 0. It was meant to behave as an AND gate on two 0/1 flags, but the inputs it now sees are real-valued. The distance of a point from the decision line is $|h|/\sqrt{w_1^2 + w_2^2}$.

- (a) Write the equation of the decision line and find where it crosses the x_1 axis and the x_2 axis. [1]
- (b) Compute the distance from the line of the three points $(0, 0)$, $(1, 1)$ and $(0.5, 0.5)$. [3]
- (c) Classify the new reading $(0.4, 0.2)$. [1]
- (d) Run the four AND truth-table rows through the perceptron and count how many are misclassified. [1]
- (e) Keeping $w_1 = 4$ and $w_2 = 3$, find the full range of w_0 for which all four AND rows are correct. [2]

by hand decision line distance and inequalities

🎯 WHAT THIS TESTS

Reading a perceptron as geometry: the line $h = 0$, the distance formula with $\sqrt{4^2 + 3^2} = 5$, which side a point falls on, and the four inequalities of slide 27 solved for the one unknown w_0 .

▶ HOW TO ATTEMPT

- Line: $4x_1 + 3x_2 - 3 = 0$. Set $x_2 = 0$ for one intercept, $x_1 = 0$ for the other.
- Compute h at each point first; the distance is $|h|/5$.
- For (e) write the four AND inequalities with w_1, w_2 filled in; the strict ones give the upper bound, the $\geq$ one gives the lower bound.

Σ SOLUTION

STEP 1 Line: $-3 + 4x_1 + 3x_2 = 0$, i.e. $4x_1 + 3x_2 = 3$.

(a)

STEP 2 x_1 -intercept ($x_2 = 0$): $4x_1 = 3 \Rightarrow x_1 = 0.75$. x_2 -intercept ($x_1 = 0$): $3x_2 = 3 \Rightarrow x_2 = 1$.

(a)

STEP 3 $\sqrt{w_1^2 + w_2^2} = \sqrt{16 + 9} = \sqrt{25} = 5$

(b)

STEP 4 $(0, 0)$: $h = -3 + 0 + 0 = -3$; distance $= 3/5 = 0.6$

(b)

STEP 5 $(1, 1)$: $h = -3 + 4 + 3 = 4$; distance $= 4/5 = 0.8$

(b)

STEP 6 $(0.5, 0.5)$: $h = -3 + 2 + 1.5 = 0.5$; distance $= 0.5/5 = 0.1$

(b)

STEP 7 $(0.4, 0.2)$: $h = -3 + 4(0.4) + 3(0.2) = -3 + 1.6 + 0.6 = -0.8 < 0 \Rightarrow \hat{y} = 0$

(c)

STEP 8 $(0, 0)$: $h = -3 \Rightarrow 0 \checkmark$ ($t = 0$). $(0, 1)$: $h = -3 + 0 + 3 = 0 \Rightarrow 1 \times$ ($t = 0$).

(d)

STEP 9 $(1, 0)$: $h = -3 + 4 + 0 = 1 \Rightarrow 1 \times$ ($t = 0$). $(1, 1)$: $h = 4 \Rightarrow 1 \checkmark$ ($t = 1$). Mis-classified: 2 rows.

(d)

STEP 10 AND needs: $w_0 < 0$; $w_0 + 3 < 0$; $w_0 + 4 < 0$; $w_0 + 4 + 3 \geq 0$.

(e)

slide 27, with $w_1 = 4$, $w_2 = 3$

STEP 11 The strict ones give $w_0 < -4$ (the tightest of 0, -3, -4). The last gives $w_0 \geq -7$. Range:

(e)

$-7 \leq w_0 < -4$. Check $w_0 = -5$: $h = -5, -2, -1, 2 \Rightarrow 0, 0, 0, 1 \checkmark$.

(a) $4x_1 + 3x_2 = 3$; crosses at $x_1 = 0.75$ and $x_2 = 1$

(b) 0.6, 0.8, 0.1

(c) $h = -0.8, \hat{y} = 0$

(d) 2 rows wrong: $(0, 1)$ and $(1, 0)$

(e) $-7 \leq w_0 < -4$

✓ ANSWERS AT A GLANCE

(a) $4x_1 + 3x_2 = 3$; 0.75, 1

(b) 0.6, 0.8, 0.1

(c) 0

(d) 2

(e) $[-7, -4)$

! WHERE MARKS ARE LOST

- Using $\sqrt{9 + 16 + 9}$ (including w_0) in the distance.
- Calling $(0, 1)$ correct: $h = 0$ fires, so it is a miss for AND.
- Writing $w_0 \leq -4$ or $w_0 > -7$: the strict inequality belongs to the rows with $t = 0$, the $\geq$ to the row with $t = 1$.

Q2.5

A logged perceptron run read backwards: recover η , the examples, the sign flip

7 marks Slides 33, 36 · Book 2, Ch. 5

A pump monitor classifies readings $(x_1, x_2) = (\text{vibration level, temperature level})$ as normal ($t = 1$) or faulty ($t = 0$). The four training examples are:

Example	x_1	x_2	t
A	0	1	1
B	1	0	1
C	3	3	0
D	4	1	0

The perceptron rule $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$ ($x_0 = 1$) was run from $\mathbf{w} = (0, 0, 0)$ with 0/1 outputs. The log kept only the weights (w_0, w_1, w_2) after each update, not which example caused it:

after update	0 (start)	1	2	3	4	5
$\mathbf{w}$	(0, 0, 0)	(-0.2, -0.6, -0.6)	(0, -0.6, -0.4)	(0.2, -0.4, -0.4)	(0.4, -0.4, -0.2)	(0.6, -0.2, -0.2)

- Recover the learning rate η . [1]
- For each of the five updates, identify which example (A-D) caused it and whether it was a false "faulty" ($t = 1, \hat{y} = 0$) or a false "normal" ($t = 0, \hat{y} = 1$). [3]
- At which update did the sign of $(t - \hat{y})$ flip, and from what to what? [1]
- Using the final weights, compute h and $\hat{y}$ for A-D and state how many are correct. Has the run converged? [2]

by hand

recover a hyperparameter

perceptron learning

WHAT THIS TESTS

Whether you can turn the update rule around. Every update is $\Delta \mathbf{w} = \eta(t - \hat{y})(1, x_1, x_2)$ with $t - \hat{y} = \pm 1$, so the change in w_0 is $\pm \eta$ and the ratios $\Delta w_1 / \Delta w_0, \Delta w_2 / \Delta w_0$ are the example's inputs.

HOW TO ATTEMPT

- Subtract consecutive weight vectors to get each $\Delta \mathbf{w}$.
- $|\Delta w_0| = \eta$; the sign of Δw_0 is the sign of $t - \hat{y}$.
- Divide $\Delta w_1, \Delta w_2$ by Δw_0 to read (x_1, x_2) , then match it to a row.

SOLUTION

Update	$\Delta \mathbf{w}$	$\Delta w_0 = \eta(t - \hat{y})$	$(\Delta w_1, \Delta w_2) / \Delta w_0$	Example	Miss
1	(-0.2, -0.6, -0.6)	-0.2 $\rightarrow t - \hat{y} = -1$	(3, 3)	C	$t = 0, \hat{y} = 1$ false "normal"
2	(+0.2, 0, +0.2)	+0.2 $\rightarrow +1$	(0, 1)	A	$t = 1, \hat{y} = 0$ false "faulty"
3	(+0.2, +0.2, 0)	+0.2 $\rightarrow +1$	(1, 0)	B	false "faulty"
4	(+0.2, 0, +0.2)	+0.2 $\rightarrow +1$	(0, 1)	A	false "faulty"
5	(+0.2, +0.2, 0)	+0.2 $\rightarrow +1$	(1, 0)	B	false "faulty"

STEP 1 Update 1: $\Delta w_0 = -0.2 - 0 = -0.2 = \eta(t - \hat{y})$ with $t - \hat{y} = \pm 1$, so $\eta = 0.2$. Check update 2:

(a) $\Delta w_0 = 0 - (-0.2) = +0.2 \checkmark$.

STEP 2 Update 1: $\Delta w_1 / \Delta w_0 = -0.6 / -0.2 = 3, \Delta w_2 / \Delta w_0 = 3 \rightarrow$ example C (3, 3); sign -1 means

(b) $t = 0, \hat{y} = 1$.

STEP 3 Updates 2-5 alternate A (0, 1), B (1, 0), A, B, all with sign +1 ($t = 1, \hat{y} = 0$). Sequence: C, A,

(b) B, A, B.

STEP 4 Update 1 has $t - \hat{y} = -1$; update 2 has +1. The sign flipped at update 2, from -1 to +1,

(c) and stayed +1 afterwards.

STEP 5 Final $\mathbf{w} = (0.6, -0.2, -0.2)$. A: $h = 0.6 - 0 - 0.2 = 0.4 \Rightarrow 1 \checkmark$. B: $h = 0.6 - 0.2 - 0 = 0.4 \Rightarrow 1 \checkmark$.

STEP 6 C: $h = 0.6 - 0.6 - 0.6 = -0.6 \Rightarrow 0 \checkmark$. D: $h = 0.6 - 0.8 - 0.2 = -0.4 \Rightarrow 0 \checkmark$. 4 of 4 correct: converged (the line is $x_1 + x_2 = 3$).

(a) $\eta = 0.2$

(b) C (false normal), A, B, A, B (all false faulty)

(c) at update 2: $-1 \rightarrow +1$

(d) $h = 0.4, 0.4, -0.6, -0.4$; $\hat{y} = 1, 1, 0, 0$; 4/4 correct — converged

ANSWERS AT A GLANCE

(a) 0.2

(b) C, A, B, A, B

(c) update 2, - to +

(d) 4/4, converged

WHERE MARKS ARE LOST

- Reading η from Δw_1 (which is ηx_1 , not η). Only the bias slot gives η directly.
- Dividing by $+0.2$ in update 1 and getting $(-3, -3)$: keep the sign of Δw_0 .
- In (d) forgetting that h uses the last logged weights, not the start.

Q2.6

A quality gate in vector form: $\mathbf{h} = \tilde{\mathbf{X}}\mathbf{w}$ for five parts at once

7 marks Slides 47–48 · Book 2, Ch. 6

A factory gate passes a machined part ($t = 1$) or rejects it ($t = 0$) from four measured features. The perceptron uses the deck's vector form: $\tilde{\mathbf{x}} = (1, x_1, x_2, x_3, x_4)^\top$, $h = \mathbf{w}^\top \tilde{\mathbf{x}}$, $\hat{y} = 1$ if $h \geq 0$ else 0, with

$$\mathbf{w} = (w_0, w_1, w_2, w_3, w_4)^\top = (-2, 1, 0.5, -1, 2)^\top.$$

Five parts are measured (features only; the bias column of ones is added by you):

Part	x_1	x_2	x_3	x_4	t
1	2	2	1	1	1
2	0	4	2	0	0
3	3	0	0	1	1
4	1	2	3	0	0
5	2	2	2	1	0

- Write the shapes of the design matrix $\tilde{\mathbf{X}}$ (with the bias column), of $\mathbf{w}$, and of $\mathbf{h} = \tilde{\mathbf{X}}\mathbf{w}$. How many multiplications does the product take? [1]
- Compute $\mathbf{h} = \tilde{\mathbf{X}}\mathbf{w}$ as one matrix product, showing the five row sums. [3]
- Give the five outputs $\hat{y}$ and the accuracy against $\mathbf{t}$ as a percentage. [2]
- How many parameters does this perceptron have for $d = 4$ features? Repeat for $d = 10$ and $d = 100$. [1]

by hand

vector form

shapes

🎯 WHAT THIS TESTS

Slide 48's matrix form: one row of $\tilde{\mathbf{X}}$ per example, one weight per column including the bias, one product for all predictions, and the count $d + 1$ of parameters.

▶ HOW TO ATTEMPT

- $\tilde{\mathbf{X}}$ is $N \times (d + 1) = 5 \times 5$; $\mathbf{w}$ is 5×1 ; so $\mathbf{h}$ is 5×1 .
- Each entry of $\mathbf{h}$ is a row of $\tilde{\mathbf{X}}$ dotted with $\mathbf{w}$: write the five terms, then add.
- Threshold each h_i ; accuracy = correct / 5.

Σ SOLUTION

STEP 1 $\tilde{\mathbf{X}}$ is 5×5 (5 parts, 1 + 4 columns); $\mathbf{w}$ is 5×1 ; $\mathbf{h} = \tilde{\mathbf{X}}\mathbf{w}$ is 5×1 . Multiplications: $5 \times 5 = 25$.

(a)

STEP 2

(b)

$$\tilde{\mathbf{X}}\mathbf{w} = \begin{pmatrix} 1 & 2 & 2 & 1 & 1 \\ 1 & 0 & 4 & 2 & 0 \\ 1 & 3 & 0 & 0 & 1 \\ 1 & 1 & 2 & 3 & 0 \\ 1 & 2 & 2 & 2 & 1 \end{pmatrix} \begin{pmatrix} -2 \\ 1 \\ 0.5 \\ -1 \\ 2 \end{pmatrix}$$

STEP 3 Row 1: $-2 + 2 + 2 + 1 - 1 + 2 = 2$. Row 2: $-2 + 0 + 2 - 2 + 0 = -2$. Row 3: $-2 + 3 + 0 - 0 +$

(b) $2 = 3$.

STEP 4 Row 4: $-2 + 1 + 1 - 3 + 0 = -3$. Row 5: $-2 + 2 + 1 - 2 + 2 = 1$. So $\mathbf{h} = (2, -2, 3, -3, 1)^\top$.

(b)

STEP 5 $\hat{\mathbf{y}} = (1, 0, 1, 0, 1)^\top$. Targets $\mathbf{t} = (1, 0, 1, 0, 0)^\top$: parts 1-4 correct, part 5 wrong. Accuracy =

(c) $4/5 = 80\%$.

STEP 6 Parameters = $d + 1$ (one weight per feature plus w_0): $d = 4 \Rightarrow 5$; $d = 10 \Rightarrow 11$; $d = 100 \Rightarrow$

(d) 101.

(a) $5 \times 5, 5 \times 1, 5 \times 1$; 25 multiplications

(b) $\mathbf{h} = (2, -2, 3, -3, 1)^\top$

(c) $\hat{\mathbf{y}} = (1, 0, 1, 0, 1)^\top$; accuracy 80 %

(d) 5, 11, 101

✓ ANSWERS AT A GLANCE

(a) $5 \times 5, 5 \times 1, 5 \times 1$; 25

(b) $(2, -2, 3, -3, 1)$

(c) $(1, 0, 1, 0, 1)$; 80 %

(d) 5, 11, 101

! WHERE MARKS ARE LOST

- Leaving out the column of ones, so $\tilde{\mathbf{X}}$ is 5×4 and the product with a 5-vector is not even defined.
- Sign slips on the $-1 \times x_3$ term (rows 2 and 4).
- Counting d parameters instead of $d + 1$.

Q2.7**Brain versus machine: synapses, serial steps, and time per pass****8 marks** Slides 5–6, 13 · Book 2, Ch. 1 and 3

Slide 6 gives the brain's numbers: about 10^{10} neurons, about 10^4 connections per neuron, a neuron switching time of about 0.001 s, and a scene-recognition time of about 1 s. A robotics team compares this with their own network: 10^6 weights, run on a chip that does 10^9 multiply-adds per second. Give every answer as a power of ten (or a plain number where it is small).

- (a) Estimate the number of synapses (connections) in the brain. [1]
- (b) How many switching steps, one after another, can fit into the 1 s recognition time? The slide says "100 inference steps"; what switching interval would give exactly 100? [2]
- (c) Justify the slide's conclusion "lot of parallel computation" using the numbers from (a) and (b). [2] **justify**
- (d) How long does one forward pass of the team's network take, if every weight costs one multiply-add? How many passes per second is that? [1]
- (e) If every synapse does one operation per switching time, how many synaptic operations per second does the brain do? By what factor does that exceed the chip? [2]

by hand**brain in numbers****justify****WHAT THIS TESTS**

Whether you can turn slide 6 into arithmetic with powers of ten and draw its one conclusion: too few serial steps for the job means the work is spread across an enormous number of units acting at once.

HOW TO ATTEMPT

- Multiply exponents by adding them: $10^{10} \times 10^4 = 10^{14}$.
- Steps = time available ÷ time per step.
- Time per pass = operations ÷ operations per second.

SOLUTION

STEP 1 synapses $\approx$ neurons $\times$ connections per neuron $= 10^{10} \times 10^4 = 10^{14}$

(a)

STEP 2 serial steps $= 1 \text{ s} / 0.001 \text{ s} = 1000 = 10^3$

(b)

STEP 3 For exactly 100 steps the interval would be $1 \text{ s} / 100 = 0.01 \text{ s} = 10 \text{ ms}$ — a realistic firing interval. Either way it is at most a few hundred to a thousand steps.

(b)

STEP 4 Key fact: a scene is recognised in only 10^2 – 10^3 serial steps, far too few for any step-by-step program, yet 10^{14} synapses are available. So each step must involve a huge number of synapses working at the same time — parallel computation, roughly 10^{14} operations per step.

(c)

STEP 5 time per pass $= 10^6 \text{ multiply-adds} / 10^9 \text{ per s} = 10^{-3} \text{ s} = 1 \text{ ms}$

(d)

STEP 6 passes per second $= 10^9 / 10^6 = 10^3 = 1000$

(d)

STEP 7 brain: $10^{14} \text{ synapses} \times 10^3 \text{ per s} = 10^{17} \text{ operations per second}$

(e)

STEP 8 factor $= 10^{17} / 10^9 = 10^8$. The chip would need 10^8 s (about 3.2 years) to do one brain-second of synaptic work.

(e)

(a) 10^{14} synapses

(b) 1000 steps at 1 ms; 100 steps needs a 10 ms interval

(c) $\leq 10^3$ serial steps but 10^{14} synapses $\Rightarrow$ about 10^{14} operations per step in parallel

(d) 10^{-3} s = 1 ms per pass; 1000 passes per second

(e) 10^{17} per second; factor 10^8

✓ ANSWERS AT A GLANCE

(a) 10^{14}

(b) 1000; 10 ms

(c) 10^{14} ops per step

(d) 1 ms; 1000/s

(e) 10^{17} ; 10^8

! WHERE MARKS ARE LOST

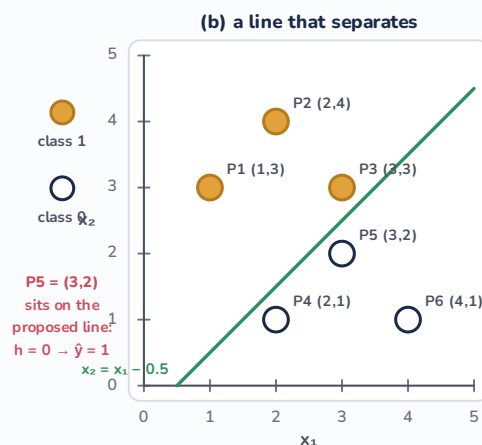
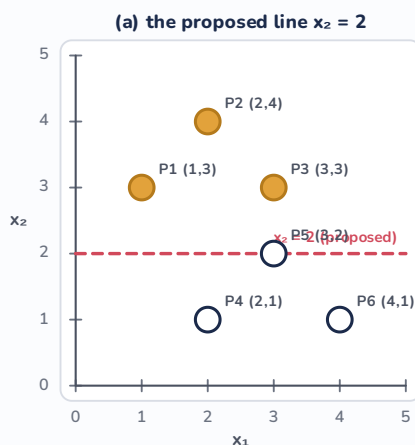
- Adding instead of multiplying the exponents in (a), or dividing the wrong way in (b) and (d).
- In (c) saying "the brain is parallel" without the two numbers that force the conclusion.
- Writing 1 ms as 10^{-2} s.

Q2.8

Six drone-image points: test a line, build a line, and the XOR contradiction

8 marks Slides 41–46 · Book 2, Ch. 6

A drone labels field patches as diseased (class 1) or healthy (class 0) from two features $(x_1, x_2) =$ (redness, leaf density). Six labelled patches are P1 (1, 3), P2 (2, 4), P3 (3, 3) in class 1 and P4 (2, 1), P5 (3, 2), P6 (4, 1) in class 0. A perceptron $\hat{y} = 1$ if $h = w_0 + w_1x_1 + w_2x_2 \geq 0$ is used.



- (a) An engineer proposes the line $x_2 = 2$, i.e. $\mathbf{w} = (-2, 0, 1)$. Compute h and $\hat{y}$ for all six points and list the mis-classified ones. [2]
- (b) Find weights $\mathbf{w}$ for a line that separates the two classes, and verify all six points with it. [3]
- (c) A second sensor gives the four points $(0, 0):0$, $(0, 1):1$, $(1, 0):1$, $(1, 1):0$. Write the four inequalities a perceptron would need and, by adding them in pairs, show that no $\mathbf{w}$ satisfies all four. [2]
- (d) Multiply your $\mathbf{w}$ from (b) by 3. Recompute h for P3 and P5 and state whether any decision changes. Justify using the sign rule $\hat{y} = 1 \iff h \geq 0$. [1] **justify**

by hand

linear separability

read the picture

justify

WHAT THIS TESTS

Slide 45's definition made concrete: a line separates two classes if every point's h has the right sign; the XOR set of slide 42 has no such line, and the reason is an arithmetic contradiction, not a picture.

HOW TO ATTEMPT

- Substitute each point into h ; remember $h = 0$ means $\hat{y} = 1$.
- From the picture, class 1 lies above the diagonal $x_2 = x_1$, class 0 below it. Shift the diagonal down a little so P3 (3, 3) has a margin: $x_2 = x_1 - 0.5$.
- For (c) copy slide 27's table style: one inequality per row, ≥ 0 for $t = 1$, < 0 for $t = 0$.

SOLUTION

Point	class	(a) $h = -2 + x_2$	$\hat{y}$	ok?	(b) $h = 1 - 2x_1 + 2x_2$	$\hat{y}$	ok?
P1 (1, 3)	1	$-2 + 3 = 1$	1	✓	$1 - 2 + 6 = 5$	1	✓
P2 (2, 4)	1	$-2 + 4 = 2$	1	✓	$1 - 4 + 8 = 5$	1	✓
P3 (3, 3)	1	$-2 + 3 = 1$	1	✓	$1 - 6 + 6 = 1$	1	✓
P4 (2, 1)	0	$-2 + 1 = -1$	0	✓	$1 - 4 + 2 = -1$	0	✓
P5 (3, 2)	0	$-2 + 2 = 0$	1	✗	$1 - 6 + 4 = -1$	0	✓
P6 (4, 1)	0	$-2 + 1 = -1$	0	✓	$1 - 8 + 2 = -5$	0	✓

STEP 1 With $\mathbf{w} = (-2, 0, 1)$: $h = -2 + x_2$. Only P5 (3, 2) fails: $h = 0$ fires, but its class is 0. One mis-classified point.

STEP 2 Class 1 sits above the diagonal, class 0 below. Take the line $x_2 = x_1 - 0.5$, i.e. $x_2 - x_1 + 0.5 = 0$; doubling gives $\mathbf{w} = (1, -2, 2)$.

STEP 3 Table: $h = 5, 5, 1$ for class 1 (all ≥ 0) and $-1, -1, -5$ for class 0 (all < 0). All six correct.

STEP 4 Rows: $(0, 0): 0 \Rightarrow w_0 < 0$ (i); $(0, 1): 1 \Rightarrow w_0 + w_2 \geq 0$ (ii); $(1, 0): 1 \Rightarrow w_0 + w_1 \geq 0$ (iii); $(1, 1): 0 \Rightarrow w_0 + w_1 + w_2 < 0$ (iv).

STEP 5 (ii) + (iii): $2w_0 + w_1 + w_2 \geq 0$. (i) + (iv): $2w_0 + w_1 + w_2 < 0$. The same quantity cannot be both ≥ 0 and < 0 : no line exists. This is XOR (slide 41–42).

STEP 6 $3\mathbf{w} = (3, -6, 6)$. P3: $3 - 18 + 18 = 3$; P5: $3 - 18 + 12 = -3$. Same signs as before (1 and -1), so no decision changes.

STEP 7 Key fact: $\hat{y}$ depends only on the sign of h , and multiplying by a positive number keeps every sign. So every positive multiple of $\mathbf{w}$ is the same line — one reason slide 45 says there are infinitely many separating lines.

(a) $h = 1, 2, 1, -1, 0, -1$; mis-classified: P5 only

(b) $\mathbf{w} = (1, -2, 2)$ (line $x_2 = x_1 - 0.5$); $h = 5, 5, 1, -1, -1, -5$, all correct

(c) (ii)+(iii) gives $2w_0 + w_1 + w_2 \geq 0$, (i)+(iv) gives $2w_0 + w_1 + w_2 < 0$: contradiction

(d) $h = 3$ and -3 ; nothing changes — the sign of h is unchanged by a positive scale

ANSWERS AT A GLANCE

(a) P5 wrong (b) $(1, -2, 2)$, all ✓ (c) $2w_0 + w_1 + w_2$ both ≥ 0 and < 0 (d) 3, -3; unchanged

WHERE MARKS ARE LOST

- Calling P5 correct because it is "on the line": $h = 0$ fires.
- In (b) giving a line without checking all six h values; the check is where the marks are.
- In (c) writing $\leq$ for the $t = 0$ rows; the strict inequality is what makes the contradiction.

A four-row dataset: propose a perceptron

Q2.9 architecture, show why one layer fails, draw the MLP, count parameters the MLP, count parameters

7 marks Slides 41–42, 50 · Book 2, Ch. 6

A binary classification dataset for a two-switch safety interlock is $(x_1, x_2, y) = (0, 0, 1), (0, 1, 0), (1, 0, 0), (1, 1, 1)$. Every unit available to you is a threshold perceptron, $\hat{y} = 1$ if $h = w_0 + w_1x_1 + w_2x_2 \geq 0$, else 0.

- Propose a suitable perceptron architecture (layers and units per layer) that classifies all four rows correctly. [1]
- Justify numerically, using the four inequalities a single-layer perceptron would need, why a single-layer perceptron cannot classify this dataset. [2] **justify**
- Draw the complete architecture with its activation functions and give all the weights. Verify it by computing every unit's weighted sum and output for the four rows. [3]
- Calculate the total number of trainable parameters (weights and biases) of your architecture. [1]

by hand

XOR wall

design an MLP

justify

WHAT THIS TESTS

The mid-sem's perceptron design question: spot that the table is XNOR (the complement of XOR), prove with the four inequalities that no single line works, build a $2 \rightarrow 2 \rightarrow 1$ network of threshold units, verify it row by row, and count its parameters.

HOW TO ATTEMPT

- Rows with $y = 1$ need $h \geq 0$, rows with $y = 0$ need $h < 0$. Write the four inequalities and add them in pairs.
- $y = 1$ exactly when both inputs agree: $y = \text{AND}(x_1, x_2)$ OR $\text{NOR}(x_1, x_2)$. Use gate weights from slides 27–31 for AND, NOR and OR.
- Parameters per unit = inputs + 1 bias.

SOLUTION

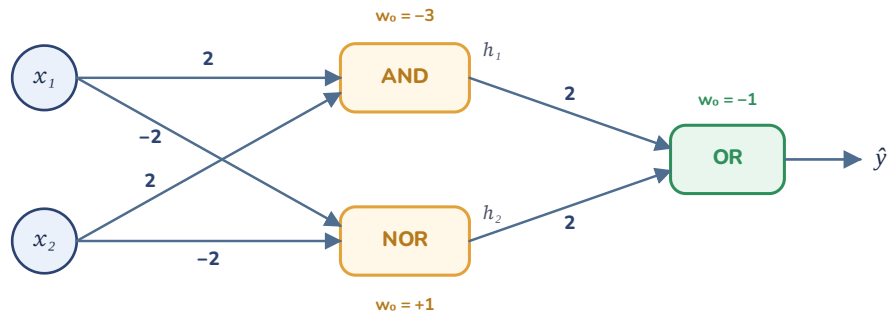
STEP 1 The output is 1 when $x_1 = x_2$ and 0 otherwise: this is XNOR, the complement of XOR (slide 41), so a single perceptron is not enough. Propose a two-layer perceptron (MLP): 2 inputs $\rightarrow$ hidden layer of 2 threshold units $\rightarrow$ 1 threshold output unit.

STEP 2 A single unit would need: $(0, 0): 1 \Rightarrow w_0 \geq 0$ (i); $(0, 1): 0 \Rightarrow w_0 + w_2 < 0$ (ii); $(1, 0): 0 \Rightarrow w_0 + w_1 < 0$ (iii); $(1, 1): 1 \Rightarrow w_0 + w_1 + w_2 \geq 0$ (iv).

STEP 3 (ii) + (iii): $2w_0 + w_1 + w_2 < 0$. (i) + (iv): $2w_0 + w_1 + w_2 \geq 0$. The same number cannot be both, so no (w_0, w_1, w_2) exists: the data are not linearly separable, and a single-layer perceptron cannot classify them (slide 42).

STEP 4 Architecture (figure): hidden unit 1 = AND with $(w_0, w_1, w_2) = (-3, 2, 2)$; hidden unit 2 = NOR with $(1, -2, -2)$; output unit = OR of (h_1, h_2) with $(-1, 2, 2)$. Every unit uses the

threshold (step) activation $\hat{y} = 1$ if $h \geq 0$.



hidden layer: 2 threshold units, AND $(-3, 2, 2)$ and NOR $(1, -2, -2)$ output: 1 threshold unit, OR $(-1, 2, 2)$
 every unit: $\hat{y} = 1$ if $h \geq 0$ else $0 \cdot 2 \rightarrow 2 \rightarrow 1$, 9 trainable parameters

(x_1, x_2)	AND: $-3 + 2x_1 + 2x_2$	h_1	NOR: $1 - 2x_1 - 2x_2$	h_2	OR: $-1 + 2h_1 + 2h_2$	$\hat{y}$	y
(0, 0)	$-3 + 0 + 0 = -3$	0	$1 - 0 - 0 = 1$	1	$-1 + 0 + 2 = 1$	1	1 ✓
(0, 1)	$-3 + 0 + 2 = -1$	0	$1 - 0 - 2 = -1$	0	$-1 + 0 + 0 = -1$	0	0 ✓
(1, 0)	$-3 + 2 + 0 = -1$	0	$1 - 2 - 0 = -1$	0	$-1 + 0 + 0 = -1$	0	0 ✓
(1, 1)	$-3 + 2 + 2 = 1$	1	$1 - 2 - 2 = -3$	0	$-1 + 2 + 0 = 1$	1	1 ✓

STEP 1 All four rows match y . In hidden space the rows become $(h_1, h_2) = (0, 1), (0, 0), (0, 0), (1, 0)$; the OR line $2h_1 + 2h_2 = 1$ separates the two class-1 points from the class-0 point $(0, 0)$.

STEP 2 Hidden layer: 2 units $\times$ (2 weights + 1 bias) = 6. Output layer: 1 unit $\times$ (2 weights + 1 bias) = 3. Total = 6 + 3 = 9 trainable parameters (6 weights, 3 biases).

(a) MLP 2 $\rightarrow$ 2 $\rightarrow$ 1 of threshold units

(b) (ii)+(iii) gives $2w_0 + w_1 + w_2 < 0$ but (i)+(iv) gives $2w_0 + w_1 + w_2 \geq 0$: not linearly separable, so no single-layer perceptron

(c) AND $(-3, 2, 2)$, NOR $(1, -2, -2) \rightarrow$ OR $(-1, 2, 2)$, step activations; outputs 1, 0, 0, 1

(d) 9 parameters (6 weights + 3 biases)

✓ ANSWERS AT A GLANCE

(a) 2 $\rightarrow$ 2 $\rightarrow$ 1 MLP **(b)** $2w_0 + w_1 + w_2$ both < 0 and ≥ 0 **(c)** 1, 0, 0, 1 **(d)** 9

! WHERE MARKS ARE LOST

- Answering (b) with "XOR is not linearly separable" and no inequalities — the marks are for the numbers.
- Feeding x_1, x_2 into the output unit instead of h_1, h_2 .
- Counting 6 parameters (weights only) and forgetting the three biases.

One perceptron, two output conventions:

Q2.10 $\{0, 1\}$ versus $\{-1, +1\}$, and inputs recoded to ± 1 **8 marks** Slides 25, 33 · Book 2, Ch. 4 and 5

Slide 25 says the classical perceptron uses outputs $\{-1, +1\}$ while the deck's examples use $\{0, 1\}$, and that "both are valid; the choice affects the bias/threshold calculation". A perceptron for an AND gate currently has $\mathbf{w} = (w_0, w_1, w_2) = (-1, 1.5, 0.5)$, fires when $h = w_0 + w_1x_1 + w_2x_2 \geq 0$,

and is updated by $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$ with $\eta = 0.5$. The example $(x_1, x_2) = (1, 0)$ with target "off" is presented.

- Show that $(1, 0)$ is mis-classified. Then compute $t - \hat{y}$, the update $\Delta \mathbf{w}$ and the new weights under each convention: (i) targets and outputs in $\{0, 1\}$; (ii) targets and outputs in $\{-1, +1\}$. Give the length of each $\Delta \mathbf{w}$ to 3 decimals. [2]
- What learning rate under the $\{0, 1\}$ convention gives exactly the same new weights as the $\{-1, +1\}$ run at $\eta = 0.5$? Compute those weights to confirm. [1]
- Repeat the $\{0, 1\}$ update from the same initial weights with $\eta = 2$. Show all three updated weights. Which learning rate, 0.5 or 2, produces the larger weight change, and by what factor? Compute h for the row $(1, 1)$ with the new weights and say which AND row is now wrong. [2]
- The firmware now stores each input as $x'_i = 2x_i - 1 \in \{-1, +1\}$ instead of $\{0, 1\}$. Find weights $\mathbf{w}' = (w'_0, w'_1, w'_2)$ such that $h' = w'_0 + w'_1x'_1 + w'_2x'_2$ equals the original h for every row, and state by how much the bias moves. [1]
- Justify, using the sign rule $\hat{y} = 1 \iff h \geq 0$, why the decisions of the $\{0, 1\}$ and the $\{-1, +1\}$ perceptron with the same $\mathbf{w}$ are identical on every input. [2] justify

by hand

output conventions

justify

🔗 WHAT THIS TESTS

What actually changes between the two conventions: $t - \hat{y}$ is ± 1 in one and ± 2 in the other, so the step doubles; the decision rule itself does not change. And what changes when the *inputs* are recoded: the bias.

▶ HOW TO ATTEMPT

- Compute h for $(1, 0)$; it fires, and AND wants "off".
- "Off" is 0 in one convention and -1 in the other; "fires" is 1 or $+1$. Compute $t - \hat{y}$, then $\Delta \mathbf{w} = \eta(t - \hat{y})(1, x_1, x_2)$.
- For (c) the step scales with η : compare lengths, then re-test the rows. For (d) substitute $x_i = (x'_i + 1)/2$ into h and collect the constant.

Σ SOLUTION

STEP 1 $h(1, 0) = -1 + 1.5(1) + 0.5(0) = 0.5 \geq 0$, so the perceptron fires; AND wants "off". Mis-classified.

STEP 2 (i) $\{0, 1\}$: $t = 0, \hat{y} = 1, t - \hat{y} = -1. \Delta \mathbf{w} = 0.5 \times (-1) \times (1, 1, 0) = (-0.5, -0.5, 0)$. New $\mathbf{w} = (-1.5, 1.0, 0.5)$.

STEP 3 (ii) $\{-1, +1\}$: $t = -1, \hat{y} = +1, t - \hat{y} = -2. \Delta \mathbf{w} = 0.5 \times (-2) \times (1, 1, 0) = (-1, -1, 0)$. New $\mathbf{w} = (-2, 0.5, 0.5)$.

STEP 4 Lengths: $\sqrt{0.5^2 + 0.5^2} = \sqrt{0.5} = 0.707$ and $\sqrt{1^2 + 1^2} = \sqrt{2} = 1.414$. The ± 1 step is exactly twice as long.

STEP 5 Since $t - \hat{y}$ doubles, use $\eta = 2 \times 0.5 = 1$ in the $\{0, 1\}$ run: $\Delta \mathbf{w} = 1 \times (-1) \times (1, 1, 0) = (-1, -1, 0)$, new $\mathbf{w} = (-2, 0.5, 0.5)$ – identical to (ii).

STEP 6 $\eta = 2$: $\Delta \mathbf{w} = 2 \times (-1) \times (1, 1, 0) = (-2, -2, 0)$. New $\mathbf{w} = (-1 - 2, 1.5 - 2, 0.5 - 0) = (-3, -0.5, 0.5)$.

STEP 7 Length = $\sqrt{2^2 + 2^2} = \sqrt{8} = 2.828$, against 0.707 at $\eta = 0.5$: $\eta = 2$ gives the larger change, by a factor $2.828/0.707 = 4$ (the step is proportional to η).

STEP 8 With $(-3, -0.5, 0.5)$: row $(1, 1)$ gives $h = -3 - 0.5 + 0.5 = -3 < 0 \Rightarrow \hat{y} = 0$, but AND

(c) wants 1. The big step fixed $(1, 0)$ ($h = -3.5$) and broke $(1, 1)$: overshooting.

STEP 9 $x_i = (x'_i + 1)/2$: $h = w_0 + w_1 \frac{x'_1+1}{2} + w_2 \frac{x'_2+1}{2} = (w_0 + \frac{w_1+w_2}{2}) + \frac{w_1}{2}x'_1 + \frac{w_2}{2}x'_2$

(d)

STEP 10 $w'_0 = -1 + \frac{1.5+0.5}{2} = -1 + 1 = 0$; $w'_1 = 0.75$; $w'_2 = 0.25$. The bias moves by +1 (from -1 to

(d)

0). Check $(1, 0) \rightarrow (1, -1)$: $h' = 0 + 0.75 - 0.25 = 0.5 = h \checkmark$.

STEP 11 Key fact: both conventions compute the same $h = \mathbf{w}^\top \tilde{\mathbf{x}}$ and both fire exactly when $h \geq 0$.

(e)

The convention only renames the two outputs (0 or -1 for "off", 1 or +1 for "on"), so the decision boundary $h = 0$ and every decision are identical. What differs is the learning step, which is twice as long under $\{-1, +1\}$.

(a) $h = 0.5$, fires, wrong. (i) $t - \hat{y} = -1$, $\Delta \mathbf{w} = (-0.5, -0.5, 0)$, $\mathbf{w} = (-1.5, 1.0, 0.5)$, length 0.707. (ii) $t - \hat{y} = -2$, $\Delta \mathbf{w} = (-1, -1, 0)$, $\mathbf{w} = (-2, 0.5, 0.5)$, length 1.414.

(b) $\eta = 1$; gives $(-2, 0.5, 0.5)$

(c) $\eta = 2$: $\mathbf{w} = (-3, -0.5, 0.5)$, length 2.828 - 4× the $\eta = 0.5$ change; $h(1, 1) = -3$, so row $(1, 1)$ is now wrong

(d) $\mathbf{w}' = (0, 0.75, 0.25)$; bias moves by +1

(e) Same h , same rule $h \geq 0 \Rightarrow$ same boundary and decisions; only the step size (×2) differs

✓ ANSWERS AT A GLANCE

(a) $(-1.5, 1, 0.5) / (-2, 0.5, 0.5)$; 0.707 / 1.414 (b) $\eta = 1$ (c) $(-3, -0.5, 0.5)$, ×4, (1, 1) wrong (d) $(0, 0.75, 0.25)$, +1 (e) same sign rule

! WHERE MARKS ARE LOST

- Using $t - \hat{y} = -1$ in the $\{-1, +1\}$ case; it is $-1 - (+1) = -2$.
- Updating w_2 even though $x_2 = 0$: the update is proportional to x_i .
- In (d) halving the bias too; only the weights halve, and the bias picks up $(w_1 + w_2)/2$.

Module 3: Linear Neural Networks for Regression

Ten mid-sem-style questions on one neuron that predicts "how much": weight updates with two learning rates, full gradient-descent iterations in matrix form, MSE/RMSE/MAE/ R^2 , feature scaling, reading loss curves, recovering η , identity versus sigmoid output, stopping rules, and a Keras regression snippet.

On your slides: **Module 3, slides 4–44** · regression · design matrix $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ · single neuron with identity activation · squared-error loss $J = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ · gradient $\frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$ · batch gradient descent · evaluation · tips and debugging

Module 3 is where Nova learns to answer "how much". One neuron with the identity activation computes $\hat{y} = w_0 + w_1x_1 + \dots + w_dx_d$. All N examples are stacked as a design matrix $\mathbf{X}$ with a column of ones, so every prediction at once is $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. The loss is the bowl $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, the gradient is $\nabla J = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$, and batch gradient descent repeats $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$ until a stopping rule fires. The deck ends with train/test error, MSE, RMSE, MAE and R^2 , and with practical tips: choosing η , when to stop, scaling features, and a debugging checklist. Every question below is built like the mid-sem paper: a scenario with all numbers given, a numeric core to compute completely, then one to three short pointed parts with one specific expected answer each.

Q	Scenario	Numeric / pointed marks	Slides	Marks
3.1	House-price DNN: one weight update with $\alpha = 0.01$ and with $\alpha = 1.5$; larger update; problems of a big α ; convergence criteria	5 / 3	20–23, 39, 41	8
3.2	Three flats: predictions, error vector, J , gradient, one update with $\eta = 0.1$, new J , percentage drop	8 / 0	12, 17, 21–23, 26–30	8
3.3	Area and bedrooms, $N = 4$: shapes, $\mathbf{X}\mathbf{w}$, J , gradient in matrix form, update, predict a new house	8 / 0	12–14, 22–23	8
3.4	Trained line $\hat{y} = 2x + 1$ on five test points: MSE, RMSE, MAE, R^2 ; compare with the mean baseline	7 / 1	33–35	8
3.5	Four flats: min-max and z-score of area and bedrooms; curvature ratio; why unscaled features zig-zag	7 / 1	40, 44	8
3.6	Three recorded loss sequences: which η is which, ratios, what to do; the largest and the one-step η for a bowl with $c = 4$	6 / 2	23, 39, 41	8
3.7	Recover η from $w^{(0)}$, $w^{(1)}$, $w^{(2)}$; verify; predict $w^{(3)}$; steps until the gap is below 0.01	8 / 0	17, 20–24	8
3.8	Same neuron with identity and with sigmoid output against a target of 250; losses; slopes; gradient flow	6 / 2	13–16	8
3.9	5,000 examples split 90/10; which stopping rule fires first from a table; time; early stopping from train/validation losses	6 / 2	33, 39	8

Q3.1

One gradient-descent update with a small and a large learning rate

8 marks Slides 20-23, 39, 41 · Book 3, Ch. 4

A deep neural network is being trained to predict house prices from three features: X_1 area (sq ft), X_2 bedrooms, X_3 distance from the city centre (km). After one forward pass and backpropagation on a batch, the computed gradient is $\nabla J = [12, 30, 6, 18]$ (the first entry belongs to the bias). The input vector after normalisation is $\mathbf{x} = [1, 0.5, 0.25, 0.4]$. The current weights, including the bias w_0 , are $\mathbf{W} = [8, 6, 2, 5]$. The price unit is ₹ lakh.

- Perform one gradient-descent weight update using $\alpha = 0.01$. Show the updated value of all four weights with complete calculations. [2]
- Repeat from the same initial weights using $\alpha = 1.5$. Show all four updated weights. [2]
- Which learning rate produces the larger weight update, and by what factor? Compute the prediction $\mathbf{W}^\top \mathbf{x}$ before the update and after each update. [2]
- State the potential problems of a very large learning rate, in terms of the factor $|1 - \eta c|$ that multiplies the distance to the minimum on a bowl of curvature c . [1] **justify**
- List all the convergence criteria of gradient descent given in the deck (slide 39), each with its formula or number. [1]

by hand

update rule

learning rate

WHAT THIS TESTS

The update rule $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J$ applied to every entry, including the bias; the fact that the step is proportional to η ; and the two facts behind "too large": overshoot when $|1 - \eta c| > 1$, and the four stopping rules of slide 39.

HOW TO ATTEMPT

- Write $w_j \leftarrow w_j - \alpha g_j$ once and do the four subtractions on separate lines.
- For (iii) compare the step vectors $\alpha \nabla J$; their ratio is just the ratio of the two α values. Then compute the dot product $\mathbf{W}^\top \mathbf{x}$ three times.
- For (iv) and (v) write short, specific facts — no essay.

SOLUTION

STEP 1 Update rule for each weight: $w_j^{\text{new}} = w_j - \alpha g_j$, with $\alpha = 0.01$ and $\mathbf{g} = \nabla J = [12, 30, 6, 18]$.

(i) the step is α times the gradient, subtracted

STEP 2 $w_0 = 8 - 0.01 \times 12 = 8 - 0.12 = 7.88$

(i)

STEP 3 $w_1 = 6 - 0.01 \times 30 = 6 - 0.30 = 5.70$

(i)

STEP 4 $w_2 = 2 - 0.01 \times 6 = 2 - 0.06 = 1.94$

(i)

STEP 5 $w_3 = 5 - 0.01 \times 18 = 5 - 0.18 = 4.82$

(i)

STEP 6 With $\alpha = 1.5$: $w_0 = 8 - 1.5 \times 12 = 8 - 18 = -10$

(ii)

STEP 7 $w_1 = 6 - 1.5 \times 30 = 6 - 45 = -39$

(ii)

STEP 8 $w_2 = 2 - 1.5 \times 6 = 2 - 9 = -7$

(ii)

STEP 9 $w_3 = 5 - 1.5 \times 18 = 5 - 27 = -22$

(ii)

every weight changed sign: the step jumped far past the minimum

STEP 10 Step vectors: $0.01 \nabla J = [0.12, 0.30, 0.06, 0.18]$ and $1.5 \nabla J = [18, 45, 9, 27]$. The ratio is

(iii)

$1.5/0.01 = 150$: the $\alpha = 1.5$ update is 150 times larger in every entry (step lengths 0.375 and 56.2).

STEP 11 Before: $\mathbf{W}^\top \mathbf{x} = 8(1) + 6(0.5) + 2(0.25) + 5(0.4) = 8 + 3 + 0.5 + 2 = 13.5$

(iii)

STEP 12 After $\alpha = 0.01$: $7.88 + 5.70(0.5) + 1.94(0.25) + 4.82(0.4) = 7.88 + 2.85 + 0.485 + 1.928 =$

(iii)

13.143

STEP 13 After $\alpha = 1.5$: $-10 - 39(0.5) - 7(0.25) - 22(0.4) = -10 - 19.5 - 1.75 - 8.8 = -40.05$, a

(iii)

negative price.

the small step nudges the prediction; the big step wrecks it

STEP 14 Key fact: on a bowl $J = \frac{c}{2}(w - w^*)^2$ one step multiplies the gap $w - w^*$ by $1 - \eta c$. If $\eta >$

(iv)

$2/c$ then $|1 - \eta c| > 1$: the gap grows every step. Problems: overshooting the minimum, oscillation, divergence (loss increases), and finally overflow to NaN/Inf (slide 41: "loss is NaN/Inf $\rightarrow$ learning rate too large"). Part (ii) shows the overshoot: all weights flipped sign.

STEP 15 Slide 39: (1) gradient magnitude $\|\nabla J\| < \epsilon$, e.g. $\epsilon = 10^{-4}$; (2) loss change $|J^{(t)} - J^{(t-1)}| < \epsilon$;

(v)

(3) maximum iterations reached, e.g. $T = 1000$ or $10,000$; (4) validation loss starts increasing (early stopping).

(i) $\mathbf{W} = [7.88, 5.70, 1.94, 4.82]$

(ii) $\mathbf{W} = [-10, -39, -7, -22]$

(iii) $\alpha = 1.5$, 150 times larger; predictions $13.5 \rightarrow 13.143$ ($\alpha = 0.01$) and $13.5 \rightarrow -40.05$ ($\alpha = 1.5$)

(iv) $|1 - \eta c| > 1$ when $\eta > 2/c$: overshoot, oscillation, divergence, NaN

(v) $\|\nabla J\| < \epsilon$; $|J^{(t)} - J^{(t-1)}| < \epsilon$; max iterations T ; validation loss rising

✓ ANSWERS AT A GLANCE

(i) 7.88, 5.70, 1.94, 4.82 (ii) -10, -39, -7, -22 (iii) $\alpha = 1.5, \times 150$; $13.5 \rightarrow 13.143 / -40.05$ (iv) $|1 - \eta c| > 1$ (v) 4 rules

! WHERE MARKS ARE LOST

- Adding $\alpha \nabla J$ instead of subtracting: the weights go uphill (8.12, 6.30, ...).
- Forgetting the bias entry: the question says all four weights.
- Answering (iii) with "the larger one" and no factor; the factor 150 is the mark.
- Listing only "max iterations" in (v): the deck gives four criteria.

Q3.2

One full iteration of batch gradient descent on three flats

8 marks Slides 12, 17, 21–23, 26–30 · Book 3, Ch. 3–5

Meena aunty gives Nova three flats. The input x is the size in thousands of sq ft and the target y is the price in units of ₹10 lakh:

Flat	Size x	Price y
1	1	5
2	2	8
3	4	13

The model is one neuron with identity activation, $\hat{y} = w_0 + w_1x$. Use the loss $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ and the gradient $\nabla J = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$. The weights start at $\mathbf{w}^{(0)} = (1, 2)^\top$ and the learning rate is $\eta = 0.1$.

- Write the design matrix $\mathbf{X}$ with its bias column and the target vector $\mathbf{y}$, with shapes. Compute the predictions $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)}$ and the error vector $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y}$. [2]
- Compute the loss $J(\mathbf{w}^{(0)})$. [1]
- Compute the gradient $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{N} \mathbf{X}^\top \mathbf{e}^{(0)}$. [2]
- Update the weights to $\mathbf{w}^{(1)}$ and compute the new predictions and the new loss $J(\mathbf{w}^{(1)})$. [2]
- Did the loss decrease? By what percentage? [1]

by hand

batch gradient descent

design matrix

WHAT THIS TESTS

The deck's worked example (slides 26–30) with fresh numbers and a non-zero start: predict, subtract, square-and-average with $\frac{1}{2N}$, multiply by $\mathbf{X}^\top$ with $\frac{1}{N}$, step, and check that the loss fell.

HOW TO ATTEMPT

- $\mathbf{X}$ has one row per flat, $[1, x^{(i)}]$; it is 3×2 .
- $\mathbf{X}^\top \mathbf{e}$ has two entries: the sum of the errors, and the sum of error $\times$ size. Divide both by $N = 3$.
- After the step, redo prediction, error and loss with the new weights; then $(J_0 - J_1)/J_0$.

SOLUTION

STEP 1
(i) $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 4 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{y} = \begin{bmatrix} 5 \\ 8 \\ 13 \end{bmatrix} \in \mathbb{R}^3, \quad \mathbf{w}^{(0)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \in \mathbb{R}^2$

$N = 3, d = 1$, so $d + 1 = 2$ columns; the first column is the ones for the bias

STEP 2 $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)}: 1 + 2(1) = 3; 1 + 2(2) = 5; 1 + 2(4) = 9$. So $\hat{\mathbf{y}}^{(0)} = (3, 5, 9)^\top$.

(i)

STEP 3 $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = (3 - 5, 5 - 8, 9 - 13)^\top = (-2, -3, -4)^\top$

(i)

STEP 4 $J(\mathbf{w}^{(0)}) = \frac{1}{2 \cdot 3} (2^2 + 3^2 + 4^2) = \frac{4+9+16}{6} = \frac{29}{6} = 4.833$

(ii)

STEP 5
(iii) $\mathbf{X}^\top \mathbf{e}^{(0)} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix} \begin{bmatrix} -2 \\ -3 \\ -4 \end{bmatrix} = \begin{bmatrix} -2 - 3 - 4 \\ -2 - 6 - 16 \end{bmatrix} = \begin{bmatrix} -9 \\ -24 \end{bmatrix}$

row 1 adds the errors; row 2 adds error $\times$ size

STEP 6
(iii) $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{3} \begin{bmatrix} -9 \\ -24 \end{bmatrix} = \begin{bmatrix} -3 \\ -8 \end{bmatrix}$

STEP 7 $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 0.1 \begin{bmatrix} -3 \\ -8 \end{bmatrix} = \begin{bmatrix} 1 + 0.3 \\ 2 + 0.8 \end{bmatrix} = \begin{bmatrix} 1.3 \\ 2.8 \end{bmatrix}$

(iv)

STEP 8 $\hat{\mathbf{y}}^{(1)}: 1.3 + 2.8(1) = 4.1; 1.3 + 2.8(2) = 6.9; 1.3 + 2.8(4) = 12.5$. So $\hat{\mathbf{y}}^{(1)} = (4.1, 6.9, 12.5)^\top$.

(iv)

STEP 9 $\mathbf{e}^{(1)} = (4.1 - 5, 6.9 - 8, 12.5 - 13)^\top = (-0.9, -1.1, -0.5)^\top$

(iv)

STEP 10 $J(\mathbf{w}^{(1)}) = \frac{1}{6} (0.81 + 1.21 + 0.25) = \frac{2.27}{6} = 0.378$

(iv)

STEP 11 Yes. Drop = $\frac{4.833 - 0.378}{4.833} = 0.922$, so the loss fell by about 92.2% in one step.

(v)

the deck's own example fell 80 % in one step; same shape of result

(i) $\mathbf{X}$ is 3×2 , $\mathbf{y}$ is 3×1 ; $\hat{\mathbf{y}}^{(0)} = (3, 5, 9)^\top$, $\mathbf{e}^{(0)} = (-2, -3, -4)^\top$

(ii) $J(\mathbf{w}^{(0)}) = 29/6 = 4.833$

(iii) $\nabla J = (-3, -8)^\top$

(iv) $\mathbf{w}^{(1)} = (1.3, 2.8)^\top$, $\hat{\mathbf{y}}^{(1)} = (4.1, 6.9, 12.5)^\top$, $J(\mathbf{w}^{(1)}) = 0.378$

(v) Yes; by 92.2 %

✓ ANSWERS AT A GLANCE

(i) $(3, 5, 9)^\top, (-2, -3, -4)^\top$ (ii) 4.833 (iii) $(-3, -8)^\top$ (iv) $(1.3, 2.8)^\top, J = 0.378$ (v) yes, 92.2 %

! WHERE MARKS ARE LOST

- Writing $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ and then still subtracting in the update: the weights move uphill and J rises to about 15.
- Dividing the loss by N instead of $2N$ (gives 9.667), or leaving out the $\frac{1}{N}$ in the gradient (gives $(-9, -24)$ and a step of $(0.9, 2.4)$).
- Stopping at $\mathbf{w}^{(1)}$ without the new predictions: part (iv) needs $J(\mathbf{w}^{(1)})$.

Q3.3

Two features in matrix form, one update, then a prediction for a new house

8 marks Slides 12–14, 22–23 · Book 3, Ch. 2–4

A linear regression model predicts house price y (in ₹ million) from two features: x_1 = area in thousands of sq ft and x_2 = number of bedrooms. Four houses are given:

House	x_1 area	x_2 bedrooms	y price
1	1	1	4
2	2	2	6
3	3	2	9
4	2	3	8

The model is $\hat{y} = w_0 + w_1 x_1 + w_2 x_2$, the loss is $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, the initial weights are $\mathbf{w}^{(0)} = (0, 2, 1)^\top$ and $\eta = 0.1$.

- Write $\mathbf{X}$, $\mathbf{y}$ and $\mathbf{w}$ with their shapes. [1]
- Compute $\mathbf{X}\mathbf{w}^{(0)}$, the error vector and $J(\mathbf{w}^{(0)})$. [2]
- Write the gradient in matrix form and compute it. [2]

(iv) Update the weights and compute the new loss. [2]

(v) Using $\mathbf{w}^{(1)}$, predict the price of a new house with area 2.5 (thousand sq ft) and 3 bedrooms. [1]

by hand

matrix form

prediction

WHAT THIS TESTS

Slides 12–14 and 22: the design matrix with $d + 1 = 3$ columns, $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$, the gradient $\frac{1}{N}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ as three sums, and using the trained neuron on an unseen input.

HOW TO ATTEMPT

- Each row of $\mathbf{X}$ is $[1, x_1, x_2]$; $\mathbf{X}$ is 4×3 , $\mathbf{w}$ is 3×1 , so $\mathbf{X}\mathbf{w}$ is 4×1 .
- $\mathbf{X}^\top \mathbf{e}$ has three entries: $\sum e$, $\sum x_1 e$, $\sum x_2 e$. Divide by $N = 4$.
- For (v) put $[1, 2.5, 3]$ through the updated weights.

SOLUTION

STEP 1

(i)
$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 3 & 2 \\ 1 & 2 & 3 \end{bmatrix} \in \mathbb{R}^{4 \times 3}, \quad \mathbf{y} = \begin{bmatrix} 4 \\ 6 \\ 9 \\ 8 \end{bmatrix} \in \mathbb{R}^4, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} \in \mathbb{R}^3$$

$N = 4$ rows, $d = 2$ features, $d + 1 = 3$ columns

STEP 2

$\mathbf{X}\mathbf{w}^{(0)}: 0 + 2(1) + 1(1) = 3; 0 + 2(2) + 1(2) = 6; 0 + 2(3) + 1(2) = 8; 0 + 2(2) + 1(3) = 7.$

(ii) So $\hat{\mathbf{y}}^{(0)} = (3, 6, 8, 7)^\top$.

STEP 3

$\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = (3 - 4, 6 - 6, 8 - 9, 7 - 8)^\top = (-1, 0, -1, -1)^\top$

(ii)

STEP 4

$J(\mathbf{w}^{(0)}) = \frac{1}{2 \cdot 4}(1 + 0 + 1 + 1) = \frac{3}{8} = 0.375$

(ii)

STEP 5

Matrix form: $\nabla J = \frac{1}{N}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = \frac{1}{4}\mathbf{X}^\top \mathbf{e}$, where $\mathbf{X}^\top$ is 3×4 and $\mathbf{e}$ is 4×1 , so ∇J is 3×1 .

(iii)

STEP 6

(iii)
$$\mathbf{X}^\top \mathbf{e}^{(0)} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 2 \\ 1 & 2 & 2 & 3 \end{bmatrix} \begin{bmatrix} -1 \\ 0 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} -1 + 0 - 1 - 1 \\ -1 + 0 - 3 - 2 \\ -1 + 0 - 2 - 3 \end{bmatrix} = \begin{bmatrix} -3 \\ -6 \\ -6 \end{bmatrix}$$

STEP 7

(iii)
$$\nabla J(\mathbf{w}^{(0)}) = \frac{1}{4} \begin{bmatrix} -3 \\ -6 \\ -6 \end{bmatrix} = \begin{bmatrix} -0.75 \\ -1.5 \\ -1.5 \end{bmatrix}$$

STEP 8

(iv)
$$\mathbf{w}^{(1)} = \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.75 \\ -1.5 \\ -1.5 \end{bmatrix} = \begin{bmatrix} 0.075 \\ 2.15 \\ 1.15 \end{bmatrix}$$

STEP 9

$\mathbf{X}\mathbf{w}^{(1)}: 0.075 + 2.15 + 1.15 = 3.375; 0.075 + 4.3 + 2.3 = 6.675; 0.075 + 6.45 + 2.3 = 8.825;$
(iv) $0.075 + 4.3 + 3.45 = 7.825$

STEP 10

$\mathbf{e}^{(1)} = (3.375 - 4, 6.675 - 6, 8.825 - 9, 7.825 - 8)^\top = (-0.625, 0.675, -0.175, -0.175)^\top$

(iv)

STEP 11

$J(\mathbf{w}^{(1)}) = \frac{1}{8}(0.3906 + 0.4556 + 0.0306 + 0.0306) = \frac{0.9075}{8} = 0.113; \text{ a fall of } 69.7\%.$

(iv)

STEP 12

$\hat{y} = 0.075 + 2.15(2.5) + 1.15(3) = 0.075 + 5.375 + 3.45 = 8.9, \text{ so } ₹8.9 \text{ million} = ₹89 \text{ lakh.}$

(v)

with $\mathbf{w}^{(0)}$ the same house would get $0 + 5 + 3 = 8.0$

(i) $\mathbf{X}: 4 \times 3$ with a ones column; $\mathbf{y}: 4 \times 1$; $\mathbf{w}: 3 \times 1$

(ii) $\mathbf{X}\mathbf{w}^{(0)} = (3, 6, 8, 7)^\top, \mathbf{e}^{(0)} = (-1, 0, -1, -1)^\top, J = 0.375$

$$(iii) \nabla J = \frac{1}{4} \mathbf{X}^T \mathbf{e} = (-0.75, -1.5, -1.5)^T$$

$$(iv) \mathbf{w}^{(1)} = (0.075, 2.15, 1.15)^T, J(\mathbf{w}^{(1)}) = 0.113$$

$$(v) \hat{y} = 8.9 \text{ million rupees (₹89 lakh)}$$

ANSWERS AT A GLANCE

$$(i) \begin{matrix} 4 \times 3, 4 \times 1, \\ 3 \times 1 \end{matrix} \quad (ii) \begin{matrix} (3, 6, 8, 7)^T, J = \\ 0.375 \end{matrix} \quad (iii) \begin{matrix} (-0.75, -1.5, -1.5)^T \\ 0.113 \end{matrix} \quad (iv) \begin{matrix} (0.075, 2.15, 1.15)^T, \\ 0.113 \end{matrix} \quad (v) \begin{matrix} 8.9 \end{matrix}$$

WHERE MARKS ARE LOST

- Leaving out the ones column: $\mathbf{X}$ becomes 4×2 and the bias never gets a gradient.
- Mixing up rows and columns of $\mathbf{X}^T$: the second entry of $\mathbf{X}^T \mathbf{e}$ must use the area column (1, 2, 3, 2), not the bedroom column.
- Predicting the new house with $\mathbf{w}^{(0)}$ (8.0) instead of $\mathbf{w}^{(1)}$ (8.9).

Q3.4

MSE, RMSE, MAE and R^2 of a trained line on a test set

8 marks Slides 33–35 · Book 3, Ch. 6

A linear model was trained on delivery data and came out as $\hat{y} = 2x + 1$, where x is the distance in km and y is the delivery time in minutes. It is now evaluated on a held-out test set of five orders:

x (km)	1	2	3	4	5
y (min)	4	4.5	8	8	10.5

- Compute the five predictions $\hat{y}^{(i)}$ and the residuals $y^{(i)} - \hat{y}^{(i)}$. [2]
- Compute the test MSE, RMSE and MAE. [2]
- Compute $\bar{y}$, SS_{res} , SS_{tot} and R^2 . [3]
- A baseline model always predicts the mean $\bar{y}$. Compute its test MSE and its R^2 . Interpret the value of R^2 from (iii) in terms of "proportion of variance explained". [1] **justify**

by hand

evaluation metrics

R^2

WHAT THIS TESTS

The four formulas of slides 34–35, computed on unseen data: MSE with $\frac{1}{N}$ (no $\frac{1}{2}$), RMSE as its square root, MAE with absolute values, and $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$ with the meaning of $R^2 = 0$.

HOW TO ATTEMPT

- Make a five-column table: x , y , $\hat{y}$, residual, residual squared, |residual|.
- SS_{res} is the sum of the squared residuals, so $\text{MSE} = SS_{\text{res}}/N$. SS_{tot} uses $y - \bar{y}$.
- The baseline's residuals are exactly $y - \bar{y}$, so its $SS_{\text{res}} = SS_{\text{tot}}$.

Σ SOLUTION

x	y	$\hat{y} = 2x + 1$	$y - \hat{y}$	$(y - \hat{y})^2$	$ y - \hat{y} $	$y - \bar{y}$	$(y - \bar{y})^2$
1	4	3	1	1	1	-3	9
2	4.5	5	-0.5	0.25	0.5	-2.5	6.25
3	8	7	1	1	1	1	1
4	8	9	-1	1	1	1	1
5	10.5	11	-0.5	0.25	0.5	3.5	12.25
sum	35		0	3.5	4.0	0	29.5

STEP 1 Predictions: $2(1) + 1 = 3, 2(2) + 1 = 5, 2(3) + 1 = 7, 2(4) + 1 = 9, 2(5) + 1 = 11$.

(i) Residuals $y - \hat{y} = (1, -0.5, 1, -1, -0.5)$.

STEP 2 $MSE = \frac{1}{5}(1 + 0.25 + 1 + 1 + 0.25) = \frac{3.5}{5} = 0.7$

(ii)

STEP 3 $RMSE = \sqrt{0.7} = 0.837$ minutes

(ii) same unit as y

STEP 4 $MAE = \frac{1}{5}(1 + 0.5 + 1 + 1 + 0.5) = \frac{4.0}{5} = 0.8$ minutes

(ii)

STEP 5 $\bar{y} = \frac{1}{5}(4 + 4.5 + 8 + 8 + 10.5) = \frac{35}{5} = 7$

(iii)

STEP 6 $SS_{\text{res}} = \sum (y - \hat{y})^2 = 3.5$

(iii)

STEP 7 $SS_{\text{tot}} = \sum (y - \bar{y})^2 = 9 + 6.25 + 1 + 1 + 12.25 = 29.5$

(iii)

STEP 8 $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{3.5}{29.5} = 1 - 0.119 = 0.881$

(iii)

STEP 9 Baseline $\hat{y} = \bar{y} = 7$: its residuals are $y - \bar{y}$, so $SS_{\text{res}} = SS_{\text{tot}} = 29.5$, $MSE = 29.5/5 = 5.9$, and $R^2 = 1 - 29.5/29.5 = 0$.

(iv)

STEP 10 Key fact (slide 35): R^2 is the proportion of the variance of y that the model explains. $R^2 = 0.881$ means the line removes 88.1% of the baseline's squared error and leaves 11.9%; its MSE is $5.9/0.7 = 8.4$ times smaller than predicting the mean. The deck calls $0.7 < R^2 < 0.9$ a good fit.

(iv)

(i) $\hat{y} = (3, 5, 7, 9, 11)$; residuals $(1, -0.5, 1, -1, -0.5)$

(ii) $MSE = 0.7$, $RMSE = 0.837$, $MAE = 0.8$

(iii) $\bar{y} = 7$, $SS_{\text{res}} = 3.5$, $SS_{\text{tot}} = 29.5$, $R^2 = 0.881$

(iv) Baseline $MSE = 5.9$, $R^2 = 0$; the line explains 88.1 % of the variance

✓ ANSWERS AT A GLANCE

(i) $(1, -0.5, 1, -1, -0.5)$ (ii) $0.7, 0.837, 0.8$ (iii) $7, 3.5, 29.5, 0.881$ (iv) $5.9, 0; 88.1 \%$

! WHERE MARKS ARE LOST

- Using $\frac{1}{2N}$ for MSE: the metric on slide 34 has $\frac{1}{N}$ only (0.7, not 0.35).
- Taking $\sqrt{MSE}$ before averaging, or computing MAE from squared residuals.
- Using $\bar{\hat{y}}$ instead of $\bar{y}$ in SS_{tot} .
- Saying " $R^2 = 0.881$ means 88.1 % accuracy": it is a proportion of variance, not an accuracy.

Q3.5

Min-max and z-score scaling of two features, and why unscaled features zig-zag

8 marks Slides 40, 44 · Book 3, Ch. 7

Four flats are described by two raw features:

Flat	Area (sq ft)	Bedrooms
1	1000	1
2	1500	2
3	2000	2
4	2500	4

- (i) Apply min-max scaling $x' = \frac{x - \min}{\max - \min}$ to each column. [2]
- (ii) Apply z-score standardisation $x' = \frac{x - \mu}{\sigma}$ to each column, using the population σ . Show μ , the deviations, σ^2 and σ . [3]
- (iii) The curvature of J along weight w_j is the diagonal entry $\frac{1}{N} \sum_i (x_j^{(i)})^2$ of $\frac{1}{N} \mathbf{X}^\top \mathbf{X}$. Compute it for the raw area column and the raw bedroom column, and their ratio. What are the two entries after z-scoring? [2]
- (iv) Using the gradient formula $\nabla J = \frac{1}{N} \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$ and the ratio from (iii), justify in one or two lines why gradient descent zig-zags and converges slowly on the raw features. Give the largest safe η for the area direction and the factor $1 - \eta c$ it leaves for the bedroom direction.

[1] justify

by hand

feature scaling

justify

WHAT THIS TESTS

The two scaling formulas on slide 40 computed exactly, and the reason behind "features with different scales can slow convergence": the gradient entry for a feature is scaled by that feature's size, so the bowl is a long narrow valley.

HOW TO ATTEMPT

- Min-max: subtract the smallest value, divide by the range.
- Z-score: mean, then the four deviations, square them, average (divide by 4, population), square root.
- For (iii) square each raw value, add, divide by 4. After z-scoring, $\frac{1}{N} \sum z^2$ is the variance of z , which is 1.

SOLUTION

STEP 1 Area: min = 1000, max = 2500, range = 1500. $x' = \frac{0}{1500}, \frac{500}{1500}, \frac{1000}{1500}, \frac{1500}{1500} = 0, 0.333, 0.667, 1$

(i)

STEP 2 Bedrooms: min = 1, max = 4, range = 3. $x' = \frac{0}{3}, \frac{1}{3}, \frac{1}{3}, \frac{3}{3} = 0, 0.333, 0.333, 1$

(i)

STEP 3 Area: $\mu = \frac{1000+1500+2000+2500}{4} = \frac{7000}{4} = 1750$; deviations $(-750, -250, 250, 750)$

(ii)

STEP 4 $\sigma^2 = \frac{562,500+62,500+62,500+562,500}{4} = \frac{1,250,000}{4} = 312,500$, so $\sigma = \sqrt{312,500} = 559.017$

(ii)

STEP 5 $z_{\text{area}} = \frac{-750}{559.017}, \frac{-250}{559.017}, \frac{250}{559.017}, \frac{750}{559.017} = -1.342, -0.447, 0.447, 1.342$

(ii)

STEP 6 Bedrooms: $\mu = \frac{1+2+2+4}{4} = \frac{9}{4} = 2.25$; deviations $(-1.25, -0.25, -0.25, 1.75)$

(ii)

STEP 7 $\sigma^2 = \frac{1.5625+0.0625+0.0625+3.0625}{4} = \frac{4.75}{4} = 1.1875$, so $\sigma = \sqrt{1.1875} = 1.090$

(ii)

STEP 8 $z_{\text{bed}} = \frac{-1.25}{1.090}, \frac{-0.25}{1.090}, \frac{-0.25}{1.090}, \frac{1.75}{1.090} = -1.147, -0.229, -0.229, 1.606$

(ii) check: each z column has mean 0 and variance 1

STEP 9 Raw area: squares $1000^2, 1500^2, 2000^2, 2500^2 =$

(iii) $1,000,000, 2,250,000, 4,000,000, 6,250,000$

STEP 10 $\frac{1}{4}(1,000,000 + 2,250,000 + 4,000,000 + 6,250,000) = \frac{13,500,000}{4} = 3,375,000 = 3.375 \times 10^6$

(iii)

STEP 11 Raw bedrooms: $\frac{1}{4}(1 + 4 + 4 + 16) = \frac{25}{4} = 6.25$. Ratio $= 3,375,000/6.25 = 540,000 \approx 5 \times$

(iii) 10^5 , about a million to one.

STEP 12 After z -scoring, $\frac{1}{4} \sum z^2 = \text{var}(z) = 1$ for both columns: ratio 1, the bowl is round.

(iii)

STEP 13 Key fact: the j -th entry of ∇J is $\frac{1}{N} \sum_i e^{(i)} x_j^{(i)}$, so the same errors give an area gradient about 10^3 times larger and an area curvature 5.4×10^5 times larger. One η must fit both: the area direction needs $\eta < 2/c = 2/3.375 \times 10^6 = 5.9 \times 10^{-7}$, and at that η the bedroom gap shrinks by only $1 - \eta c = 1 - 5.9 \times 10^{-7} \times 6.25 = 0.9999963$ per step. So the steps bounce across the narrow area valley (zig-zag) while barely moving along the flat bedroom direction; the contour picture on slide 24 becomes a long thin ellipse. After scaling both curvatures are 1 and one η suits both.

(i) area $(0, 0.333, 0.667, 1)$; bedrooms $(0, 0.333, 0.333, 1)$

(ii) area: $\mu = 1750, \sigma = 559.017, z = (-1.342, -0.447, 0.447, 1.342)$; bedrooms: $\mu = 2.25, \sigma = 1.090, z = (-1.147, -0.229, -0.229, 1.606)$

(iii) 3.375×10^6 and 6.25, ratio 540,000; after z -scoring both are 1

(iv) gradient entries scale with the feature; safe $\eta \approx 5.9 \times 10^{-7}$ leaves the bedroom factor 0.9999963: zig-zag across, crawl along

✓ ANSWERS AT A GLANCE

(i) $(0, \frac{1}{3}, \frac{2}{3}, 1)$, (ii) $\sigma = 559.017, 1.090$, (iii) $3.375 \times 10^6, 6.25, \times 540,000$, (iv) $\eta < 5.9 \times 10^{-7}$, factor 0.9999963

! WHERE MARKS ARE LOST

- Dividing by $N - 1 = 3$ for the variance: the deck's σ is the population one (gives 645.5 and 1.258 instead).
- Using the range in the z -score, or the mean in min-max.
- In (iv) saying "big numbers are bad" without the gradient formula and the ratio: the mark is for $\frac{1}{N} \sum e x_j$ scaling with x_j .

Three recorded loss sequences: which

Q3.6 learning rate is which, and the safe range of η **8 marks** Slides 23, 39, 41 · Book 3, Ch. 4 and 7

The same linear regression was trained three times from the same start with three different learning rates. Nova recorded the loss J for the first five iterations of each run:

Iteration t	0	1	2	3	4
Run A	7.5	7.35	7.20	7.06	6.92
Run B	7.5	1.5	0.3	0.06	0.012
Run C	7.5	10.8	15.6	22.4	32.3

- (i) Compute the ratio $J^{(t+1)}/J^{(t)}$ for each run (to two decimals). Identify which run used a learning rate that is too small, a good one, and one that is too large. [3]
- (ii) For each run, state the action the deck's debugging checklist (slide 41) prescribes. [2]
- (iii) For a one-dimensional bowl $J(w) = \frac{c}{2}(w - w^*)^2$ with $c = 4$, each step multiplies the gap $w - w^*$ by $1 - \eta c$. Find the largest η for which gradient descent still converges, and the η that reaches w^* in one step. [2]
- (iv) If run C were on this bowl with $c = 4$, what learning rate was used? [1]

[read a table](#)
[learning rate](#)
[debugging checklist](#)

🔗 WHAT THIS TESTS

Reading a loss table the way slide 39 says ("plot loss vs iterations; if loss increases decrease η , if convergence is too slow increase η "), and the convergence condition $|1 - \eta c| < 1$ for the quadratic bowl.

▶ HOW TO ATTEMPT

- Divide each loss by the one before it. A ratio near 1 means crawling; well below 1 means fast; above 1 means growing.
- The loss is the gap squared, so the loss ratio is $(1 - \eta c)^2$. Converges when $|1 - \eta c| < 1$.
- For (iv) take the square root of run C's ratio and solve $1 - \eta c = -1.2$.

Σ SOLUTION

STEP 1 Run A: $7.35/7.5 = 0.98$, $7.20/7.35 = 0.98$, $7.06/7.20 = 0.98$, $6.92/7.06 = 0.98$. The loss loses 2% per step: η too small.

STEP 2 Run B: $1.5/7.5 = 0.20$, $0.3/1.5 = 0.20$, $0.06/0.3 = 0.20$, $0.012/0.06 = 0.20$. The loss drops to a fifth each step: good η .

STEP 3 Run C: $10.8/7.5 = 1.44$, $15.6/10.8 = 1.44$, $22.4/15.6 = 1.44$, $32.3/22.4 = 1.44$. The loss grows 44% per step: η too large (diverging).
at this rate run A needs about 100 steps to get below 1.0; run B did it in two

STEP 4 Run A ("loss not decreasing / too slow"): increase η ; also check the gradient sign and that the update uses minus. Run B: keep η ; stop with $\|\nabla J\| < \epsilon$ or $|J^{(t)} - J^{(t-1)}| < \epsilon$. Run C ("loss increasing, heading to NaN/Inf"): decrease η , scale the features, or use learning-rate decay.

STEP 5 Gap after t steps: $(1 - \eta c)^t (w^{(0)} - w^*)$. This shrinks only if $|1 - \eta c| < 1$, i.e. $-1 < 1 - 4\eta < 1$, i.e. $0 < \eta < 2/c = 2/4 = 0.5$. Largest converging η is just below 0.5.

STEP 6 One step: $1 - \eta c = 0 \Rightarrow \eta = 1/c = 1/4 = 0.25$.

STEP 7 Loss ratio $(1 - \eta c)^2 = 1.44 \Rightarrow |1 - \eta c| = 1.2$. Growth needs the negative root: $1 - 4\eta = -1.2 \Rightarrow \eta = 2.2/4 = 0.55$, which is above the limit 0.5.

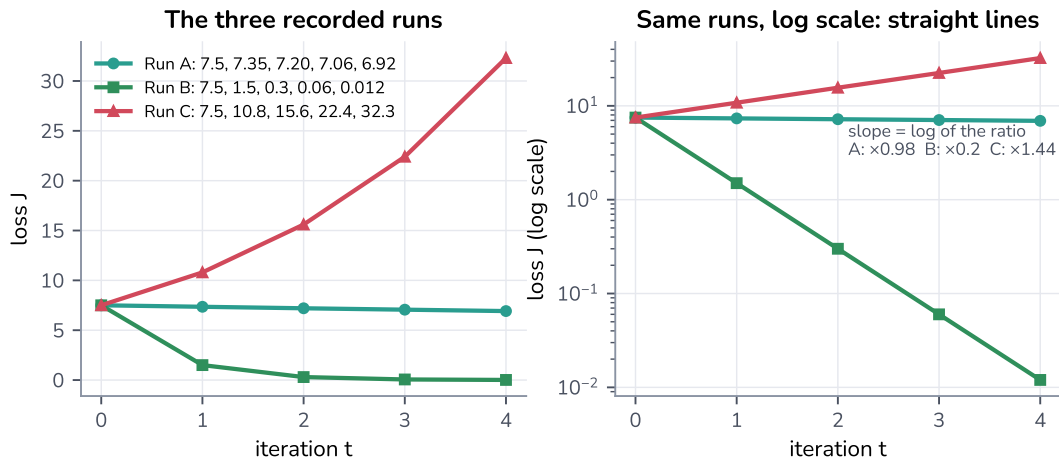


Figure 3.1 The three runs. On the log scale each run is a straight line; its slope is the log of the ratio you computed: A crawls ($\times 0.98$), B drops ($\times 0.2$), C explodes ($\times 1.44$).

(i) ratios 0.98 / 0.20 / 1.44; A too small, B good, C too large

(ii) A: increase η (check gradient sign); B: keep, use a stopping rule; C: decrease η or scale features

(iii) $\eta < 2/c = 0.5$; one step at $\eta = 1/c = 0.25$

(iv) $\eta = 0.55$

ANSWERS AT A GLANCE

(i) 0.98, 0.20, 1.44; small / good / large

(ii) $\uparrow\eta$ / keep / $\downarrow\eta$

(iii) 0.5, 0.25

(iv) 0.55

WHERE MARKS ARE LOST

- Calling run A "converged" because its loss goes down: the ratio 0.98 is the evidence that it is far too slow.
- Using the loss ratio directly as $1 - \eta c$: the loss goes as the gap squared, so take the square root first.
- Writing $\eta < 1/c$ for convergence: $1/c$ is the one-step rate; the limit is $2/c$.

Q3.7

Recover the learning rate from a recorded trajectory

8 marks Slides 17, 20–24 · Book 3, Ch. 4

Two flats of the same size, $x = 2$ (thousand sq ft), sold for $y = 3$ and $y = 5$ (units of ₹10 lakh). A neuron with no bias, $\hat{y} = wx$, is trained on them by batch gradient descent with the loss $J(w) = \frac{1}{2N} \sum_i (wx^{(i)} - y^{(i)})^2$. Nova's screen recorded $w^{(0)} = 0$, $w^{(1)} = 1.2$, $w^{(2)} = 1.68$. The learning rate was not written down.

- Expand $J(w)$ into the form $\frac{c}{2}(w - w^*)^2 + J^*$ and write $\frac{dJ}{dw}$. [2]
- Find the learning rate η from the first step, and verify it reproduces $w^{(2)} = 1.68$. [3]
- Predict $w^{(3)}$. [1]
- By what factor does the gap $w^* - w^{(t)}$ shrink each step? Find the number of steps after which the gap is below 0.01. [2]

by hand

recover a hyperparameter

gradient descent

WHAT THIS TESTS

Turning the update rule around: $w^{(1)} = w^{(0)} - \eta g^{(0)}$ gives η once you know the gradient at $w^{(0)}$. Also the geometric shrinking of the gap by $1 - \eta c$ and a logarithm to count steps.

HOW TO ATTEMPT

- $N = 2$, so $\frac{1}{2N} = \frac{1}{4}$. Expand the two squares and complete the square, or differentiate directly.
- Gradient at $w = 0$, then $\eta = (w^{(1)} - w^{(0)})/(-g^{(0)})$. Check with step two.
- Gap after t steps = $2(0.4)^t$; solve $2(0.4)^t < 0.01$ with logs and round up.

SOLUTION

STEP 1 $J(w) = \frac{1}{2 \cdot 2} [(2w - 3)^2 + (2w - 5)^2] = \frac{1}{4} [4w^2 - 12w + 9 + 4w^2 - 20w + 25]$

(i) $N = 2$, so $\frac{1}{2N} = \frac{1}{4}$

STEP 2 $= \frac{1}{4} [8w^2 - 32w + 34] = 2w^2 - 8w + 8.5$

(i)

STEP 3 $2w^2 - 8w + 8.5 = 2(w - 2)^2 + 0.5$, so $c = 4$, $w^* = 2$, $J^* = 0.5$.

(i)

$J^* = 0.5$ is not zero: one line cannot pass through both points

STEP 4 $\frac{dJ}{dw} = 4(w - 2)$

(i)

STEP 5 At $w^{(0)} = 0$: $g^{(0)} = 4(0 - 2) = -8$. Update: $w^{(1)} = 0 - \eta(-8) = 8\eta = 1.2 \Rightarrow \eta = 0.15$.

(ii)

STEP 6 Check: $g^{(1)} = 4(1.2 - 2) = -3.2$; $w^{(2)} = 1.2 - 0.15(-3.2) = 1.2 + 0.48 = 1.68 \checkmark$

(ii)

STEP 7 $g^{(2)} = 4(1.68 - 2) = -1.28$; $w^{(3)} = 1.68 + 0.15 \times 1.28 = 1.68 + 0.192 = 1.872$

(iii)

STEP 8 Gap: $w^* - w^{(t+1)} = (1 - \eta c)(w^* - w^{(t)}) = (1 - 0.6)(w^* - w^{(t)}) = 0.4(w^* - w^{(t)})$. Check: $2 \rightarrow$

(iv)

$0.8 \rightarrow 0.32 \rightarrow 0.128$.

STEP 9 $2(0.4)^t < 0.01 \Rightarrow (0.4)^t < 0.005 \Rightarrow t > \frac{\ln 0.005}{\ln 0.4} = \frac{-5.298}{-0.916} = 5.78$, so $t = 6$. Check: gap after 5 steps = $2(0.4)^5 = 0.0205$; after 6 steps = $0.0082 < 0.01$.

(iv)

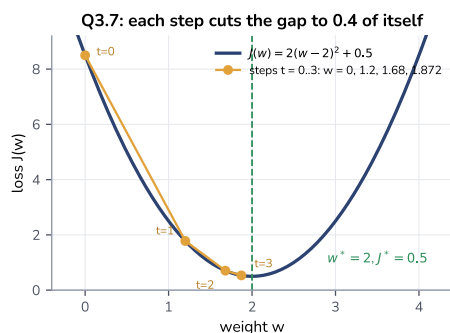


Figure 3.2 The bowl $J(w) = 2(w - 2)^2 + 0.5$ and the recorded steps.

Each step lands at 0.4 of the previous distance from $w^* = 2$; the bottom of the bowl is at 0.5, not 0.

(i) $J(w) = 2(w - 2)^2 + 0.5$; $dJ/dw = 4(w - 2)$

(ii) $\eta = 0.15$; second step gives 1.68 $\checkmark$

(iii) $w^{(3)} = 1.872$

(iv) factor 0.4; gap < 0.01 after $t = 6$ steps

✓ ANSWERS AT A GLANCE

- (i) $2(w-2)^2 + 0.5, 4(w-2)$ (ii) $\eta = 0.15$ (iii) 1.872 (iv) 0.4; 6 steps

! WHERE MARKS ARE LOST

- Forgetting the $\frac{1}{2N} = \frac{1}{4}$: the gradient becomes $16(w-2)$ and η comes out as 0.0375.
- Solving η from the second step without the gradient at $w^{(1)}$, or writing $w^{(1)} = 0 - \eta \cdot 8$ with the wrong sign.
- Rounding 5.78 down to 5 steps: at $t = 5$ the gap is still 0.0205.

Q3.8

Identity versus sigmoid output for a price target of 250

8 marks Slides 13–16 · Book 3, Ch. 2

A single neuron predicts a flat's price in thousands of rupees from one input $x_1 = \text{area in thousands of sq ft}$. Its weights are $w_0 = 10, w_1 = 110$. For a flat with $x_1 = 2$ the true price is $y = 250$. The per-example loss is $\ell = \frac{1}{2}(\hat{y} - y)^2$. Two versions of the neuron are compared: output $\hat{y} = f(z)$ with the identity $f(z) = z$, and with the sigmoid $f(z) = \sigma(z) = 1/(1 + e^{-z})$.

- (i) Compute z , then $\hat{y}$, the error $\hat{y} - y$ and the loss ℓ for the identity version. [2]
- (ii) Compute $\hat{y}$, the error and the loss for the sigmoid version. [2]
- (iii) What is the largest value the sigmoid neuron can ever output, and so the smallest error and smallest loss it can ever reach on this flat? For the identity neuron, find the w_1 that makes the error exactly zero. [2]
- (iv) The gradient of ℓ with respect to w_1 is $(\hat{y} - y) f'(z) x_1$. State $f'(z)$ for the identity and for the sigmoid ($\sigma'(z) = \sigma(z)(1 - \sigma(z))$), and compute $\partial\ell/\partial w_1$ for both versions at z from (i). Justify, in terms of $f'(z) = 1$, the deck's statement that the gradient flows unchanged through the identity neuron. [2] justify

by hand

identity activation

justify

⊙ WHAT THIS TESTS

Slides 15–16: the identity neuron outputs any real number and has $f'(z) = 1$; a sigmoid output is trapped in $(0, 1)$ and its slope $\sigma(1 - \sigma)$ is at most 0.25 and almost zero for large z , so the neuron cannot reach a price and cannot even learn towards it.

▶ HOW TO ATTEMPT

- $z = w_0 + w_1 x_1$ is the same for both versions; only f differs.
- $\sigma(230)$ is 1 to every decimal you can write; $1 - \sigma(230) = e^{-230}$ is about 10^{-100} .
- For the gradient, multiply error $\times$ slope $\times$ input on one line for each version.

Σ SOLUTION

STEP 1 $z = w_0 + w_1 x_1 = 10 + 110 \times 2 = 10 + 220 = 230$

(i)

STEP 2 Identity: $\hat{y} = f(230) = 230$; error $\hat{y} - y = 230 - 250 = -20$; $\ell = \frac{1}{2}(-20)^2 = \frac{1}{2} \times 400 = 200$

(i)

STEP 3 Sigmoid: $\hat{y} = \sigma(230) = \frac{1}{1 + e^{-230}} = 1.000$ (since $e^{-230} \approx 1.3 \times 10^{-100}$); error $= 1 - 250 = -249$; $\ell = \frac{1}{2}(249)^2 = \frac{1}{2} \times 62,001 = 31,000.5$

(ii)

- STEP 4** $\sigma(z) < 1$ for every z , so the largest output is 1 (thousand rupees). Smallest possible error = $1 - 250 = -249$, smallest possible loss = 31,000.5: no choice of weights can do better. the target 250 is outside the sigmoid's range $(0, 1)$
- STEP 5** Identity: $10 + w_1 \times 2 = 250 \Rightarrow w_1 = \frac{250 - 10}{2} = 120$. Then $\hat{y} = 250$, error 0, loss 0.
- STEP 6** Identity: $f'(z) = 1$ for every z . $\partial\ell/\partial w_1 = (\hat{y} - y) \cdot 1 \cdot x_1 = (-20)(1)(2) = -40$ (and $\partial\ell/\partial w_0 = -20$). A step $w_1 \leftarrow w_1 + 40\eta$ moves the prediction towards 250.
- STEP 7** Sigmoid: $\sigma'(230) = \sigma(230)(1 - \sigma(230)) \approx 1 \times 1.3 \times 10^{-100} \approx 0$. $\partial\ell/\partial w_1 = (-249)(1.3 \times 10^{-100})(2) \approx -6 \times 10^{-98} \approx 0$: the error is huge but the weight does not move.
- STEP 8** Key fact (slide 16): with $f'(z) = 1$ the chain rule gives $\partial\ell/\partial w_j = (\hat{y} - y)x_j$ exactly, the error signal reaches the weights unchanged, with no shrinking factor. A sigmoid multiplies it by $\sigma(1 - \sigma) \leq 0.25$, which is essentially zero once $|z|$ is large.

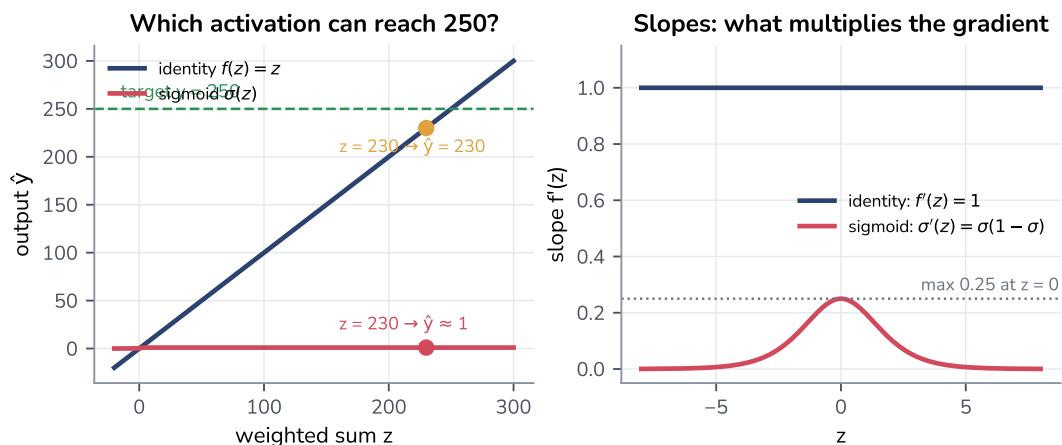


Figure 3.3 Left: the identity output climbs to any price; the sigmoid is flat at 1, far below the target 250. Right: the slopes. Identity has slope 1 everywhere; the sigmoid's slope peaks at 0.25 and is zero for large $|z|$.

- (i) $z = 230$; $\hat{y} = 230$, error -20 , $\ell = 200$
- (ii) $\hat{y} = 1.000$, error -249 , $\ell = 31,000.5$
- (iii) max output 1; best possible error -249 , loss 31,000.5; identity needs $w_1 = 120$
- (iv) $f' = 1$ vs $\sigma' \approx 10^{-100}$; gradients -40 vs ≈ 0 ; with $f' = 1$ the error reaches the weights unchanged

ANSWERS AT A GLANCE

- (i) 230, -20 , 200
- (ii) 1, -249 , 31 000.5
- (iii) 1; -249 ; $w_1 = 120$
- (iv) -40 vs ≈ 0

WHERE MARKS ARE LOST

- Writing $\sigma(230) = 230$ or "very large": the sigmoid output is 1 to every decimal.
- Forgetting the $\frac{1}{2}$ in ℓ (400 instead of 200).
- In (iv) leaving out $x_1 = 2$: the gradient is -40 , not -20 .
- Saying "sigmoid is for classification" without the number: the mark is for the range $(0, 1)$ against the target 250 and the slope ≈ 0 .

Q3.9

Train/test split, which stopping rule fires first, and early stopping

8 marks Slides 33, 39 · Book 3, Ch. 6–7

A regression dataset has $N = 5,000$ examples. It is split 90 % / 10 % into training and test sets. Batch gradient descent is run on the training set with tolerance $\epsilon = 10^{-4}$; one iteration (one gradient over the whole training set and one update) takes 2 ms. Nova logged the gradient size and the change in loss:

t	$\ \nabla J\ $	$ J^{(t)} - J^{(t-1)} $	t	$\ \nabla J\ $	$ J^{(t)} - J^{(t-1)} $
0	8.00	–	8	0.00524	1.80×10^{-5}
1	3.20	6.72	9	0.00210	2.89×10^{-6}
2	1.28	1.08	10	0.000839	4.62×10^{-7}
3	0.512	0.172	11	0.000336	7.39×10^{-8}
4	0.205	0.0275	12	0.000134	1.18×10^{-8}
5	0.0819	0.00440	13	5.37×10^{-5}	1.89×10^{-9}
6	0.0328	0.000705	14	2.15×10^{-5}	3.03×10^{-10}
7	0.0131	0.000113			

Separately, a second model is trained for 8 epochs with a validation set, and the losses are:

Epoch	1	2	3	4	5	6	7	8
Training loss	5.20	3.10	2.05	1.50	1.20	1.00	0.86	0.75
Validation loss	5.40	3.30	2.30	1.80	1.62	1.66	1.75	1.90

- How many examples are in the training set and in the test set? [1]
- With $\epsilon = 10^{-4}$, at which iteration does the rule $\|\nabla J\| < \epsilon$ stop the run, and at which iteration does the rule $|J^{(t)} - J^{(t-1)}| < \epsilon$ stop it? Which fires first? [3]
- Compute the training time under each of the two rules, and the time if only a maximum of $T = 1,000$ iterations were used. [1]
- For the second model, find the epoch at which early stopping should stop, and justify it using the validation curve: give the validation minimum, the epoch from which it rises, and the gap between validation and training loss at the stopping epoch and at epoch 8. [3] **justify**

read a table

stopping criteria

early stopping

🔗 WHAT THIS TESTS

The split percentages on slide 33 and the four stopping rules on slide 39, each applied exactly to a log: the loss-change rule fires earlier than the gradient rule (the loss change goes as the gradient squared), and early stopping is triggered by the validation loss turning upward while the training loss still falls.

▶ HOW TO ATTEMPT

- Read down each column and mark the first row strictly below $10^{-4} = 0.0001$.
- Time = iterations $\times$ 2 ms.
- Find the smallest validation loss; the epoch after it is the first rise. Subtract training from validation at epochs 5 and 8.

Σ SOLUTION

STEP 1 Training = $0.9 \times 5,000 = 4,500$; test = $5,000 - 4,500 = 500$.

(i)

STEP 2 Gradient rule: at $t = 12$, $\|\nabla J\| = 0.000134 > 10^{-4}$; at $t = 13$, $5.37 \times 10^{-5} < 10^{-4}$. Stops at $t = 13$.

STEP 3 Loss-change rule: at $t = 7$, $|\Delta J| = 0.000113 > 10^{-4}$; at $t = 8$, $1.80 \times 10^{-5} < 10^{-4}$. Stops at $t = 8$.

STEP 4 The loss-change rule fires first, 5 iterations earlier.

(ii) the gradient shrinks by $\times 0.4$ per step, the loss change by $\times 0.16$: the square

STEP 5 Gradient rule: $13 \times 2 \text{ ms} = 26 \text{ ms}$. Loss-change rule: $8 \times 2 \text{ ms} = 16 \text{ ms}$. Max-iterations only: $1,000 \times 2 \text{ ms} = 2,000 \text{ ms} = 2 \text{ s}$, about 77 times longer than needed.

STEP 6 Validation loss: 5.40, 3.30, 2.30, 1.80, **1.62**, 1.66, 1.75, 1.90. Minimum 1.62 at epoch 5; it rises from epoch 6 onward ($1.66 > 1.62$) while the training loss keeps falling ($1.20 \rightarrow 0.75$).

STEP 7 Gaps (validation – training): epoch 5: $1.62 - 1.20 = 0.42$; epoch 8: $1.90 - 0.75 = 1.15$.
(iv) From epoch 5 to 8 training falls by 0.45 but validation rises by 0.28: the model is fitting the training set, not generalising.

STEP 8 Key fact (slide 39): "validation loss starts increasing $\rightarrow$ early stopping". Stop at epoch 5 and keep those weights.

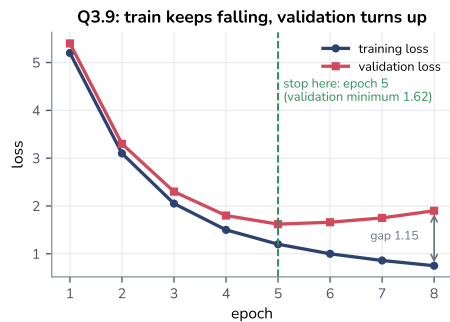


Figure 3.4 The second model. Training loss falls all the way; validation loss bottoms out at epoch 5 (1.62) and then turns up. The gap between the curves grows from 0.42 to 1.15.

(i) 4,500 training, 500 test

(ii) $\|\nabla J\| < \epsilon$ at $t = 13$; $|\Delta J| < \epsilon$ at $t = 8$; the loss-change rule fires first

(iii) 26 ms, 16 ms, 2 s

(iv) stop at epoch 5: validation minimum 1.62, rising from epoch 6; gap 0.42 $\rightarrow$ 1.15

✓ ANSWERS AT A GLANCE

(i) 4500 / 500 **(ii)** 13; 8; loss-change first **(iii)** 26 ms, 16 ms, 2 s **(iv)** epoch 5; 1.62; 0.42 $\rightarrow$ 1.15

! WHERE MARKS ARE LOST

- Reading 0.000134 as below 10^{-4} : $1.34 \times 10^{-4} > 1.0 \times 10^{-4}$, so the gradient rule waits for $t = 13$.
- Stopping the second model at epoch 6 (the first rise) instead of epoch 5 (the minimum): the weights to keep are those at the minimum.
- Justifying early stopping with the training curve: the training loss never rises; only the validation curve carries the signal.

A Keras regression snippet: nature,

Q3.10

output layer, parameters, loss, updates, inference

8 marks Slides 13–17, 39 · Book 3, Ch. 2–3 and 7

Consider the following Python code snippet, written to predict the resale price of a used car (in thousands of rupees) from five features: age (years), kilometres driven, engine size (cc), number of previous owners and mileage (km/l). The training set has 3,200 cars.

```
model = Sequential()
model.add(Dense(16, input_dim=5, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='linear'))
model.compile(loss='mse', optimizer='sgd', metrics=['mae'])
model.fit(X_train, y_train, epochs=30, batch_size=32)
model.evaluate(X_test, y_test)
```

- (i) State the nature of the problem (binary classification / multi-class classification / regression), giving the two lines of code that show it. [1]
- (ii) Why is a linear (identity) activation used in the output layer instead of sigmoid or softmax? Answer with the output range of each. [1] **justify**
- (iii) Calculate the number of trainable parameters (weights + biases) for each layer and the total. [3]
- (iv) Why is MSE chosen as the loss? Name the properties from the deck. [1] **justify**
- (v) How many weight updates happen in one epoch, and in the whole training run? [1]
- (vi) The trained model outputs 312.5 for a car. Give the inference of this output. [1]

count parameters

regression head

justify

Ⓢ WHAT THIS TESTS

Reading a model definition the way the mid-sem does: the last layer and the loss tell you the task; parameters per Dense layer are (inputs × units) + units; an epoch is one pass over the data, made of $N/\text{batch size}$ updates; and a regression output is a number in the target's unit, not a probability.

▶ HOW TO ATTEMPT

- Layer transitions: $5 \rightarrow 16 \rightarrow 8 \rightarrow 1$. For each, weights = in × out, biases = out.
- Updates per epoch = $3200/32$; multiply by 30 epochs.
- For (ii) and (iv) write the exact facts from slides 16 and 18.

Σ SOLUTION

STEP 1 Regression: the target (price) is a continuous number. The evidence is `Dense(1, activation='linear')` (one real-valued output) and `loss='mse'`.

STEP 2 Key fact (slide 16): the identity output is continuous on $(-\infty, \infty)$, so it can equal any price. A sigmoid output lies in $(0, 1)$: it could never output 312.5. Softmax on a single unit always outputs exactly 1, and on several units gives probabilities that sum to 1, which is for choosing a class, not for a quantity. (Q3.8 shows the numbers.)

STEP 3 Layer 1 ($5 \rightarrow 16$): weights $5 \times 16 = 80$, biases 16, total 96

(iii)

STEP 4 Layer 2 ($16 \rightarrow 8$): weights $16 \times 8 = 128$, biases 8, total 136

(iii)

STEP 5 Layer 3 ($8 \rightarrow 1$): weights $8 \times 1 = 8$, biases 1, total 9

(iii)

STEP 6 Total = $96 + 136 + 9 = 241$ trainable parameters (216 weights + 25 biases).

(iii)

STEP 7 Key facts (slide 18): MSE is differentiable (smooth, so gradient descent works), convex for a linear model (single global minimum), penalises large errors quadratically, and equals maximum likelihood under Gaussian noise. It measures distance between two numbers, which is what a regression needs; cross-entropy needs probabilities.

(iv)

STEP 8 Updates per epoch = $3200/32 = 100$ mini-batches. Whole run = $100 \times 30 = 3,000$ updates; each car is seen 30 times.

(v)

STEP 9 The output is a predicted price in the target's unit: 312.5 thousand rupees = ₹3,12,500 (about ₹3.1 lakh). It is not a probability and there is no threshold; its typical error is the test MAE printed by `evaluate`.

(vi)

(i) regression: `Dense(1, 'linear')` and `loss='mse'`

(ii) identity range $(-\infty, \infty)$; sigmoid $(0, 1)$ cannot reach 312.5; softmax on one unit is always 1

(iii) $96 + 136 + 9 = 241$

(iv) differentiable, convex, penalises big errors, Gaussian MLE

(v) 100 per epoch; 3,000 in total

(vi) predicted price ₹3,12,500; a quantity, not a probability

✓ ANSWERS AT A GLANCE

(i) regression	(ii) $(-\infty, \infty)$ vs $(0, 1)$ vs sums to 1	(iii) 96, 136, 9; 241	(iv) 4 properties	(v) 100; 3000	(vi) ₹3,12,500
----------------	---	-----------------------	-------------------	---------------	----------------

! WHERE MARKS ARE LOST

- Forgetting the biases (216 instead of 241), or counting a bias for the input layer.
- Answering (v) with "30 updates": an epoch has $3200/32 = 100$ mini-batch updates.
- Interpreting 312.5 as "31,250 % confidence" or thresholding it at 0.5: a regression output is a number in the target's unit.

Module 4: Linear Neural Networks for Classification

Ten mid-sem-style questions on one neuron that answers "which one": softmax and cross-entropy with the gradient, a logistic-regression update with two learning rates, two SGD steps with a trace table, confusion matrices for two and three classes, parameter counts and mini-batch arithmetic, one mini-batch step in matrix form, a Keras classifier, the softmax properties with numbers, and top-K accuracy.

On your slides: **Module 4, slides 4–95** · sigmoid neuron $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$ · decision rule $\hat{y} \geq 0.5 \Leftrightarrow \mathbf{w}^\top \mathbf{x} \geq 0$ · binary cross-entropy · SGD with gradient $(\hat{y} - y)\mathbf{x}$ · confusion matrix, accuracy, precision, recall, F1 · one-hot labels · $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$, logits $\mathbf{Z} = \mathbf{XW}$ · softmax · categorical cross-entropy · mini-batch SGD $\frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$ · inference by $\arg \max$ · $K \times K$ confusion matrix, macro and weighted averages, top-K · stable softmax

Module 4 is where Nova learns to choose. For two classes the neuron of Module 3 gets a sigmoid on its output: $z = \mathbf{w}^\top \mathbf{x}$ with $x_0 = 1$, $\hat{y} = \sigma(z) = P(y = 1 \mid \mathbf{x})$, predict 1 when $\hat{y} \geq 0.5$, which is the same as $\mathbf{w}^\top \mathbf{x} \geq 0$. The loss is binary cross-entropy $\ell = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$, and stochastic gradient descent uses one example at a time with the gradient $(\hat{y} - y)\mathbf{x}$. For K classes the labels become one-hot rows of $\mathbf{Y} \in \{0, 1\}^{N \times K}$, the weights become a matrix $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$, the logits are $\mathbf{Z} = \mathbf{XW}$, softmax turns each row into probabilities that sum to 1, the loss is categorical cross-entropy $-\frac{1}{N} \sum_i \sum_k y_k^{(i)} \ln \hat{y}_k^{(i)}$, and mini-batch SGD updates with $\frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$, $\lceil N/B \rceil$ times per epoch. Evaluation uses the confusion matrix (rows actual, columns predicted), accuracy, precision, recall, F1, macro and weighted averages, and top-K accuracy. Every question below is built like the mid-sem paper: a scenario with all numbers given, a numeric core to compute completely, then one to three short pointed parts with one specific expected answer each. Logs are natural logs throughout, as in the deck.

Q	Scenario	Numeric / pointed marks	Slides	Marks
4.1	Fraud detector, 4 classes, logits (2.0, 0.5, -1.0, 1.0): softmax, predicted class, cross-entropy for the 4th class, gradient $\hat{\mathbf{y}} - \mathbf{y}$; weights of a $25 \rightarrow 16 \rightarrow 8 \rightarrow 4$ network	7 / 1	54–61	8
4.2	Loan-default neuron: $z, \hat{y}$, BCE, gradient $(\hat{y} - y)\mathbf{x}$, update with $\eta = 0.1$ and $\eta = 2.0$, which is larger, the two forms of the decision rule	6 / 2	19–21, 26–27	8
4.3	Hours and attendance $\rightarrow$ pass: two SGD steps with a trace table, probabilities for all four rows, how many are on the right side of 0.5, mean loss before and after	7 / 1	31–35, 29	8
4.4	Spam filter with TP 90, FP 30, FN 10, TN 870: accuracy, precision, recall, F1, specificity; "always ham" baseline; which metric when FP or FN are costly	6 / 2	39–43	8
4.5	3×3 sentiment confusion matrix: accuracy, per-class P/R/F1, macro-F1, weighted-F1, the biggest confusion, which average when	7 / 1	84–87	8

4.6	Sentiment net $500 \rightarrow 16 \rightarrow 5$: parameters per transition, with biases; $N = 12,000$, $B = 64$: iterations per epoch, updates in 15 epochs; hidden and output activations	6 / 2	54, 65–71, 94	8
4.7	One mini-batch step by hand, $B = 2$, $K = 3$: $\mathbf{Z}_B$, softmax rows, $\hat{\mathbf{Y}} - \mathbf{Y}$, gradient, $\mathbf{W}^{(1)}$ with $\eta = 0.5$, shapes of every matrix, loss before and after	8 / 0	52–57, 68–70	8
4.8	Keras snippet $12 \rightarrow 32 \rightarrow 16 \rightarrow 1$ (sigmoid): architecture, nature, why sigmoid and when softmax, parameters, iterations and updates, interpret 0.27	5 / 3	18–21, 67, 94–95	8
4.9	Softmax properties with numbers: z and $z - 3$, $2z$, and $K = 2$ with $z = (1.5, 0)$ against $\sigma(1.5)$; name the property each shows	6 / 2	57–58, 91	8
4.10	Five-class model on four test images: top-1 and top-2 accuracy, mean cross-entropy, the "predict only if max ≥ 0.6 " rule	7 / 1	60, 81–82, 88	8

Q4.1 Softmax, cross-entropy and its gradient for a fraud detector; weights of a four-layer network

8 marks Slides 54–61 · Book 4, Ch. 5–6

A payment company classifies each card transaction into 4 categories: [Card-not-present fraud, Account takeover, Merchant fraud, Legitimate]. For one transaction the output layer produces the logits $\mathbf{z} = [2.0, 0.5, -1.0, 1.0]$.

- Compute the softmax probability of each category, to 3 decimal places. Show the four exponentials and their sum. [2]
- Which category does the model predict, and with what confidence? [1]
- The true class is Legitimate (the 4th). Write the one-hot label and compute the cross-entropy loss. [1]
- Compute the gradient of the loss with respect to the logits, $\partial\ell/\partial\mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$. State the sum of its four entries and which entry is negative, and say why in one line. [2]
- The detector is a fully connected network with Input 25 $\rightarrow$ Hidden 16 $\rightarrow$ Hidden 8 $\rightarrow$ Output 4. Compute the total number of trainable weights, biases excluded, showing each layer transition. Then give the total with biases. [2]

by hand

softmax + cross-entropy

count parameters

WHAT THIS TESTS

The softmax formula of slide 57 computed exactly, $\arg \max$ as the prediction (slide 60), the categorical cross-entropy of slide 61 (only the true class's probability matters), the gradient $\hat{\mathbf{y}} - \mathbf{y}$ that the mini-batch formula of slide 68 is built on, and the weight count (in $\times$ out) per transition.

HOW TO ATTEMPT

- Exponentiate all four logits, add, divide each by the sum. Check they add to 1.
- The loss uses only $\hat{y}_4$: $-\ln \hat{y}_4$. Natural log.
- The gradient is the probability vector with 1 subtracted at the true class.
- Weights: 25×16 , 16×8 , 8×4 ; biases: one per neuron after the input.

Σ SOLUTION

STEP 1 $e^{2.0} = 7.389$, $e^{0.5} = 1.649$, $e^{-1.0} = 0.368$, $e^{1.0} = 2.718$

(i)

STEP 2 Sum = $7.389 + 1.649 + 0.368 + 2.718 = 12.124$

(i)

STEP 3 $\hat{y}_1 = \frac{7.389}{12.124} = 0.609$, $\hat{y}_2 = \frac{1.649}{12.124} = 0.136$, $\hat{y}_3 = \frac{0.368}{12.124} = 0.030$, $\hat{y}_4 = \frac{2.718}{12.124} = 0.224$

(i)

check: $0.609 + 0.136 + 0.030 + 0.224 = 0.999 \approx 1$ (rounding)

STEP 4 $\hat{k} = \arg \max_k \hat{y}_k = 1$: Card-not-present fraud, with confidence 0.609 (60.9 %).

(ii)

STEP 5 One-hot label $\mathbf{y} = [0, 0, 0, 1]$. Loss $\ell = -\sum_k y_k \ln \hat{y}_k = -\ln \hat{y}_4 = -\ln 0.224 = 1.495$

(iii)

the model gave the true class only 22.4 %, so the loss is large; a perfect model would give $-\ln 1 = 0$

STEP 6 $\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} = [0.609, 0.136, 0.030, 0.224 - 1] = [0.609, 0.136, 0.030, -0.776]$

(iv)

STEP 7 Sum of entries = $0.609 + 0.136 + 0.030 - 0.776 = 0$ (exactly, since $\sum \hat{y}_k = 1$ and $\sum y_k = 1$). Only the true-class entry is negative: gradient descent pushes z_4 up and every other logit down.

(iv)

STEP 8 $25 \rightarrow 16: 25 \times 16 = 400; 16 \rightarrow 8: 16 \times 8 = 128; 8 \rightarrow 4: 8 \times 4 = 32$

(v)

STEP 9 Total weights = $400 + 128 + 32 = 560$. Biases = $16 + 8 + 4 = 28$, so with biases $560 + 28 = 588$.

(v)

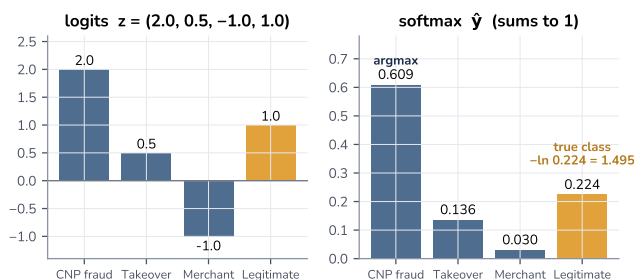


Figure 4.1 Logits on the left, probabilities on the right. The exponential turns a gap of 1.0 between z_1 and z_4 into a ratio of $e \approx 2.7$ between $\hat{y}_1$ and $\hat{y}_4$. The loss looks only at the amber bar.

(i) $\hat{\mathbf{y}} = (0.609, 0.136, 0.030, 0.224)$

(ii) Class 1, Card-not-present fraud, 0.609

(iii) $\mathbf{y} = (0, 0, 0, 1)$; $\ell = -\ln 0.224 = 1.495$

(iv) $(0.609, 0.136, 0.030, -0.776)$; sum 0; only the true class is negative

(v) $400 + 128 + 32 = 560$ weights; 588 with biases

✓ ANSWERS AT A GLANCE

(i) 0.609, 0.136, 0.030, 0.224 (ii) class 1, 0.609 (iii) 1.495 (iv) $(0.609, 0.136, 0.030, -0.776)$, sum 0 (v) 560; 588

! WHERE MARKS ARE LOST

- Taking the loss at the predicted class: $-\ln 0.609 = 0.495$ is wrong; the loss uses the true class.
- Using $\log_{10}$: $-\log_{10} 0.224 = 0.649$. The deck's cross-entropy is a natural log.

- Writing the gradient as $\mathbf{y} - \hat{\mathbf{y}}$ (all signs flipped), or forgetting to subtract 1 at the true class.
- Counting the input layer's 25 nodes as having biases: only neurons after the input have a bias (28, not 53).

One SGD update of a logistic-regression

Q4.2 neuron with a small and a large learning rate

8 marks Slides 19–21, 26–27 · Book 4, Ch. 2–3

A bank trains a single sigmoid neuron to predict whether a loan applicant will default ($y = 1$) from three normalised features: x_1 debt-to-income ratio, x_2 years at current job, x_3 number of late payments. The current weights, including the bias w_0 , are $\mathbf{w} = [0.3, 1.5, -1.0, 2.0]$. One training example is $\mathbf{x} = [1, 0.4, 0.5, 0.2]$ (the leading 1 is for the bias) with label $y = 0$ (did not default).

- Compute the logit $z = \mathbf{w}^\top \mathbf{x}$, the prediction $\hat{y} = \sigma(z)$ to 3 decimals, and the predicted class. Is it correct? [2]
- Compute the binary cross-entropy loss of this example. [1]
- Compute the gradient $\nabla \ell = (\hat{y} - y) \mathbf{x}$. [1]
- Perform one SGD update with $\eta = 0.1$; show all four new weights. Repeat from the same initial weights with $\eta = 2.0$. For each, recompute z and $\hat{y}$ on the same example. [2]
- Which learning rate gives the larger weight update, and by what factor? Which of the two updates changes the predicted class? [1]
- State the decision rule in its two equivalent forms from the deck, and give the one-line reason they are equivalent. Which form does the sign of z in (iv) illustrate? [1] **justify**

by hand

SGD update

learning rate

WHAT THIS TESTS

The whole binary pipeline of slides 19–27 on one example: weighted sum, sigmoid, threshold, BCE, the gradient $(\hat{y} - y)\mathbf{x}$ (slide 26), the update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell$, the fact that the step scales with η , and the equivalence $\hat{y} \geq 0.5 \Leftrightarrow \mathbf{w}^\top \mathbf{x} \geq 0$ (slide 20).

HOW TO ATTEMPT

- Four products, add, then $\sigma(z) = 1/(1 + e^{-z})$.
- With $y = 0$ the loss is $-\ln(1 - \hat{y})$ and the error $\hat{y} - y$ is just $\hat{y}$.
- The gradient is the error times each x_j , including $x_0 = 1$. Subtract η times it.
- After each update put $\mathbf{x}$ through the new weights again to see where z landed.

SOLUTION

STEP 1 $z = 0.3(1) + 1.5(0.4) + (-1.0)(0.5) + 2.0(0.2) = 0.3 + 0.6 - 0.5 + 0.4 = 0.8$

(i) **STEP 2** $\hat{y} = \sigma(0.8) = \frac{1}{1 + e^{-0.8}} = \frac{1}{1 + 0.449} = 0.690$

(i) **STEP 3** $0.690 \geq 0.5$, so the predicted class is 1 (default). The true label is 0: the prediction is wrong.

(ii) **STEP 4** $\ell = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})] = -\ln(1 - 0.690) = -\ln 0.310 = 1.171$
a confident wrong answer costs more than $-\ln 0.5 = 0.693$

(iii) **STEP 5** Error $\hat{y} - y = 0.690 - 0 = 0.690$. $\nabla \ell = 0.690 \times [1, 0.4, 0.5, 0.2] = [0.690, 0.276, 0.345, 0.138]$

- STEP 6** $\eta = 0.1$: step = [0.069, 0.028, 0.035, 0.014]; $w_0 = 0.3 - 0.069 = 0.231$; $w_1 = 1.5 - 0.028 = 1.472$; $w_2 = -1.0 - 0.035 = -1.034$ (-1.0345); $w_3 = 2.0 - 0.014 = 1.986$
(iv)
- STEP 7** New $z = 0.231 + 1.472(0.4) - 1.034(0.5) + 1.986(0.2) = 0.700$; $\hat{y} = \sigma(0.700) = 0.668$; still class 1.
(iv)
- STEP 8** $\eta = 2.0$: step = [1.380, 0.552, 0.690, 0.276]; $w_0 = 0.3 - 1.380 = -1.080$; $w_1 = 1.5 - 0.552 = 0.948$; $w_2 = -1.0 - 0.690 = -1.690$; $w_3 = 2.0 - 0.276 = 1.724$
(iv)
- STEP 9** New $z = -1.080 + 0.948(0.4) - 1.690(0.5) + 1.724(0.2) = -1.201$; $\hat{y} = \sigma(-1.201) = 0.231$; now class 0.
(iv)
- STEP 10** The step is $\eta \nabla \ell$, so $\eta = 2.0$ gives an update $2.0/0.1 = 20$ times larger in every entry (step lengths 0.083 and 1.662). Only the $\eta = 2.0$ update changes the predicted class (1 $\rightarrow$ 0); the small step leaves $\hat{y}$ at 0.668 on the wrong side.
(v)
- STEP 11** Slide 20: predict 1 if $\hat{y} \geq 0.5$, else 0; equivalently predict 1 if $\mathbf{w}^\top \mathbf{x} \geq 0$, else 0. They are the same rule because $\sigma(0) = 0.5$ and σ is increasing, so $\sigma(z) \geq 0.5 \Leftrightarrow z \geq 0$; the boundary is the hyperplane $\mathbf{w}^\top \mathbf{x} = 0$. Part (iv) shows the second form: $z = 0.700 > 0$ gives class 1, $z = -1.201 < 0$ gives class 0, without evaluating σ .
(vi)

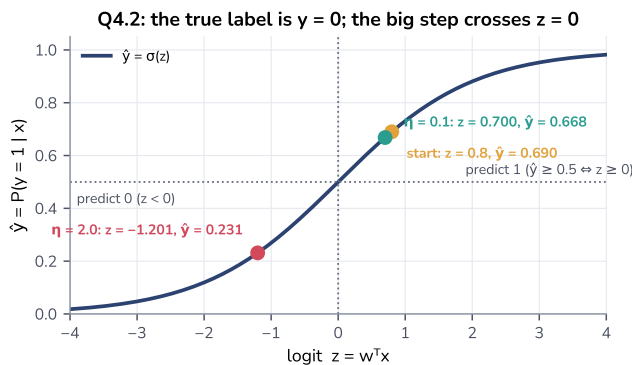


Figure 4.2 The three points on the sigmoid. The small step moves z from 0.8 to 0.700; the big step jumps to -1.201 , across $z = 0$, so the class flips to 0.

(i) $z = 0.8, \hat{y} = 0.690$, class 1, wrong

(ii) $\ell = -\ln 0.310 = 1.171$

(iii) $\nabla \ell = (0.690, 0.276, 0.345, 0.138)$

(iv) $\eta = 0.1$: (0.231, 1.472, -1.034, 1.986), $z = 0.700, \hat{y} = 0.668$; $\eta = 2.0$: (-1.080, 0.948, -1.690, 1.724), $z = -1.201, \hat{y} = 0.231$

(v) $\eta = 2.0$, 20 times; only it flips the class to 0

(vi) $\hat{y} \geq 0.5 \Leftrightarrow \mathbf{w}^\top \mathbf{x} \geq 0$ because $\sigma(0) = 0.5$; (iv) uses the sign of z

ANSWERS AT A GLANCE

(i) 0.8, 0.690, (ii) 1.171 (iii) (0.690, 0.276, 0.345, 0.138) (iv) $\hat{y}$: 0.668 / 0.231 (v) $\eta = 2.0$, $\times 20$ (vi) $\sigma(0) = 0.5$

WHERE MARKS ARE LOST

- Using $y = 1$ in the error: the gradient becomes $-0.310 \mathbf{x} = (-0.310, -0.124, -0.155, -0.062)$ and the weights move the wrong way.
- Leaving the bias out of the update: w_0 has gradient $\hat{y} - y$ too, since $x_0 = 1$.

- Computing the loss as $-\ln \hat{y} = -\ln 0.690 = 0.371$: with $y = 0$ the loss is $-\ln(1 - \hat{y})$.
- Answering (v) with "the larger one" and no factor: 20 is the mark.

Q4.3

Two SGD steps on a pass/fail dataset with a trace table

8 marks Slides 29, 31–35 · Book 4, Ch. 3–4

Ravi wants Nova to predict whether a student passes from two features: x_1 study hours per day and x_2 attendance (1 if above 75 %, else 0). Four students are given:

Row	x_1 hours	x_2 attendance	y pass
1	1	0	0
2	2	1	0
3	3	0	1
4	4	1	1

The model is one sigmoid neuron $\hat{y} = \sigma(w_0 + w_1x_1 + w_2x_2)$ trained by SGD with $\eta = 0.5$, starting from $\mathbf{w}^{(0)} = (0, 0, 0)^\top$. The shuffled order picks row 2 first, then row 3.

- Write the design matrix $\mathbf{X}$ with its bias column, with shape. With $\mathbf{w}^{(0)}$, what is $\hat{y}$ for every row, and what is the mean binary cross-entropy $J(\mathbf{w}^{(0)})$? [1]
- Perform the SGD step on row 2: z , $\hat{y}$, error, gradient $(\hat{y} - y)\mathbf{x}$, and $\mathbf{w}^{(1)}$. [2]
- Perform the SGD step on row 3 from $\mathbf{w}^{(1)}$, giving $\mathbf{w}^{(2)}$. Present both steps in a trace table. [2]
- With $\mathbf{w}^{(2)}$, compute $\hat{y}$ for all four rows. How many rows are on the correct side of 0.5? Which ones? [2]
- Compute the mean loss $J(\mathbf{w}^{(2)})$ over the four rows and compare it with $J(\mathbf{w}^{(0)})$. Did the two SGD steps lower the total loss? Give the one-line reason from the deck's list of SGD challenges. [1] **justify**

by hand

SGD trace

read a table

WHAT THIS TESTS

The deck's numerical example (slides 31–35) with a second feature and fresh numbers: forward pass, error, gradient, update, twice in a row, then the progress check on all rows and the fact (slide 29) that single-example steps are noisy and need not lower the total loss.

HOW TO ATTEMPT

- At $\mathbf{w} = \mathbf{0}$ every $z = 0$, so every $\hat{y} = 0.5$ and every per-row loss is $-\ln 0.5$.
- Each step: $z = \mathbf{w}^\top \mathbf{x}$, $\hat{y} = \sigma(z)$, $e = \hat{y} - y$, $\nabla = e \mathbf{x}$, $\mathbf{w} \leftarrow \mathbf{w} - 0.5 \nabla$. Keep three decimals.
- For (iv) compute z for each row with $\mathbf{w}^{(2)}$, then σ . For (v) the per-row loss is $-\ln \hat{y}$ when $y = 1$ and $-\ln(1 - \hat{y})$ when $y = 0$.

SOLUTION

STEP 1

- $\mathbf{X} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 1 & 3 & 0 \\ 1 & 4 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 3}$, $\mathbf{y} = (0, 0, 1, 1)^\top$. With $\mathbf{w}^{(0)} = \mathbf{0}$: $z = 0$ for every row, $\hat{y} = \sigma(0) = 0.5$ for all four.

STEP 2

$J(\mathbf{w}^{(0)}) = \frac{1}{4} \sum_i \ell_i$, and each $\ell_i = -\ln 0.5 = 0.693$, so $J(\mathbf{w}^{(0)}) = 0.693$.

STEP 3 Row 2: $\mathbf{x} = (1, 2, 1)^\top, y = 0. z = 0(1) + 0(2) + 0(1) = 0; \hat{y} = \sigma(0) = 0.5; \text{error} = 0.5 - 0 = 0.5$
(ii)

STEP 4 $\nabla \ell = 0.5 \times (1, 2, 1)^\top = (0.5, 1.0, 0.5)^\top$
(ii)

STEP 5 $\mathbf{w}^{(1)} = (0, 0, 0)^\top - 0.5 (0.5, 1.0, 0.5)^\top = (-0.25, -0.5, -0.25)^\top$
(ii)

STEP 6 Row 3: $\mathbf{x} = (1, 3, 0)^\top, y = 1. z = -0.25(1) - 0.5(3) - 0.25(0) = -0.25 - 1.5 = -1.75$
(iii)

STEP 7 $\hat{y} = \sigma(-1.75) = \frac{1}{1 + e^{1.75}} = \frac{1}{1 + 5.755} = 0.148; \text{error} = 0.148 - 1 = -0.852$
(iii)

STEP 8 $\nabla \ell = -0.852 \times (1, 3, 0)^\top = (-0.852, -2.556, 0)^\top$
(iii)

STEP 9 $\mathbf{w}^{(2)} = \mathbf{w}^{(1)} - 0.5 \nabla \ell = (-0.25, -0.5, -0.25)^\top - 0.5 (-0.852, -2.556, 0)^\top$
(iii)

STEP 10 $= (-0.25 + 0.426, -0.5 + 1.278, -0.25)^\top = (0.176, 0.778, -0.25)^\top$
(iii)

Step	Row	x	y	z	$\hat{y}$	$\hat{y} - y$	$\nabla \ell = (\hat{y} - y)\mathbf{x}$	w after
0	-	-	-	-	-	-	-	(0, 0, 0)
1	2	(1, 2, 1)	0	0	0.500	0.500	(0.5, 1.0, 0.5)	(-0.25, -0.5, -0.25)
2	3	(1, 3, 0)	1	-1.75	0.148	-0.852	(-0.852, -2.556, 0)	(0.176, 0.778, -0.25)

STEP 1 Row 1: $z = 0.176 + 0.778(1) - 0.25(0) = 0.954, \hat{y} = \sigma(0.954) = 0.722 \rightarrow \text{class 1, true 0: wrong}$
(iv)

STEP 2 Row 2: $z = 0.176 + 0.778(2) - 0.25(1) = 1.482, \hat{y} = 0.815 \rightarrow \text{class 1, true 0: wrong}$
(iv)

STEP 3 Row 3: $z = 0.176 + 0.778(3) - 0.25(0) = 2.510, \hat{y} = 0.925 \rightarrow \text{class 1, true 1: correct}$
(iv)

STEP 4 Row 4: $z = 0.176 + 0.778(4) - 0.25(1) = 3.038, \hat{y} = 0.954 \rightarrow \text{class 1, true 1: correct}$
(iv)

STEP 5 2 of 4 rows are on the correct side of 0.5: rows 3 and 4. Rows 1 and 2 are wrong, but the ordering is right ($0.722 < 0.815 < 0.925 < 0.954$): the bias is still too high.
(iv)

STEP 6 Per-row losses: row 1 $-\ln(1 - 0.722) = 1.280$; row 2 $-\ln(1 - 0.815) = 1.687$; row 3 $-\ln 0.925 = 0.078$; row 4 $-\ln 0.954 = 0.047$
(v)

STEP 7 $J(\mathbf{w}^{(2)}) = \frac{1}{4}(1.280 + 1.687 + 0.078 + 0.047) = \frac{3.092}{4} = 0.773 > 0.693 = J(\mathbf{w}^{(0)})$. No: the total loss rose. Key fact (slide 29): SGD's updates are noisy; each step lowers the loss of the one example it used (row 3's loss fell from 0.693 to 0.078) but may raise the loss on the others, so the path to the minimum is not smooth. More epochs with shuffling bring it down.
(v)

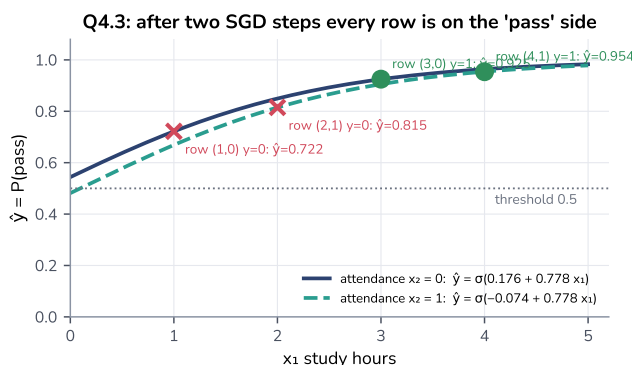


Figure 4.3 $\hat{y}$ against study hours with $\mathbf{w}^{(2)}$, one curve per attendance value. All four points sit above the 0.5 line: the two fail rows are still misclassified, but the curve already rises with hours.

(i) $\mathbf{X}$ is 4×3 ; all $\hat{y} = 0.5$; $J = -\ln 0.5 = 0.693$

(ii) $z = 0$, $\hat{y} = 0.5$, **error 0.5**, $\nabla = (0.5, 1.0, 0.5)$, $\mathbf{w}^{(1)} = (-0.25, -0.5, -0.25)$

(iii) $z = -1.75$, $\hat{y} = 0.148$, **error -0.852** , $\nabla = (-0.852, -2.556, 0)$, $\mathbf{w}^{(2)} = (0.176, 0.778, -0.25)$

(iv) $\hat{y} = (0.722, 0.815, 0.925, 0.954)$; **2 of 4 correct (rows 3, 4)**

(v) $J = 0.773 > 0.693$: **no**; SGD updates are noisy (slide 29)

✓ ANSWERS AT A GLANCE

(i) 0.5 each; (ii) $(-0.25, -0.5, -0.25)$ (iii) $(0.176, 0.778, -0.25)$ (iv) 0.722, 0.815, 0.925, 0.954; 2 of 4 (v) 0.773, rose

! WHERE MARKS ARE LOST

- Using row 1 first because it is first in the table: the question fixes the order (row 2, then row 3), and the answer depends on it.
- Forgetting the $x_2 = 0$ entry in step 2: w_2 must stay at -0.25 , not move.
- Sign slip in step 2: the error is negative, so subtracting $\eta \nabla$ adds to w_0 and w_1 .
- Reporting "4 of 4 predicted 1, so accuracy 100 %": two of the true labels are 0.

A spam filter's confusion matrix: accuracy, precision, recall, F1, specificity and the lazy baseline

Q4.4

8 marks Slides 38–43 · Book 4, Ch. 4

A spam filter (spam = positive class, $y = 1$) is evaluated on 1,000 test emails. It produced TP = 90, FP = 30, FN = 10 and TN = 870.

- (i) Draw the confusion matrix in the deck's layout (rows actual, columns predicted). How many emails are actually spam, and how many did the filter mark as spam? [2]
- (ii) Compute accuracy, precision, recall and F1, each to 3 decimals. [3]
- (iii) Compute the specificity $TN/(TN + FP)$, the recall of the negative class. [1]
- (iv) A lazy filter marks every email as ham. Compute its accuracy and its recall. Using these two numbers, state which of the four metrics in (ii) is misleading here and why. [1] **justify**
- (v) Name the metric the deck says to use when false positives are costly and the one when false negatives are costly, with the deck's example for each in one sentence. Which applies to a spam filter? [1]

by hand

confusion matrix

which metric

🎯 WHAT THIS TESTS

The four formulas of slides 40–41 on an imbalanced test set (only 10 % spam), the layout of slide 39, and the exact guidance of slide 42: accuracy misleads on imbalance, precision for costly FP (spam), recall for costly FN (disease).

▶ HOW TO ATTEMPT

- Fill the 2×2 table: TP top-left, FN top-right, FP bottom-left, TN bottom-right. Row sums are the actual counts; column sums are the predicted counts.

- Precision divides by the predicted-positive column (TP + FP); recall divides by the actual-positive row (TP + FN).
- F1 is $2PR/(P + R)$, not the average of P and R .

Σ SOLUTION

	Predicted spam (1)	Predicted ham (0)	Row sum
Actual spam (1)	TP = 90	FN = 10	100
Actual ham (0)	FP = 30	TN = 870	900
Column sum	120	880	1,000

STEP 1 Actually spam = TP + FN = 90 + 10 = 100 (10% of the set). Marked as spam = TP +
(i) FP = 90 + 30 = 120.

STEP 2 Accuracy = $\frac{TP + TN}{TP + TN + FP + FN} = \frac{90 + 870}{1000} = \frac{960}{1000} = 0.960$
(ii)

STEP 3 Precision = $\frac{TP}{TP + FP} = \frac{90}{90 + 30} = \frac{90}{120} = 0.750$
(ii) of the 120 emails it flagged, 30 were real mail

STEP 4 Recall = $\frac{TP}{TP + FN} = \frac{90}{90 + 10} = \frac{90}{100} = 0.900$
(ii) of the 100 spam emails, it caught 90

STEP 5 $F_1 = 2 \cdot \frac{P \times R}{P + R} = 2 \cdot \frac{0.750 \times 0.900}{0.750 + 0.900} = \frac{1.350}{1.650} = 0.818$
(ii)

STEP 6 Specificity = $\frac{TN}{TN + FP} = \frac{870}{870 + 30} = \frac{870}{900} = 0.967$
(iii)

STEP 7 Always-ham: TP = 0, FP = 0, FN = 100, TN = 900. Accuracy = 900/1000 = 0.900; recall =
(iv) 0/100 = 0 (precision is 0/0, undefined).

STEP 8 Key fact (slide 42): accuracy is misleading with imbalanced data. A filter that catches no
(iv) spam at all still scores 0.900, only 0.060 below the real filter's 0.960; recall (0 versus 0.900) and F1 show the difference, accuracy hides it.

STEP 9 Slide 42: precision when false positives are costly (spam detection: do not mark real
(v) emails as spam); recall when false negatives are costly (disease detection: do not miss sick patients). A spam filter therefore watches precision, here 0.750: one flagged email in four is real mail lost.

(i) 100 actual spam; 120 marked spam

(ii) accuracy 0.960, precision 0.750, recall 0.900, F1 0.818

(iii) specificity 0.967

(iv) always-ham: accuracy 0.900, recall 0; accuracy misleads on imbalance

(v) precision for costly FP (spam), recall for costly FN (disease); spam filter → precision

✓ ANSWERS AT A GLANCE

(i) 100; 120 (ii) 0.960, 0.750, 0.900, 0.818 (iii) 0.967 (iv) 0.900, 0; accuracy (v) precision / recall

! WHERE MARKS ARE LOST

- Swapping FP and FN in the matrix: precision and recall trade places (0.900 and 0.750).

- Averaging P and R for F1: $(0.750 + 0.900)/2 = 0.825$ is not the harmonic mean 0.818.
- Calling 0.960 "excellent" in (iv) without the 0.900 baseline: the mark is for the comparison.
- Answering (v) the other way round; the deck's examples fix it: spam $\rightarrow$ precision, disease $\rightarrow$ recall.

Q4.5

A 3×3 confusion matrix: per-class precision, recall, F1, macro and weighted averages

8 marks Slides 84–87 · Book 4, Ch. 9

A sentiment classifier with $K = 3$ classes (Negative, Neutral, Positive) is tested on 150 reviews. Rows are the actual class, columns the predicted class:

Actual \ Predicted	Negative	Neutral	Positive
Negative	40	8	2
Neutral	6	30	4
Positive	4	6	50

- Compute the row sums and column sums and say what each represents. Compute the accuracy. [2]
- For each class compute TP, FP, FN (one-vs-all), then precision, recall and F1, to 3 decimals. [3]
- Compute the macro-F1 and the weighted-F1 (weights $w_k = N_k/N$, the actual class counts). [2]
- Which pair of classes does the model confuse most (the largest off-diagonal entry, with its direction)? State in one line when the deck prefers the macro average and when the weighted average. [1]

by hand

multi-class metrics

read a table

WHAT THIS TESTS

Slides 85–87: reading a $K \times K$ confusion matrix (row = actual, column = predicted), one-vs-all TP/FP/FN, per-class precision and recall, and the two averaging strategies with their use cases.

HOW TO ATTEMPT

- For class k : TP is the diagonal cell; FP is the rest of column k ; FN is the rest of row k .
- Precision divides by the column sum, recall by the row sum.
- Macro: plain average of the three F1 values. Weighted: multiply each F1 by $N_k/150$ and add.

SOLUTION

STEP 1 Row sums (actual counts): $40 + 8 + 2 = 50$, $6 + 30 + 4 = 40$, $4 + 6 + 50 = 60$; total $N = 150$.

STEP 2 Column sums (predicted counts): $40 + 6 + 4 = 50$, $8 + 30 + 6 = 44$, $2 + 4 + 50 = 56$.

STEP 3 Accuracy = $\frac{\text{diagonal}}{N} = \frac{40 + 30 + 50}{150} = \frac{120}{150} = 0.800$

STEP 4 Negative: TP = 40, FP = $6 + 4 = 10$, FN = $8 + 2 = 10$. $P = \frac{40}{50} = 0.800$, $R = \frac{40}{50} = 0.800$,
 $F_1 = 2 \cdot \frac{0.800 \times 0.800}{0.800 + 0.800} = 0.800$

STEP 5 Neutral: TP = 30, FP = $8 + 6 = 14$, FN = $6 + 4 = 10$. $P = \frac{30}{44} = 0.682$, $R = \frac{30}{40} = 0.750$,
 $F_1 = 2 \cdot \frac{0.682 \times 0.750}{0.682 + 0.750} = \frac{1.023}{1.432} = 0.714$

STEP 6 Positive: TP = 50, FP = $2 + 4 = 6$, FN = $4 + 6 = 10$. $P = \frac{50}{56} = 0.893$, $R = \frac{50}{60} = 0.833$, $F_1 =$
 $2 \cdot \frac{0.893 \times 0.833}{0.893 + 0.833} = \frac{1.488}{1.726} = 0.862$

Class	N_k	TP	FP	FN	Precision	Recall	F1	$w_k = N_k/N$
Negative	50	40	10	10	0.800	0.800	0.800	0.333
Neutral	40	30	14	10	0.682	0.750	0.714	0.267
Positive	60	50	6	10	0.893	0.833	0.862	0.400

STEP 1 $\text{Macro-}F_1 = \frac{0.800 + 0.714 + 0.862}{3} = \frac{2.376}{3} = 0.792$

STEP 2 $\text{Weighted-}F_1 = \frac{50}{150}(0.800) + \frac{40}{150}(0.714) + \frac{60}{150}(0.862) = 0.267 + 0.190 + 0.345 = 0.802$
 same as $(50 \times 0.800 + 40 \times 0.714 + 60 \times 0.862)/150 = 120.30/150$; the biggest class (Positive) has the best F1, so the weighted score is higher than the macro one

STEP 3 Largest off-diagonal entry is 8, in row Negative, column Neutral: 8 negative reviews were predicted Neutral (16 % of the negatives). Slide 87: macro treats all classes equally (use when classes are equally important); weighted accounts for class imbalance (use when some classes are more common).

(i) rows (50, 40, 60) = actual counts, columns (50, 44, 56) = predicted counts; accuracy 0.800

(ii) P/R/F1: Negative 0.800/0.800/0.800; Neutral 0.682/0.750/0.714; Positive 0.893/0.833/0.862

(iii) macro-F1 0.792; weighted-F1 0.802

(iv) Negative → Neutral (8); macro when classes matter equally, weighted when they are imbalanced

✓ ANSWERS AT A GLANCE

(i) 0.800 (ii) F1: 0.800, 0.714, 0.862 (iii) 0.792; 0.802 (iv) Negative → Neutral, 8

! WHERE MARKS ARE LOST

- Using the row sum for precision: Neutral precision would come out as $30/40 = 0.750$ instead of $30/44 = 0.682$.
- Weighting by the predicted counts (50, 44, 56) instead of the actual counts (50, 40, 60).
- Reading the 8 as "Neutral predicted as Negative": rows are actual, so it is Negative predicted as Neutral.

A five-way sentiment network: parameter count, mini-batch arithmetic and the two activations

Q4.6

8 marks Slides 54, 65–71, 94 · Book 4, Ch. 5, 7

A sentiment classification system sorts customer reviews into 5 categories. Each review is represented by 500 input features. The network has a single hidden layer of 16 neurons and 5 output neurons. It is trained with mini-batch SGD on $N = 12,000$ reviews with batch size $B = 64$ for 15 epochs.

- Calculate the total trainable parameters excluding biases, showing each layer transition. [2]
- Calculate the total including biases. What fraction of all parameters sits in the first transition? [1]
- How many iterations (weight updates) are there per epoch, and how many examples are in the last mini-batch of an epoch? How many updates in the whole run? [2]

- (iv) How many updates per epoch would $B = 128$ give, and how many would single-example SGD give? [1]
- (v) Recommend the activation for the hidden layer and for the output layer. Justify each in terms of the output range and the number of outputs, using the deck's activation table (slide 94). [2]

justify

count parameters

mini-batch

justify

⊙ WHAT THIS TESTS

The weight count (in $\times$ out) per transition (slide 54 gives $(d+1) \times K$ for the linear model), iterations per epoch $\lceil N/B \rceil$ (slide 67), and the activation table of slide 94: softmax gives K coupled outputs in $[0, 1]^K$ that sum to 1.

▶ HOW TO ATTEMPT

- Transitions: $500 \rightarrow 16$ and $16 \rightarrow 5$. Multiply; then add one bias per neuron.
- $12000/64$ is not a whole number: round up, and the last batch is the remainder.
- For (v) quote the table: sigmoid $\rightarrow$ 1 output in $[0, 1]$, softmax $\rightarrow K$ coupled outputs summing to 1; ReLU for hidden layers.

Σ SOLUTION

STEP 1 Input $\rightarrow$ Hidden: $500 \times 16 = 8,000$ weights

(i)

STEP 2 Hidden $\rightarrow$ Output: $16 \times 5 = 80$ weights

(i)

STEP 3 Total excluding biases $= 8,000 + 80 = 8,080$

(i)

STEP 4 Biases: $16 + 5 = 21$. Total $= 8,080 + 21 = 8,101$. First transition with its biases: $8,000 + 16 = 8,016$, which is $8016/8101 = 0.990$, about 99 % of all parameters.

(ii)

the 500 input features dominate; the output layer is tiny

STEP 5 $N/B = 12000/64 = 187.5$, so iterations per epoch $= \lceil 187.5 \rceil = 188$. The first 187 batches use $187 \times 64 = 11,968$ reviews; the last batch has $12000 - 11968 = 32$.

(iii)

STEP 6 Updates in 15 epochs $= 188 \times 15 = 2,820$.

(iii)

STEP 7 $B = 128$: $\lceil 12000/128 \rceil = \lceil 93.75 \rceil = 94$ updates per epoch (half as many). Single-example SGD ($B = 1$): 12,000 updates per epoch.

(iv)

STEP 8 Hidden layer: ReLU, $\text{ReLU}(z) = \max(0, z)$. It is not in the table of slide 94 because that table lists output activations; for hidden layers the deck's later modules use ReLU because it does not saturate the way sigmoid does (slide 91 warns of sigmoid saturation) and its gradient is 1 for $z > 0$.

(v)

STEP 9 Output layer: softmax. Key facts (slide 94): output range $[0, 1]^K$ with $\sum_k \hat{y}_k = 1$; use case multi-class; number of outputs K , coupled. The 5 reviews classes are mutually exclusive, so we need one probability distribution over 5 outputs, and the prediction is $\arg \max_k \hat{y}_k$. Five separate sigmoids would give 5 independent numbers that need not sum to 1.

(v)

(i) $8,000 + 80 = 8,080$

(ii) 8,101; first transition $\approx 99\%$

(iii) 188 per epoch, last batch 32; 2,820 updates

(iv) 94; 12,000

(v) hidden ReLU; output softmax — $K = 5$ coupled outputs in $[0, 1]$ summing to 1

✓ ANSWERS AT A GLANCE

(i) 8,080 (ii) 8,101 (iii) 188; 32; 2,820 (iv) 94; 12,000 (v) ReLU / softmax

! WHERE MARKS ARE LOST

- Adding the layer sizes ($500 + 16 + 5$) instead of multiplying the transitions.
- Rounding 187.5 down to 187: the remainder batch of 32 still causes an update.
- Recommending sigmoid on the output for 5 classes, or softmax in the hidden layer.

Q4.7

**One mini-batch SGD step in matrix form
with every shape written**

8 marks Slides 52–57, 68–70 · Book 4, Ch. 5–7

A three-class linear model ($K = 3$) has one feature x_1 plus the bias column, so $d + 1 = 2$. The current weight matrix and one mini-batch of $B = 2$ examples are

$$\mathbf{W}^{(0)} = \begin{bmatrix} 0.1 & -0.2 & 0.3 \\ 0.2 & 0.1 & -0.1 \end{bmatrix}, \quad \mathbf{X}_B = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}$$

The true classes are: example 1 $\rightarrow$ class 3, example 2 $\rightarrow$ class 2. Use $\eta = 0.5$. Row 0 of $\mathbf{W}$ holds the biases.

- (i) Write the one-hot label matrix $\mathbf{Y}_B$. State the shapes of $\mathbf{X}_B$, $\mathbf{W}$, $\mathbf{Z}_B$, $\hat{\mathbf{Y}}_B$, $\mathbf{Y}_B$, $\mathbf{X}_B^\top$ and the gradient $\nabla_{\mathbf{W}} J$. [1]
- (ii) Compute the logits $\mathbf{Z}_B = \mathbf{X}_B \mathbf{W}^{(0)}$. [1]
- (iii) Apply softmax to each row to get $\hat{\mathbf{Y}}_B$ (3 decimals) and compute the mini-batch loss $J = -\frac{1}{B} \sum_i \sum_k y_k^{(i)} \ln \hat{y}_k^{(i)}$. [2]
- (iv) Compute $\hat{\mathbf{Y}}_B - \mathbf{Y}_B$ and the gradient $\nabla_{\mathbf{W}} J = \frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$. [2]
- (v) Update to $\mathbf{W}^{(1)} = \mathbf{W}^{(0)} - \eta \nabla_{\mathbf{W}} J$. Then recompute $\mathbf{Z}_B$, $\hat{\mathbf{Y}}_B$ and the loss with $\mathbf{W}^{(1)}$. Which classes are predicted before and after the step? [2]

by hand

matrix form

mini-batch SGD

🔗 WHAT THIS TESTS

The deck's mini-batch example (slides 73–78) with fresh numbers, including the loss before and after: shapes of every matrix, $\mathbf{Z} = \mathbf{XW}$, row-wise softmax, $\frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$, and the update.

▶ HOW TO ATTEMPT

- $\mathbf{Z}_B$ is 2×3 : each row is $[1, x_1]$ times the two rows of $\mathbf{W}$.
- Softmax one row at a time; keep four decimals in the middle so the gradient sums are right.
- $\mathbf{X}_B^\top$ is 2×2 ; multiply by the 2×3 error matrix, divide by $B = 2$.
- Loss: the true-class entries are $\hat{y}_3^{(1)}$ and $\hat{y}_2^{(2)}$.

Σ SOLUTION

STEP 1 $\mathbf{Y}_B = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ (row 1: class 3; row 2: class 2). Shapes: $\mathbf{X}_B$ is $B \times (d+1) = 2 \times 2$; $\mathbf{W}$ is $(d+1) \times K = 2 \times 3$; $\mathbf{Z}_B, \hat{\mathbf{Y}}_B, \mathbf{Y}_B$ are $B \times K = 2 \times 3$; $\mathbf{X}_B^\top$ is 2×2 ; the gradient is $(d+1) \times K = 2 \times 3$, the same shape as $\mathbf{W}$.

STEP 2 Row 1: $[1, 2]\mathbf{W} = [0.1 + 2(0.2), -0.2 + 2(0.1), 0.3 + 2(-0.1)] = [0.5, 0.0, 0.1]$

(i)

STEP 3 Row 2: $[1, 3]\mathbf{W} = [0.1 + 3(0.2), -0.2 + 3(0.1), 0.3 + 3(-0.1)] = [0.7, 0.1, 0.0]$. So $\mathbf{Z}_B =$

(ii)

$$\begin{bmatrix} 0.5 & 0.0 & 0.1 \\ 0.7 & 0.1 & 0.0 \end{bmatrix}$$

STEP 4 Row 1: $e^{0.5} = 1.6487, e^0 = 1, e^{0.1} = 1.1052$; sum 3.7539. $\hat{\mathbf{y}}^{(1)} = [0.4392, 0.2664, 0.2944] \approx [0.439, 0.266, 0.294]$

(iii)

STEP 5 Row 2: $e^{0.7} = 2.0138, e^{0.1} = 1.1052, e^0 = 1$; sum 4.1189. $\hat{\mathbf{y}}^{(2)} = [0.4889, 0.2683, 0.2428] \approx [0.489, 0.268, 0.243]$

(iii)

STEP 6 $J = -\frac{1}{2} [\ln \hat{y}_3^{(1)} + \ln \hat{y}_2^{(2)}] = \frac{1}{2} [-\ln 0.2944 - \ln 0.2683] = \frac{1}{2} (1.223 + 1.316) = 1.269$

(iii)

STEP 7 $\hat{\mathbf{Y}}_B - \mathbf{Y}_B = \begin{bmatrix} 0.4392 & 0.2664 & 0.2944 - 1 \\ 0.4889 & 0.2683 - 1 & 0.2428 \end{bmatrix} = \begin{bmatrix} 0.4392 & 0.2664 & -0.7056 \\ 0.4889 & -0.7317 & 0.2428 \end{bmatrix}$

(iv)

each row sums to 0

STEP 8 $\mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B) = \begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 0.4392 & 0.2664 & -0.7056 \\ 0.4889 & -0.7317 & 0.2428 \end{bmatrix} = \begin{bmatrix} 0.9281 & -0.4653 & -0.4628 \\ 2.3451 & -1.6623 & -0.6828 \end{bmatrix}$

(iv)

row 0 adds the two error rows (bias); row 1 weights them by $x_1 = 2, 3$

STEP 9 $\nabla_{\mathbf{W}} J = \frac{1}{2} \begin{bmatrix} 0.9281 & -0.4653 & -0.4628 \\ 2.3451 & -1.6623 & -0.6828 \end{bmatrix} = \begin{bmatrix} 0.464 & -0.233 & -0.231 \\ 1.173 & -0.831 & -0.341 \end{bmatrix}$

(iv)

STEP 10 $\mathbf{W}^{(1)} = \mathbf{W}^{(0)} - 0.5 \nabla J = \begin{bmatrix} 0.1 - 0.232 & -0.2 + 0.116 & 0.3 + 0.116 \\ 0.2 - 0.586 & 0.1 + 0.416 & -0.1 + 0.171 \end{bmatrix} = \begin{bmatrix} -0.132 & -0.084 & 0.416 \\ -0.386 & 0.516 & 0.071 \end{bmatrix}$

(v)

STEP 11 New logits: $\mathbf{Z}_B = \begin{bmatrix} -0.905 & 0.947 & 0.557 \\ -1.291 & 1.463 & 0.628 \end{bmatrix}$; softmax $\hat{\mathbf{Y}}_B = \begin{bmatrix} 0.086 & 0.545 & 0.369 \\ 0.043 & 0.668 & 0.290 \end{bmatrix}$

(v)

STEP 12 New loss $J = \frac{1}{2} (-\ln 0.369 - \ln 0.668) = 0.700$, down from 1.269. Predicted classes (arg max per row): before the step (1, 1), after it (2, 2); the true classes are (3, 2), so example 2 is now right and example 1 still wrong, though its true-class probability rose from 0.294 to 0.369.

(v)

(i) $\mathbf{Y}_B = [[0, 0, 1], [0, 1, 0]]$; $2 \times 2, 2 \times 3, 2 \times 3, 2 \times 3, 2 \times 3, 2 \times 2, 2 \times 3$

(ii) $\mathbf{Z}_B = [[0.5, 0.0, 0.1], [0.7, 0.1, 0.0]]$

(iii) $\hat{\mathbf{Y}}_B = [[0.439, 0.266, 0.294], [0.489, 0.268, 0.243]]$; $J = 1.269$

(iv) $\nabla J = [[0.464, -0.233, -0.231], [1.173, -0.831, -0.341]]$

(v) $\mathbf{W}^{(1)} = [[-0.132, -0.084, 0.416], [-0.386, 0.516, 0.071]]$; $J = 0.700$; predictions (1, 1) $\rightarrow$ (2, 2)

✓ ANSWERS AT A GLANCE

(ii) $[[0.5, 0.0, 0.1], [0.7, 0.1, 0.0]]$ (iii) $J = 1.269$ (iv) $[[0.464, -0.233, -0.231], [1.173, -0.831, -0.341]]$ (v) $J = 0.700$

! WHERE MARKS ARE LOST

- Computing $\mathbf{W}\mathbf{X}_B$ instead of $\mathbf{X}_B\mathbf{W}$: the shapes $(2 \times 3)(2 \times 2)$ do not even multiply.
- Softmax over a column instead of a row: each example's three probabilities must sum to 1.
- Forgetting the $\frac{1}{B}$: the gradient doubles and $\mathbf{W}^{(1)}$ is wrong in every entry.
- Rounding $\hat{\mathbf{Y}}_B$ to 3 decimals before the products: entries of the gradient drift by ± 0.001 .

A Keras binary classifier: architecture, nature, output activation, parameters, updates, inference

Q4.8

8 marks Slides 18–21, 67, 94–95 · Book 4, Ch. 2–4, 7

Consider the following Python code snippet, written to predict whether a telecom customer will leave next month (churn) from 12 features. The training set has 4,000 customers.

```
model = Sequential()
model.add(Dense(32, input_dim=12, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='SGD', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=25, batch_size=32)
model.evaluate(X_test, y_test)
```

- (a) Draw the architecture, marking the number of neurons and the activation of each layer. [1]
- (b) State the nature of the problem (binary / multi-class / regression) and the two lines of code that show it. [1]
- (c) Why is sigmoid used in the output layer instead of ReLU or softmax? Answer with the output range of each. Under what condition would you replace sigmoid with softmax, and what two lines of code would change? [2] **justify**
- (d) Calculate the number of trainable parameters (weights + biases) for each layer and the total. [2]
- (e) How many weight updates happen in one epoch, and in the whole run? [1]
- (f) The model outputs 0.27 for a customer. Give the inference of this output: the probability of each class, the predicted class, and the largest threshold that would flip the prediction. [1]

count parameters

sigmoid head

justify

@ WHAT THIS TESTS

Reading a model definition the way the mid-sem does: last layer plus loss tell the task; slide 94's ranges (sigmoid $[0, 1]$, softmax $[0, 1]^K$ summing to 1) say why the head is a sigmoid; parameters per Dense layer are $\text{in} \times \text{out} + \text{out}$; an epoch has N/B updates; and a sigmoid output is $P(y = 1 \mid \mathbf{x})$ thresholded at 0.5 (slide 20).

► HOW TO ATTEMPT

- Layer transitions: $12 \rightarrow 32 \rightarrow 16 \rightarrow 1$. For each, weights = $\text{in} \times \text{out}$, biases = out.
- Updates per epoch = $4000/32$; multiply by 25.
- For (c) quote the ranges; for (f) use $P(y = 0) = 1 - 0.27$.

SOLUTION

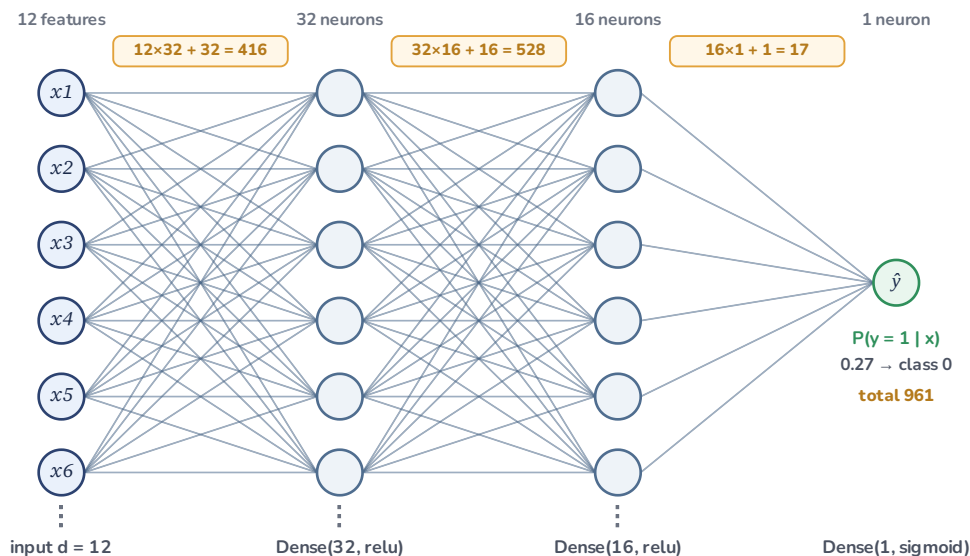


Figure 4.4 The architecture for (a): 12 inputs, two ReLU hidden layers of 32 and 16 neurons, one sigmoid output neuron. The parameter count of each transition is written above it.

- STEP 1** Input layer: 12 features (no parameters). Hidden 1: 32 neurons, ReLU. Hidden 2: 16 neurons, ReLU. Output: 1 neuron, sigmoid, giving $\hat{y} = P(\text{churn} \mid \mathbf{x})$. See Figure 4.4.
- STEP 2** Binary classification: churn or not. The evidence is `Dense(1, activation='sigmoid')` (one probability output) and `loss='binary_crossentropy'`.
- STEP 3** Key facts (slide 94): sigmoid maps z to $[0, 1]$, which is a probability $P(y = 1 \mid \mathbf{x})$ that can be thresholded at 0.5. ReLU outputs $[0, \infty)$: not a probability, no natural threshold, and it cannot be used in $\ln(1 - \hat{y})$ of the BCE. Softmax on a single unit is $e^z / e^z = 1$ for every input, so it would predict 1 always; softmax is for K coupled outputs.
- STEP 4** Replace sigmoid with softmax when there are $K > 2$ mutually exclusive classes (say churn / downgrade / stay, $K = 3$): change the head to `Dense(K, activation='softmax')` and the loss to `categorical_crossentropy` with one-hot labels.
- STEP 5** Layer 1 ($12 \rightarrow 32$): weights $12 \times 32 = 384$, biases 32, total 416
- STEP 6** Layer 2 ($32 \rightarrow 16$): weights $32 \times 16 = 512$, biases 16, total 528
- STEP 7** Layer 3 ($16 \rightarrow 1$): weights $16 \times 1 = 16$, biases 1, total 17
- STEP 8** Total = $416 + 528 + 17 = 961$ trainable parameters (912 weights + 49 biases).
- STEP 9** Updates per epoch = $4000 / 32 = 125$. Whole run = $125 \times 25 = 3,125$ updates; every customer is seen 25 times.
- STEP 10** $\hat{y} = 0.27 = P(y = 1 \mid \mathbf{x})$: a 27% chance of churn, so $P(y = 0 \mid \mathbf{x}) = 1 - 0.27 = 0.73$. Since $0.27 < 0.5$ the predicted class is 0 (stays). The logit behind it is $z = \ln \frac{0.27}{0.73} = -0.995 < 0$, the same decision. Any threshold $t \leq 0.27$ would flip the prediction to churn; the largest such threshold is 0.27 itself (for example 0.25 if the company wants to catch more leavers at the cost of precision).

(a) 12 → 32 (ReLU) → 16 (ReLU) → 1 (sigmoid), Figure 4.4

(b) binary: `Dense(1, 'sigmoid')` and `binary_crossentropy`

(c) sigmoid $[0, 1]$ = probability; ReLU $[0, \infty)$; softmax on one unit is always 1; switch when $K > 2$ exclusive classes: `Dense(K, 'softmax') + categorical_crossentropy`

(d) $416 + 528 + 17 = 961$

(e) 125 per epoch; 3,125 in total

(f) $P(1) = 0.27$, $P(0) = 0.73$, class 0; threshold 0.27 flips it

✓ ANSWERS AT A GLANCE

(b) binary (c) $[0, 1]$ vs $[0, \infty)$ vs always 1 (d) 416, 528, 17; 961 (e) 125; 3,125 (f) class 0, 0.27 / 0.73

! WHERE MARKS ARE LOST

- Forgetting the biases (912 instead of 961), or giving the input layer parameters.
- Answering (e) with "25 updates": an epoch has $4000/32 = 125$ mini-batch updates.
- Reading 0.27 as "27 % accuracy" or as class 1: it is $P(y = 1 | \mathbf{x})$, below 0.5.
- In (c) saying "softmax is for multi-class" without the ranges: the mark is for $[0, 1]$, $[0, \infty)$ and the one-unit softmax equalling 1.

Q4.9

Softmax properties with numbers: shift, scale, and the $K = 2$ case

8 marks Slides 57–58, 91 · Book 4, Ch. 6, 10

A three-class model produces the logits $\mathbf{z} = (3, 1, 0)$.

- Compute $\text{softmax}(\mathbf{z})$ to 3 decimals. [1]
- Compute $\text{softmax}(\mathbf{z} + c)$ with $c = -3$, showing the exponentials and their sum. Compare with (i) and name the property. Why does the deck's stable softmax choose $c = -\max_k z_k$, and what would go wrong for $\mathbf{z} = (1000, 999, 0)$ without it? [3]
- Compute $\text{softmax}(2\mathbf{z})$. What happens to the largest probability and to the ratio $\hat{y}_1 / \hat{y}_2$? Name the property from slide 58 this shows, and the one that guarantees the $\arg \max$ does not change. [2]
- For $K = 2$ and logits $\mathbf{z} = (a, 0)$ with $a = 1.5$, compute $\text{softmax}(\mathbf{z})_1$ and $\sigma(a)$ to 3 decimals. Name the property. [2]

by hand

softmax properties

stable softmax

🎯 WHAT THIS TESTS

The six properties on slide 58 turned into numbers: translation invariance (and the stable softmax of slide 91 built on it), the exponential amplifying differences while preserving order, and softmax reducing to the sigmoid when $K = 2$.

▶ HOW TO ATTEMPT

- Each part is the same three steps: exponentiate, add, divide.
- For (ii) the exponentials all shrink by the same factor e^{-3} , so the ratios do not change.
- For (iv) write $\frac{e^a}{e^a + e^0}$ and divide top and bottom by e^a .

Σ SOLUTION

STEP 1 $e^3 = 20.086, e^1 = 2.718, e^0 = 1$; sum = 23.804. $\text{softmax}(\mathbf{z}) = \left(\frac{20.086}{23.804}, \frac{2.718}{23.804}, \frac{1}{23.804}\right) =$
 (i) (0.844, 0.114, 0.042)

STEP 2 $\mathbf{z} + \mathbf{c} = (0, -2, -3): e^0 = 1, e^{-2} = 0.135, e^{-3} = 0.050$; sum = 1.185. $\text{softmax} =$
 (ii) $\left(\frac{1}{1.185}, \frac{0.135}{1.185}, \frac{0.050}{1.185}\right) = (0.844, 0.114, 0.042)$: identical to (i).

STEP 3 Property 4, translation invariance: $\text{softmax}(\mathbf{z} + \mathbf{c}) = \text{softmax}(\mathbf{z})$, because $e^{z_k + c} = e^c e^{z_k}$ and the common factor e^c cancels in the ratio. Slide 91's stable softmax uses $c = \max_k z_k$ (here $c = 3$, subtracted) so the largest exponent is $e^0 = 1$ and every other exponential is at most 1: no overflow, and the denominator is at least 1, so no division by zero. For $\mathbf{z} = (1000, 999, 0)$, e^{1000} overflows to ∞ in floating point and the result is NaN; after subtracting 1000 the exponentials are 1, e^{-1}, e^{-1000} , all safe.

STEP 4 $2\mathbf{z} = (6, 2, 0): e^6 = 403.429, e^2 = 7.389, e^0 = 1$; sum = 411.818. $\text{softmax}(2\mathbf{z}) =$
 (iii) (0.980, 0.018, 0.002)

STEP 5 The top probability rises from 0.844 to 0.980 and the ratio $\hat{y}_1/\hat{y}_2 = e^{z_1 - z_2}$ goes from $e^2 = 7.389$ to $e^4 = 54.598$: the distribution sharpens. This is the slide-58 remark that the exponential amplifies differences (larger logits get much larger probabilities). Property 3, order preservation ($z_i > z_j \Rightarrow \hat{y}_i > \hat{y}_j$), guarantees the arg max stays at class 1.

STEP 6 $\text{softmax}(1.5, 0)_1 = \frac{e^{1.5}}{e^{1.5} + e^0} = \frac{4.482}{4.482 + 1} = \frac{4.482}{5.482} = 0.818$
 (iv)

STEP 7 $\sigma(1.5) = \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.223} = 0.818$. Equal, because dividing top and bottom of $\frac{e^a}{e^a + 1}$ by e^a gives $\frac{1}{1 + e^{-a}}$. Property 6: for $K = 2$ softmax reduces to the sigmoid (and the second output is $1 - \sigma(a) = 0.182$).

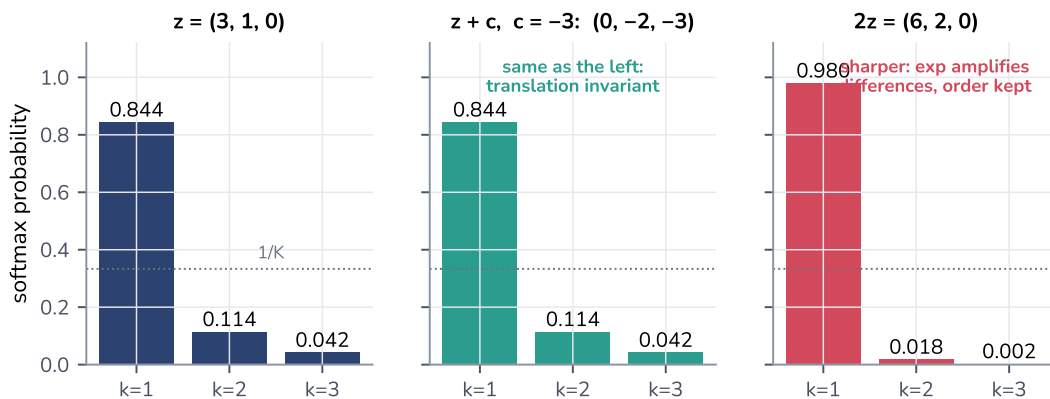


Figure 4.5 Left and middle are the same bars: shifting every logit by -3 changes nothing. Doubling the logits (right) pushes almost everything onto class 1 but keeps the order.

(i) (0.844, 0.114, 0.042)

(ii) same (0.844, 0.114, 0.042); translation invariance; $c = \max z$ makes the largest exponential 1, so e^{1000} cannot overflow

(iii) (0.980, 0.018, 0.002); ratio $7.389 \rightarrow 54.598$; exponential amplifies differences; order preserved

(iv) $0.818 = \sigma(1.5)$; softmax reduces to sigmoid for $K = 2$

✓ ANSWERS AT A GLANCE

(i) 0.844, 0.114, 0.042 (ii) same; invariance; $c = \max z$ (iii) 0.980, 0.018, 0.002 (iv) 0.818 both

! WHERE MARKS ARE LOST

- Saying the probabilities in (ii) "get smaller because the logits are smaller": the exponentials shrink, the probabilities do not.
- Claiming that doubling the logits doubles the probabilities: softmax is not linear; the top probability goes $0.844 \rightarrow 0.980$.
- Writing $\sigma(a)$ as $e^a / (1 + e^a)$ and softmax_1 as something different: they are the same expression.

Inference on four test images: top-1, top-2, mean cross-entropy, and a confidence threshold

Q4.10

8 marks Slides 60, 81–82, 88 · Book 4, Ch. 8–9

Nova's camera model classifies objects into $K = 5$ classes. For four test images the softmax outputs and the true labels are:

Image	$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_3$	$\hat{y}_4$	$\hat{y}_5$	True class
A	0.60	0.20	0.10	0.05	0.05	1
B	0.30	0.45	0.15	0.05	0.05	1
C	0.05	0.10	0.70	0.10	0.05	3
D	0.25	0.20	0.15	0.30	0.10	5

- For each image give the predicted class $\hat{k} = \arg \max_k \hat{y}_k$ and its confidence. Compute the top-1 accuracy. [2]
- List the top-2 set for each image and compute the top-2 accuracy. [2]
- Compute the cross-entropy loss of each image and the mean loss J over the four. Which image contributes the most, and what fraction of the total? [2]
- A deployment rule says: predict only if $\max_k \hat{y}_k \geq 0.6$, otherwise send the image to a human. How many images get a prediction, what is the accuracy among them, and how many correct predictions were sent away? [2]

read a table

inference

top-K accuracy

🎯 WHAT THIS TESTS

Inference by $\arg \max$ with the probability as confidence (slides 81–82), top- K accuracy as "is the true class among the K largest" (slide 88), the categorical cross-entropy as $-\ln$ of the true-class probability (slide 62), and reasoning with a confidence threshold.

▶ HOW TO ATTEMPT

- Circle the largest entry in each row; compare with the true class.
- For top-2 circle the two largest; a hit if the true class is either.
- Loss per image = $-\ln(\hat{y}_{\text{true}})$; then average.

Σ SOLUTION

Image	$\hat{k}$ (conf.)	True	Top-1 hit	Top-2 set	Top-2 hit	$\hat{y}_{\text{true}}$	$-\ln \hat{y}_{\text{true}}$	$\max \hat{y} \geq 0.6?$
A	1 (0.60)	1	yes	{1, 2}	yes	0.60	0.511	yes
B	2 (0.45)	1	no	{2, 1}	yes	0.30	1.204	no
C	3 (0.70)	3	yes	{3, 2}	yes	0.70	0.357	yes
D	4 (0.30)	5	no	{4, 1}	no	0.10	2.303	no

STEP 1 Predictions: $A \rightarrow 1$ (0.60), $B \rightarrow 2$ (0.45), $C \rightarrow 3$ (0.70), $D \rightarrow 4$ (0.30). Correct: A and C. Top-1 accuracy = $\frac{2}{4} = 0.50$.

STEP 2 Top-2 sets: A {1, 2}, B {2, 1}, C {3, 2}, D {4, 1}. The true class is inside for A, B, C; not for D (class 5 has the smallest probability, 0.10). Top-2 accuracy = $\frac{3}{4} = 0.75$.
top-2 can only be $\geq$ top-1; B is the image it rescues

STEP 3 $\ell_A = -\ln 0.60 = 0.511$; $\ell_B = -\ln 0.30 = 1.204$; $\ell_C = -\ln 0.70 = 0.357$; $\ell_D = -\ln 0.10 = 2.303$

STEP 4 $J = \frac{1}{4}(0.511 + 1.204 + 0.357 + 2.303) = \frac{4.374}{4} = 1.094$. Image D contributes the most: $2.303/4.374 = 0.53$, about 53 % of the total from one image, because it gave the true class only 10 %.

STEP 5 Maximum probabilities: A 0.60, B 0.45, C 0.70, D 0.30. Only A and C reach 0.6: 2 of 4 images get a prediction (coverage 50 %), both correct, so accuracy among them = $\frac{2}{2} = 1.00$. The two rejected images (B, D) were both wrong under top-1, so 0 correct predictions were sent away: the rule keeps every correct answer and removes both errors, at the cost of answering only half the images.

(i) $\hat{k} = (1, 2, 3, 4)$; top-1 accuracy 0.50

(ii) {1,2}, {2,1}, {3,2}, {4,1}; top-2 accuracy 0.75

(iii) losses 0.511, 1.204, 0.357, 2.303; $J = 1.094$; D, 53 %

(iv) 2 predicted (A, C); accuracy 1.00 among them; 0 correct ones rejected

✓ ANSWERS AT A GLANCE

(i) 0.50

(ii) 0.75

(iii) $J = 1.094$, D 53 %

(iv) 2; 1.00; 0

! WHERE MARKS ARE LOST

- Counting image B as correct in top-1 because class 1 has "a high 0.30": top-1 uses only the arg max.
- Using the predicted class's probability in the loss (e.g. $-\ln 0.45$ for B): the loss uses the true class.
- Reporting the threshold rule's accuracy as $2/4 = 0.5$: among the images it actually predicts it is $2/2$.

Module 5: Deep Feed-Forward Neural Networks

Ten mid-sem-style questions on networks with hidden layers: an XOR-like alarm that one neuron cannot learn, parameter counts, a full forward pass, backward pass and update on a 2-3-2 network, an architecture spec to examine, gradient flow through five layers, activation and loss derivatives, softmax with cross-entropy, and the deck's sizing rules.

On your slides: **Module 5, slides 4–61** and **Module 5 Numerical, slides 3–80** · XOR and the MLP · activations · $z^{(\ell)} = h^{(\ell-1)}W^{(\ell)} + b^{(\ell)}$ · parameters $n_\ell(n_{\ell-1} + 1)$ · forward pass · BCE / CCE · $\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$ · $\delta^{(\ell)} = (\delta^{(\ell+1)}W^{(\ell+1)\top}) \odot \sigma'(z^{(\ell)})$ · update · gradient flow · design rules

Module 5 is where Nova gets a real brain: layers. One neuron draws one straight line, so it fails on XOR; two hidden units plus one output unit solve it. Each layer computes $z^{(\ell)} = h^{(\ell-1)}W^{(\ell)} + b^{(\ell)}$ then $h^{(\ell)} = \sigma(z^{(\ell)})$, with $n_\ell(n_{\ell-1} + 1)$ parameters. Training is forward pass → loss → backward pass → update: the output error is $\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$ for all three loss-activation pairs, it travels back as $\delta^{(\ell)} = (\delta^{(\ell+1)}W^{(\ell+1)\top}) \odot \sigma'(z^{(\ell)})$, and the gradients are $h^{(\ell-1)\top}\delta^{(\ell)}$ and $\delta^{(\ell)}$. The Numerical deck does all of this by hand on a 2-3-2 network (column convention $z^{[1]} = W^{[1]}x + b^{[1]}$), then studies gradient flow, three sizing examples, sigmoid/tanh/softmax with their derivatives, and MSE/MAE/BCE/CE with theirs. Every question below is built like the mid-sem paper: a scenario with all numbers given, a numeric core to compute completely, then one to three short pointed parts with one specific expected answer each.

Q	Scenario	Numeric / pointed marks	Slides	Marks
5.1	Door alarm on two sensor bits (XOR-like): four inequalities, 2-2-1 MLP with step units, 9 parameters, verify all rows	7 / 1	M5 4–8, 21	8
5.2	Sentiment network 1,000 → 8 → 5: parameters with and without biases; why it overfits; activations; mini-batches for $N = 20,000$, $B = 128$	5 / 3	M5 17, 21, 44–46, 50–52	8
5.3	Forward pass through 2 → 3 ReLU → 2 sigmoid with fresh numbers: $z^{[1]}, a^{[1]}, z^{[2]}, a^{[2]}$, BCE	8 / 0	M5N 3–13; M5 19, 28–31	8
5.4	Backward pass for the same network: $\delta^{[2]}$, all four gradients, the zero entries	7 / 1	M5N 14–27; M5 32–37	8
5.5	Update with $\alpha = 0.1$, re-run the forward pass, show the loss fell; the dead unit	7 / 1	M5N 28–31; M5 39	8
5.6	Engineer's spec 12 → 6 tanh → 10 sigmoid → 3 tanh → 4 softmax + MSE + zero init: parameters, flaws, corrected spec	5 / 3	M5 21, 37, 50–52, 55, 60	8
5.7	Gradient flow through 5 sigmoid layers with $\ W\ = 0.9$; the same with ReLU; layers until $ \delta < 10^{-3}$	7 / 1	M5N 33–34; M5 13, 47	8
5.8	Sigmoid, tanh, ReLU and their derivatives at $z = (1.5, -0.5, 0, 2.0)$; largest slope at 0; tanh versus sigmoid	6 / 2	M5N 44–56; M5 37	8
5.9	Stable softmax + CE + gradient for four logits; MSE, MAE, BCE losses and derivatives	7 / 1	M5N 47–49, 57–61, 64–73	8

Q5.1

**A door alarm that one neuron cannot learn:
design and verify a 2-2-1 MLP**

8 marks M5 slides 4–8, 21 · Book 5, Ch. 1

Asha wires two sensors to Nova: $x_1 = 1$ when the hostel door is open, $x_2 = 1$ when the window is open. The alarm y must ring when *exactly one* of them is open (someone slipped in through one opening), and stay quiet when both are shut or both are open (the room is being aired). The recorded dataset is:

Row	x_1 door	x_2 window	y alarm
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

A step unit outputs $\text{step}(z) = 1$ if $z > 0$ and 0 otherwise, with $z = w_1x_1 + w_2x_2 + b$.

- Propose a suitable perceptron architecture to classify this dataset correctly. [1]
- Justify why a single-layer perceptron cannot do it: write the four inequalities one step unit would have to satisfy and show they contradict each other. [2] **justify**
- Draw the complete MLP with its activation functions and say what each hidden unit computes. [2]
- Calculate the total number of trainable parameters of your architecture. [1]
- Using hidden weights $W^{[1]} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}$, $b^{[1]} = (-0.5, 1.5)^\top$ and output weights $W^{[2]} = [1 \ 1]$, $b^{[2]} = -1.5$, compute z and the output of every unit for all four rows and confirm the alarm is right each time. [2]

by hand **XOR** **design**

WHAT THIS TESTS

Slides 5–8: XOR is not linearly separable, so one step unit fails; two hidden units (each a line) plus an output unit that combines them succeed. Also the parameter-count formula $n_\ell(n_{\ell-1} + 1)$ of slide 21 and a full forward check with step units.

HOW TO ATTEMPT

- For (b) write $z > 0$ for the two alarm rows and $z \leq 0$ for the two quiet rows. Add the two "alarm" inequalities, add the two "quiet" ones, compare.
- For (c) name the hidden units by the gate they compute (OR and NAND); the output is AND of the two.
- For (e) make a table: one row per input, columns $z_{h_1}, h_1, z_{h_2}, h_2, z_{\text{out}}, \hat{y}$.

SOLUTION

STEP 1 A multi-layer perceptron with 2 inputs, one hidden layer of 2 step units, and 1 output step unit: a 2-2-1 MLP. One layer is not enough (part b).

STEP 2 One step unit needs $w_1x_1 + w_2x_2 + b > 0$ on alarm rows and ≤ 0 on quiet rows:

(b) row 1 (0, 0) $\rightarrow 0: b \leq 0$; row 2 (0, 1) $\rightarrow 1: w_2 + b > 0$; row 3 (1, 0) $\rightarrow 1: w_1 + b > 0$; row 4

$$(1, 1) \rightarrow 0: w_1 + w_2 + b \leq 0.$$

STEP 3 Add rows 2 and 3: $w_1 + w_2 + 2b > 0$. Add rows 1 and 4: $w_1 + w_2 + 2b \leq 0$. The same quantity cannot be both > 0 and ≤ 0 , so no (w_1, w_2, b) exists.

(b) a single unit draws one line; the two alarm points sit on opposite corners

STEP 4 Key fact (slide 6): the decision boundary $w_1x_1 + w_2x_2 + b = 0$ is one straight line, and no line puts $(0, 1), (1, 0)$ on one side and $(0, 0), (1, 1)$ on the other.

STEP 5 Hidden unit h_1 : $\text{step}(x_1 + x_2 - 0.5) = \text{OR}$ (fires if any opening is open). Hidden unit h_2 : $\text{step}(-x_1 - x_2 + 1.5) = \text{NAND}$ (fires unless both are open). Output: $\text{step}(h_1 + h_2 - 1.5) = \text{AND}$ of the two, which is "at least one open AND not both": XOR. Each hidden unit is one line in input space; in hidden space (h_1, h_2) the alarm rows land on $(1, 1)$ and the quiet rows on $(0, 1)$ and $(1, 0)$, which one line separates (slide 8).

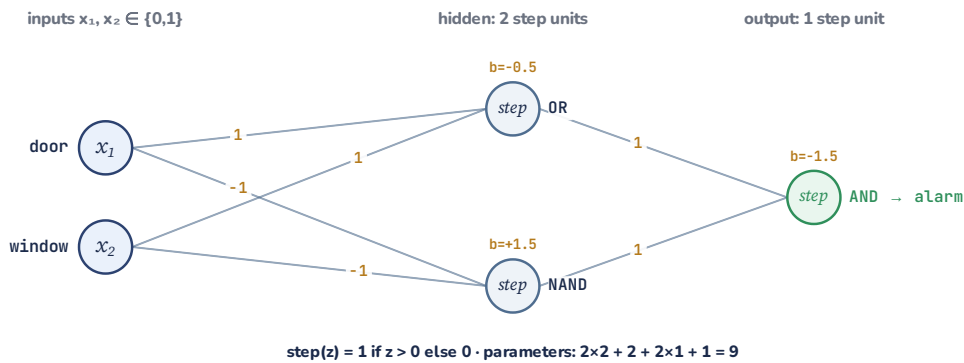


Figure 5.1 The 2-2-1 MLP for part (c) with the weights of part (e). Read the biases above the units; every unit is a step unit.

STEP 1 Layer 1: $n_1(n_0 + 1) = 2(2 + 1) = 6$. Layer 2: $n_2(n_1 + 1) = 1(2 + 1) = 3$. Total $6 + 3 = 9$ parameters.

STEP 2 Row 1 $(0, 0)$: $z_{h_1} = 0 + 0 - 0.5 = -0.5 \rightarrow h_1 = 0$; $z_{h_2} = 0 + 0 + 1.5 = 1.5 \rightarrow h_2 = 1$; $z_{\text{out}} = 0 + 1 - 1.5 = -0.5 \rightarrow \hat{y} = 0 \checkmark$

STEP 3 Row 2 $(0, 1)$: $z_{h_1} = 0 + 1 - 0.5 = 0.5 \rightarrow 1$; $z_{h_2} = 0 - 1 + 1.5 = 0.5 \rightarrow 1$; $z_{\text{out}} = 1 + 1 - 1.5 = 0.5 \rightarrow \hat{y} = 1 \checkmark$

STEP 4 Row 3 $(1, 0)$: $z_{h_1} = 1 + 0 - 0.5 = 0.5 \rightarrow 1$; $z_{h_2} = -1 + 0 + 1.5 = 0.5 \rightarrow 1$; $z_{\text{out}} = 1 + 1 - 1.5 = 0.5 \rightarrow \hat{y} = 1 \checkmark$

STEP 5 Row 4 $(1, 1)$: $z_{h_1} = 1 + 1 - 0.5 = 1.5 \rightarrow 1$; $z_{h_2} = -1 - 1 + 1.5 = -0.5 \rightarrow 0$; $z_{\text{out}} = 1 + 0 - 1.5 = -0.5 \rightarrow \hat{y} = 0 \checkmark$

(x_1, x_2)	z_{h_1}	h_1 (OR)	z_{h_2}	h_2 (NAND)	z_{out}	$\hat{y}$	y
$(0, 0)$	-0.5	0	1.5	1	-0.5	0	0
$(0, 1)$	0.5	1	0.5	1	0.5	1	1
$(1, 0)$	0.5	1	0.5	1	0.5	1	1
$(1, 1)$	1.5	1	-0.5	0	-0.5	0	0

(a) 2-2-1 MLP (2 inputs, 2 hidden step units, 1 output step unit)

(b) rows 2 + 3 give $w_1 + w_2 + 2b > 0$, rows 1 + 4 give $w_1 + w_2 + 2b \leq 0$: contradiction

(c) $h_1 = \text{OR}$, $h_2 = \text{NAND}$, output = $\text{AND}(h_1, h_2) = \text{XOR}$; all step units

(d) $6 + 3 = 9$ parameters

(e) outputs 0, 1, 1, 0 — all four rows correct

✓ ANSWERS AT A GLANCE

(a) 2-2-1 MLP (b) $w_1 + w_2 + 2b$ both > 0 and ≤ 0 (c) OR, NAND $\rightarrow$ AND (d) 9 (e) 0, 1, 1, 0 ✓

! WHERE MARKS ARE LOST

- Saying "XOR is not linearly separable" in (b) without the four inequalities: the marks are for the added inequalities.
- Counting 6 weights and forgetting the 3 biases in (d): the deck's formula is $n_\ell(n_{\ell-1} + 1)$.
- In (e) using $\text{step}(z) = 1$ at $z = 0$ or mixing up which row a bias belongs to; the table has no zero z , so every row is unambiguous.

A sentiment network 1,000 $\rightarrow$ 8 $\rightarrow$ 5:

Q5.2 parameters, why it overfits, activations, mini-batches

8 marks M5 slides 17, 21, 44–46, 50–52 · Book 5, Ch. 3, 7

A sentiment classification system sorts product reviews into 5 categories (very negative ... very positive). Each review is a vector of 1,000 TF-IDF features. The network has a single hidden layer of 8 neurons and 5 output neurons. It is trained on $N = 20,000$ reviews with mini-batches of $B = 128$ for 10 epochs. After training, the training accuracy is 99 % and the validation accuracy is 61 %.

- (a) Calculate the total number of trainable parameters *excluding* biases, showing each layer transition. [2]
- (b) Write the deck's per-layer formula for the parameter count including biases, and use it to give the total *with* biases. [1]
- (c) The model overfits. Analyse, with a number from the architecture in each case, how (i) the narrow hidden layer (8 neurons) and (ii) the single hidden layer (shallow depth) contribute. [2] **justify**
- (d) Recommend the activation for the hidden layer and for the output layer, each with the deck's reason. [1] **justify**
- (e) How many mini-batches (parameter updates) are there per epoch, how many examples are in the last one, and how many updates happen in the 10 epochs? [2]

by hand

parameter count

design

🔗 WHAT THIS TESTS

Slide 21's formula $n_\ell(n_{\ell-1} + 1)$ applied per transition; slide 44–46 on width versus depth (a narrow layer squeezes information, depth builds a hierarchy); slide 50–52 on output and hidden activations; and iterations per epoch $\lceil N/B \rceil$ from slide 17.

▶ HOW TO ATTEMPT

- Weights per transition = (units in) $\times$ (units out). Biases = units out. Two transitions.
- For (c) give the compression ratio 1,000 : 8 and the fact that one hidden layer has no layer above it to combine its features.
- For (e) divide N by B and round up; the remainder is the size of the last batch.

Σ SOLUTION

STEP 1 Input → hidden: $1,000 \times 8 = 8,000$ weights.

(a)

STEP 2 Hidden → output: $8 \times 5 = 40$ weights.

(a)

STEP 3 Total weights = $8,000 + 40 = 8,040$.

(a)

STEP 4 Slide 21: Parameters^(ℓ) = $n_{\ell-1} \times n_{\ell} + n_{\ell} = n_{\ell}(n_{\ell-1} + 1)$. Layer 1: $8(1,000 + 1) = 8,008$.

(b)

Layer 2: $5(8 + 1) = 45$. Total $8,008 + 45 = 8,053$ (13 more than (a): the $8 + 5$ biases).

STEP 5 (i) Narrow: each of the 8 hidden units must summarise all 1,000 features, a $1,000 : 8 =$

(c)

$125 : 1$ squeeze. The layer is an information bottleneck: it cannot keep the many separate word patterns that decide a 5-way sentiment, so it keeps whatever separates the *training* reviews best (rare words tied to particular reviews) and that does not carry over to new reviews. (ii) Shallow: with one hidden layer there is no layer above it to combine simple features into higher-level ones (slide 45: edges → parts → objects; here words → phrases → tone). Without that hierarchy the model cannot represent "not good" as one concept, so it memorises input-label pairs — 99 % on the training set — and its 8 features do not generalise: 61 % on validation.

slide 44: width alone adds parallel features and risks overfitting; depth adds hierarchy and generalises better

STEP 6 Hidden: ReLU (slide 52, "default choice: fast, no vanishing gradient"; gradient 1 for $z > 0$).

(d)

Output: softmax with categorical cross-entropy (slide 50: multi-class with $K = C = 5$ units; outputs in $(0, 1)$ and sum to 1, so $\arg \max$ gives the class).

STEP 7 $N/B = 20,000/128 = 156.25$, so $\lceil 156.25 \rceil = 157$ mini-batches per epoch: 156 full batches

(e)

of 128 ($156 \times 128 = 19,968$) and a last batch of $20,000 - 19,968 = 32$ reviews.

STEP 8 Updates in 10 epochs: $157 \times 10 = 1,570$.

(e)

(a) $8,000 + 40 = 8,040$ weights

(b) $n_{\ell}(n_{\ell-1} + 1): 8,008 + 45 = 8,053$

(c) (i) $125 : 1$ bottleneck, keeps training-specific patterns; (ii) no hierarchy → memorises pairs

(d) ReLU hidden; softmax + CCE output

(e) 157 per epoch, last batch 32, 1,570 updates

✓ ANSWERS AT A GLANCE

(a) 8,040 (b) 8,053 (c) $125 : 1$; no hierarchy (d) ReLU / softmax (e) 157, 32, 1,570

! WHERE MARKS ARE LOST

- Adding the 1,000 inputs as if they were parameters, or writing $1,000 + 8 + 5 = 1,013$.
- In (c) writing "too few neurons" without the $125 : 1$ number, or "too shallow" without saying what depth adds (a hierarchy of features).
- Rounding 156.25 down to 156 in (e): the last 32 reviews would never be used.

Full forward pass through a 2-3-2

Q5.3

network (ReLU hidden, sigmoid outputs)

8 marks M5N slides 3–13; M5 19, 28–31 · Book 5, Ch. 4

Nova's wake-word detector is a small network: 2 inputs (loudness, pitch), a hidden layer of 3 ReLU units, and 2 sigmoid outputs ($\hat{y}_1$ = "is a voice", $\hat{y}_2$ = "is my name"). Use the Numerical deck's column convention $z^{[1]} = W^{[1]}x + b^{[1]}$ with

$$x = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}, \quad W^{[1]} = \begin{bmatrix} 0.4 & 0.3 \\ -0.5 & 0.2 \\ 0.6 & -0.2 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.2 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

$$W^{[2]} = \begin{bmatrix} 0.5 & -0.2 & 0.4 \\ -0.3 & 0.6 & 0.2 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}.$$

- Compute $z^{[1]} = W^{[1]}x + b^{[1]}$, writing every multiplication. [2]
- Apply ReLU to get $a^{[1]}$. Which hidden unit is dead for this input, and why? [1]
- Compute $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$. [2]
- Apply the sigmoid to get $a^{[2]} = \hat{y}$ (3 decimals). [1]
- Compute the binary cross-entropy loss averaged over the two outputs, as the deck does: $L = -\frac{1}{2} \sum_{i=1}^2 [y_i \ln a_i^{[2]} + (1 - y_i) \ln(1 - a_i^{[2]})]$. [2]

by hand

forward pass

BCE

WHAT THIS TESTS

The Numerical deck's forward problem (slides 3–13) with fresh numbers: matrix–vector products written out, ReLU killing a negative unit, sigmoid to 3 decimals, and the deck's averaged BCE. The same steps in the row convention of slide 19 are $z^{(1)} = x^T W^{(1)\top} + b^{(1)\top}$ – the same numbers written sideways.

HOW TO ATTEMPT

- Row j of $W^{[1]}$ times x , plus $b_j^{[1]}$: three lines.
- ReLU: negative $\rightarrow 0$. The zero then kills its whole column in the next product.
- $\sigma(z) = 1/(1 + e^{-z})$; write e^{-z} to 3 decimals first.
- With $y = (0, 1)$ only $\ln(1 - a_1)$ and $\ln a_2$ survive.

SOLUTION

STEP 1 $z_1^{[1]} = 0.4(0.5) + 0.3(1.0) + 0.1 = 0.2 + 0.3 + 0.1 = 0.6$

(a)

STEP 2 $z_2^{[1]} = -0.5(0.5) + 0.2(1.0) - 0.1 = -0.25 + 0.2 - 0.1 = -0.15$

(a)

STEP 3 $z_3^{[1]} = 0.6(0.5) + (-0.2)(1.0) + 0.2 = 0.3 - 0.2 + 0.2 = 0.3$. So $z^{[1]} = (0.6, -0.15, 0.3)^\top$.

(a)

STEP 4 $a^{[1]} = \text{ReLU}(z^{[1]}) = (\max(0, 0.6), \max(0, -0.15), \max(0, 0.3))^\top = (0.6, 0, 0.3)^\top$. Unit h_2 is dead for this input: $z_2^{[1]} = -0.15 \leq 0$, so it outputs 0 and (slide 37) its derivative is 0.

(b)

STEP 5 $z_1^{[2]} = 0.5(0.6) + (-0.2)(0) + 0.4(0.3) + 0.2 = 0.30 + 0 + 0.12 + 0.2 = 0.62$

(c)

STEP 6 $z_2^{[2]} = -0.3(0.6) + 0.6(0) + 0.2(0.3) - 0.1 = -0.18 + 0 + 0.06 - 0.1 = -0.22$. So $z^{[2]} = (0.62, -0.22)^\top$.

(c)

the middle column of $W^{[2]}$ is multiplied by 0: the dead unit contributes nothing

STEP 7 $a_1^{[2]} = \frac{1}{1 + e^{-0.62}} = \frac{1}{1 + 0.538} = 0.650$

STEP 8 $a_2^{[2]} = \frac{1}{1 + e^{0.22}} = \frac{1}{1 + 1.246} = 0.445$. So $\hat{y} = a^{[2]} = (0.650, 0.445)^\top$.

(d) **STEP 9** $y_1 = 0$: term $= (1 - 0) \ln(1 - 0.650) = \ln 0.350 = -1.050$

(e) **STEP 10** $y_2 = 1$: term $= 1 \cdot \ln 0.445 = -0.810$

(e) **STEP 11** $L = -\frac{1}{2} [-1.050 + (-0.810)] = -\frac{1}{2} (-1.860) = 0.930$

(e) both outputs are on the wrong side of 0.5: the loss is high

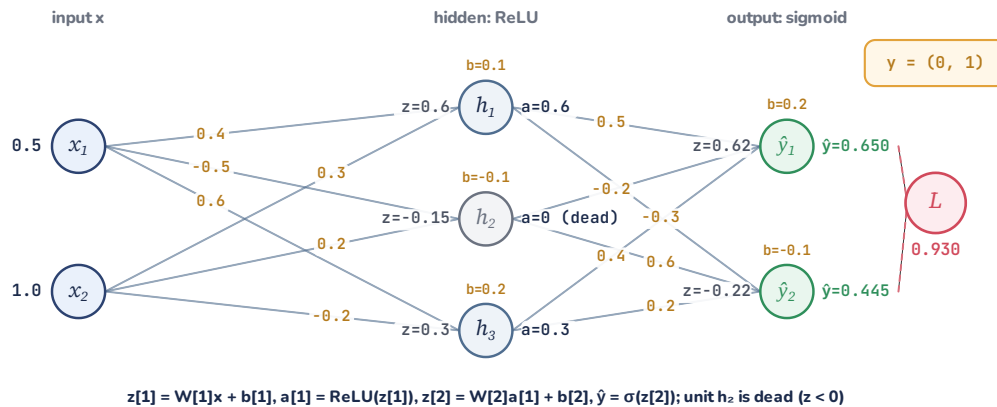


Figure 5.2 The forward pass with every number on its node. Notice h_2 : its z is negative, ReLU makes it 0, and the three weights leaving it are multiplied by 0.

(a) $z^{[1]} = (0.6, -0.15, 0.3)^\top$

(b) $a^{[1]} = (0.6, 0, 0.3)^\top$; h_2 dead ($z = -0.15 \leq 0$)

(c) $z^{[2]} = (0.62, -0.22)^\top$

(d) $a^{[2]} = (0.650, 0.445)^\top$

(e) $L = 0.930$

ANSWERS AT A GLANCE

(a) $(0.6, -0.15, 0.3)$ **(b)** $(0.6, 0, 0.3)$, h_2 dead **(c)** $(0.62, -0.22)$ **(d)** $(0.650, 0.445)$ **(e)** 0.930

WHERE MARKS ARE LOST

- Multiplying x by the columns of $W^{[1]}$ instead of its rows (column convention: $W^{[1]}$ is 3×2 , row j belongs to unit j).
- Leaving $a_2^{[1]} = -0.15$: ReLU must clip it to 0, which changes $z^{[2]}$.
- Sign slip in $\sigma(-0.22)$: $e^{+0.22} = 1.246$, so $a_2 = 0.445 < 0.5$.
- Forgetting the $\frac{1}{2}$ in the deck's averaged BCE (gives 1.860).

Q5.4

Backward pass for the same network: $\delta^{[2]}, \delta^{[1]}$ and all four gradients

8 marks M5N slides 14–27; M5 32–37 · Book 5, Ch. 5

Continuing Q5.3. You have $x = (0.5, 1.0)^\top$, $z^{[1]} = (0.6, -0.15, 0.3)^\top$, $a^{[1]} = (0.6, 0, 0.3)^\top$, $a^{[2]} = (0.650, 0.445)^\top$, $y = (0, 1)^\top$, $W^{[2]} = \begin{bmatrix} 0.5 & -0.2 & 0.4 \\ -0.3 & 0.6 & 0.2 \end{bmatrix}$. Use the deck's conventions: $\delta^{[2]} = a^{[2]} - y$, $\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]}(a^{[1]})^\top$, $\frac{\partial L}{\partial b^{[2]}} = \delta^{[2]}$, $\delta^{[1]} = (W^{[2]})^\top \delta^{[2]} \odot \text{ReLU}'(z^{[1]})$, $\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]}x^\top$, $\frac{\partial L}{\partial b^{[1]}} = \delta^{[1]}$.

(a) Compute the output error $\delta^{[2]}$. [1]

- (b) Compute $\frac{\partial L}{\partial W^{[2]}}$ (a 2×3 matrix, every product written) and $\frac{\partial L}{\partial b^{[2]}}$. [2]
- (c) Backpropagate to the hidden layer: compute $(W^{[2]})^\top \delta^{[2]}$. [2]
- (d) Multiply element-wise by $\text{ReLU}'(z^{[1]})$ to get $\delta^{[1]}$. [1]
- (e) Compute $\frac{\partial L}{\partial W^{[1]}}$ (3×2) and $\frac{\partial L}{\partial b^{[1]}}$. List every entry of the four gradients that is exactly zero and give the reason for each group. [2]

by hand

backpropagation

dead ReLU

WHAT THIS TESTS

The deck's backward problem (slides 14–27) on fresh numbers: the clean output error $\hat{y} - y$ of slide 32, the outer product for weight gradients, the transpose-and-Hadamard step of slide 34, and what a dead ReLU unit does to the gradients.

HOW TO ATTEMPT

- $\delta^{[2]}$ is two subtractions. Its outer product with $a^{[1]}$ gives a 2×3 matrix: row k is δ_k times $a^{[1]}$.
- $(W^{[2]})^\top$ is 3×2 : each hidden unit collects $\delta_1 w_{1j} + \delta_2 w_{2j}$.
- $\text{ReLU}'(z) = 1$ for $z > 0$, else 0 (slide 37) — the middle entry is 0.

SOLUTION

STEP 1 $\delta^{[2]} = a^{[2]} - y = (0.650 - 0, 0.445 - 1)^\top = (0.650, -0.555)^\top$

(a) slide 32: for sigmoid + BCE the output error is simply $\hat{y} - y$

STEP 2 $\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]}(a^{[1]})^\top = \begin{bmatrix} 0.650 \\ -0.555 \end{bmatrix} \begin{bmatrix} 0.6 & 0 & 0.3 \end{bmatrix} = \begin{bmatrix} 0.650(0.6) & 0.650(0) & 0.650(0.3) \\ -0.555(0.6) & -0.555(0) & -0.555(0.3) \end{bmatrix}$

STEP 3 $= \begin{bmatrix} 0.390 & 0 & 0.195 \\ -0.333 & 0 & -0.166 \end{bmatrix}$

(b) row k is δ_k times the whole of $a^{[1]}$; the middle column is 0

STEP 4 $\frac{\partial L}{\partial b^{[2]}} = \delta^{[2]} = (0.650, -0.555)^\top$

STEP 5 $(W^{[2]})^\top \delta^{[2]} = \begin{bmatrix} 0.5 & -0.3 \\ -0.2 & 0.6 \\ 0.4 & 0.2 \end{bmatrix} \begin{bmatrix} 0.650 \\ -0.555 \end{bmatrix}$: row 1: $0.5(0.650) + (-0.3)(-0.555) = 0.325 + 0.167 = 0.492$

STEP 6 row 2: $-0.2(0.650) + 0.6(-0.555) = -0.130 - 0.333 = -0.463$

(c)

STEP 7 row 3: $0.4(0.650) + 0.2(-0.555) = 0.260 - 0.111 = 0.149$. So $(W^{[2]})^\top \delta^{[2]} = (0.492, -0.463, 0.149)^\top$.

STEP 8 $\text{ReLU}'(z^{[1]}) = (1, 0, 1)^\top$ since $0.6 > 0$, $-0.15 \leq 0$, $0.3 > 0$. $\delta^{[1]} = (0.492, -0.463, 0.149)^\top \odot (1, 0, 1)^\top = (0.492, 0, 0.149)^\top$.

(d) the dead unit's error is switched off before it reaches its weights

STEP 9 $\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]}x^\top = \begin{bmatrix} 0.492 \\ 0 \\ 0.149 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 \end{bmatrix} = \begin{bmatrix} 0.492(0.5) & 0.492(1.0) \\ 0(0.5) & 0(1.0) \\ 0.149(0.5) & 0.149(1.0) \end{bmatrix} = \begin{bmatrix} 0.246 & 0.492 \\ 0 & 0 \\ 0.075 & 0.149 \end{bmatrix}$

STEP 10 $\frac{\partial L}{\partial b^{[1]}} = \delta^{[1]} = (0.492, 0, 0.149)^\top$

STEP 11 Zero entries. Group 1 — the middle column of $\partial L / \partial W^{[2]}$ (two entries): they multiply $a_2^{[1]} = 0$; the weights leaving the dead unit get no gradient because the unit sent nothing forward. Group 2 — the middle row of $\partial L / \partial W^{[1]}$ (two entries) and $\partial L / \partial b_2^{[1]}: \text{ReLU}'(z_2^{[1]}) = 0$

makes $\delta_2^{[1]} = 0$; the weights *entering* the dead unit get no gradient. Five zeros in total: the dead unit is not updated on either side (slide 24: "dead neuron gradient = 0, no update").

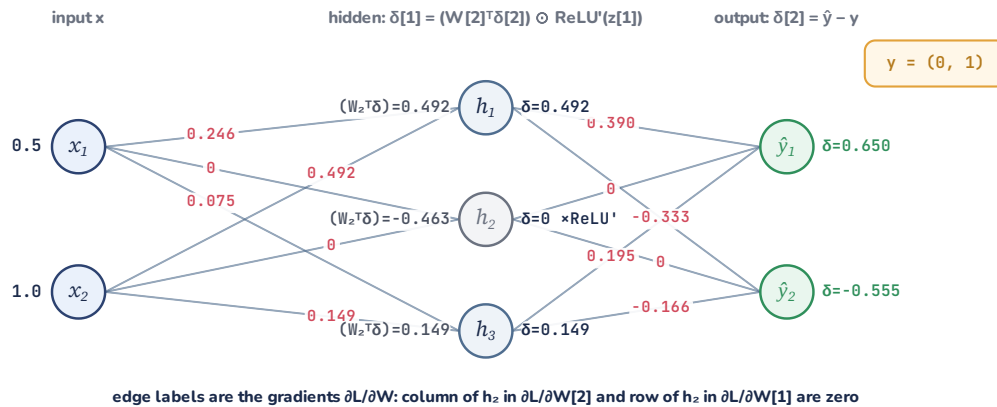


Figure 5.3 The backward pass. Node labels are the error signals δ ; edge labels are the weight gradients. Every edge touching the dead unit h_2 carries a zero.

(a) $\delta^{[2]} = (0.650, -0.555)^T$

(b) $\partial L / \partial W^{[2]} = \begin{bmatrix} 0.390 & 0 & 0.195 \\ -0.333 & 0 & -0.166 \end{bmatrix}, \partial L / \partial b^{[2]} = (0.650, -0.555)^T$

(c) $(W^{[2]})^T \delta^{[2]} = (0.492, -0.463, 0.149)^T$

(d) $\delta^{[1]} = (0.492, 0, 0.149)^T$

(e) $\partial L / \partial W^{[1]} = \begin{bmatrix} 0.246 & 0.492 \\ 0 & 0 \\ 0.075 & 0.149 \end{bmatrix}, \partial L / \partial b^{[1]} = (0.492, 0, 0.149)^T$; 5 zeros: column of $a_2^{[1]} = 0$, row of $\text{ReLU}' = 0$

ANSWERS AT A GLANCE

(a) $(0.650, -0.555)$ (b) rows $(0.390, 0, 0.195), (-0.333, 0, -0.166)$ (c) $(0.492, -0.463, 0.149)$ (d) $(0.492, 0, 0.149)$ (e) rows $(0.246, 0.492), (0, 0), (0.075, 0.149)$

WHERE MARKS ARE LOST

- Writing $\delta^{[2]} = y - a^{[2]}$: every gradient flips sign and the update in Q5.5 goes uphill.
- Using $W^{[2]}$ instead of $(W^{[2]})^T$ in (c): the shapes $(2 \times 3)(2 \times 1)$ do not even multiply.
- Forgetting the $\odot \text{ReLU}'$ step and letting -0.463 through: the dead unit would wrongly get an update.
- Building $\partial L / \partial W^{[1]}$ as $x \delta^{[1]T}$ ($a \times 3$): it must match the shape of $W^{[1]}$, 3×2 .

Q5.5

Update all four parameter blocks with $\alpha = 0.1$, then show the loss fell

8 marks M5N slides 28–31; M5 39 · Book 5, Ch. 5

Continuing Q5.4. The old parameters are those of Q5.3 and the gradients are those of Q5.4:

$$\frac{\partial L}{\partial W^{[2]}} = \begin{bmatrix} 0.390 & 0 & 0.195 \\ -0.333 & 0 & -0.166 \end{bmatrix}, \quad \frac{\partial L}{\partial b^{[2]}} = \begin{bmatrix} 0.650 \\ -0.555 \end{bmatrix},$$

$$\frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} 0.246 & 0.492 \\ 0 & 0 \\ 0.075 & 0.149 \end{bmatrix}, \quad \frac{\partial L}{\partial b^{[1]}} = \begin{bmatrix} 0.492 \\ 0 \\ 0.149 \end{bmatrix}.$$

- (a) Update $W^{[2]}$ and $b^{[2]}$ with $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \partial L / \partial \theta, \alpha = 0.1$. [2]
- (b) Update $W^{[1]}$ and $b^{[1]}$. [2]
- (c) Re-run the forward pass of Q5.3 with the new parameters (same x , same y): $z^{[1]}, a^{[1]}, z^{[2]}, a^{[2]}$ and the new loss. By how much, and by what percentage, did the loss fall? [3]
- (d) What happened to the weights entering the dead unit h_2 ? Is it still dead after the update? Name one change from the deck that would stop this. [1] **justify**

by hand

parameter update

dying ReLU

🔗 WHAT THIS TESTS

The deck's update problem (slides 28–31): subtract α times each gradient entry, then prove the step helped by recomputing the loss. Also the dying-ReLU caveat of slide 13: a unit with zero gradient never changes.

▶ HOW TO ATTEMPT

- Multiply every gradient entry by 0.1, then subtract from the old value. Entries with gradient 0 stay as they were.
- Redo the four forward lines of Q5.3 with the new numbers; keep 3 decimals.
- Compare L_{new} with 0.930.

Σ SOLUTION

STEP 1 $W_{\text{new}}^{[2]} = \begin{bmatrix} 0.5 - 0.1(0.390) & -0.2 - 0.1(0) & 0.4 - 0.1(0.195) \\ -0.3 - 0.1(-0.333) & 0.6 - 0.1(0) & 0.2 - 0.1(-0.166) \end{bmatrix}$

STEP 2 $= \begin{bmatrix} 0.5 - 0.039 & -0.2 & 0.4 - 0.020 \\ -0.3 + 0.033 & 0.6 & 0.2 + 0.017 \end{bmatrix} = \begin{bmatrix} 0.461 & -0.2 & 0.381 \\ -0.267 & 0.6 & 0.217 \end{bmatrix}$

a negative gradient entry means the weight goes up

STEP 3 $b_{\text{new}}^{[2]} = \begin{bmatrix} 0.2 - 0.1(0.650) \\ -0.1 - 0.1(-0.555) \end{bmatrix} = \begin{bmatrix} 0.2 - 0.065 \\ -0.1 + 0.056 \end{bmatrix} = \begin{bmatrix} 0.135 \\ -0.045 \end{bmatrix}$

STEP 4 $W_{\text{new}}^{[1]} = \begin{bmatrix} 0.4 - 0.1(0.246) & 0.3 - 0.1(0.492) \\ -0.5 - 0 & 0.2 - 0 \\ 0.6 - 0.1(0.075) & -0.2 - 0.1(0.149) \end{bmatrix}$

STEP 5 $= \begin{bmatrix} 0.4 - 0.025 & 0.3 - 0.049 \\ -0.5 & 0.2 \\ 0.6 - 0.008 & -0.2 - 0.015 \end{bmatrix} = \begin{bmatrix} 0.375 & 0.251 \\ -0.5 & 0.2 \\ 0.593 & -0.215 \end{bmatrix}$

STEP 6 $b_{\text{new}}^{[1]} = \begin{bmatrix} 0.1 - 0.1(0.492) \\ -0.1 - 0 \\ 0.2 - 0.1(0.149) \end{bmatrix} = \begin{bmatrix} 0.1 - 0.049 \\ -0.1 \\ 0.2 - 0.015 \end{bmatrix} = \begin{bmatrix} 0.051 \\ -0.1 \\ 0.185 \end{bmatrix}$

STEP 7 $z_1^{[1]} = 0.3754(0.5) + 0.2508(1.0) + 0.0508 = 0.1877 + 0.2508 + 0.0508 = 0.489$ carry a fourth decimal in the updated weights so the sums round correctly

STEP 8 $z_2^{[1]} = -0.5(0.5) + 0.2(1.0) - 0.1 = -0.15$ (unchanged); $z_3^{[1]} = 0.593(0.5) - 0.215(1.0) + 0.185 = 0.296 - 0.215 + 0.185 = 0.266$

STEP 9 $a^{[1]} = (0.489, 0, 0.266)^T$

STEP 10 $z_1^{[2]} = 0.461(0.489) + (-0.2)(0) + 0.381(0.266) + 0.135 = 0.2254 + 0 + 0.1013 + 0.135 = 0.462$

STEP 11 $z_2^{[2]} = -0.267(0.489) + 0.6(0) + 0.217(0.266) - 0.0445 = -0.1306 + 0 + 0.0577 -$

(c) $0.0445 = -0.117$

STEP 12 $a_1^{[2]} = \sigma(0.462) = 0.614; a_2^{[2]} = \sigma(-0.117) = 0.471$

(c) $\hat{y}_1$ moved down toward 0 and $\hat{y}_2$ moved up toward 1: both in the right direction

STEP 13 $L_{\text{new}} = -\frac{1}{2} [\ln(1 - 0.614) + \ln 0.471] = -\frac{1}{2} [\ln 0.386 + \ln 0.471] = -\frac{1}{2} [-0.952 - 0.753] =$

(c) 0.852

STEP 14 Fall: $0.930 - 0.852 = 0.078$, i.e. $0.078/0.930 = 8.4\%$ in one step.

(c)

STEP 15 Row 2 of $W^{[1]}$ and $b_2^{[1]}$ are unchanged $(-0.5, 0.2; -0.1)$ because their gradients were 0. So $z_2^{[1]} = -0.15$ again: the unit is still dead for this input, and will stay dead for it forever – the dying-ReLU problem of slide 13. Fixes from the deck: use Leaky ReLU (or PReLU/ELU), which has a small non-zero slope for $z < 0$ so $\delta_2^{[1]} \neq 0$; or proper initialisation (He init, slide 55) / a smaller learning rate so units are less likely to be pushed into $z < 0$ for all inputs.

(a) $W^{[2]} = \begin{bmatrix} 0.461 & -0.2 & 0.381 \\ -0.267 & 0.6 & 0.217 \end{bmatrix}, b^{[2]} = (0.135, -0.045)^\top$

(b) $W^{[1]} = \begin{bmatrix} 0.375 & 0.251 \\ -0.5 & 0.2 \\ 0.593 & -0.215 \end{bmatrix}, b^{[1]} = (0.051, -0.1, 0.185)^\top$

(c) $z^{[1]} = (0.489, -0.15, 0.266), a^{[1]} = (0.489, 0, 0.266), z^{[2]} = (0.462, -0.117), a^{[2]} = (0.614, 0.471), L = 0.852$; fell by **0.078 (8.4 %)**

(d) unchanged, still dead; Leaky ReLU (or He init / smaller α)

✓ ANSWERS AT A GLANCE

(a) $b^{[2]} = (0.135, -0.045)$ (b) $b^{[1]} = (0.051, -0.1, 0.185)$ (c) $\hat{y} = (0.614, 0.471), L = 0.852, -8.4 \%$ (d) unchanged; Leaky ReLU

! WHERE MARKS ARE LOST

- Adding $\alpha \partial L / \partial \theta$ instead of subtracting: the new loss would rise above 0.930.
- Updating with $\alpha = 1$ (forgetting the 0.1): $W_{11}^{[2]}$ would become 0.11, not 0.461.
- In (c) reusing the old $a^{[1]}$ with the new $W^{[2]}$: both layers changed, so the whole pass must be redone.
- Saying the dead unit "will recover next step": with ReLU and this input its gradient is 0 every time.

Examine an engineer's

Q5.6 architecture spec: parameters, flaws, and a corrected spec

8 marks M5 slides 21, 37, 50–52, 55, 60 · Book 5, Ch. 3, 7, 8

An engineer proposes the following network for classifying a machine's vibration reading (12 sensor features) into 4 fault classes:

Input: 12 nodes · **Hidden 1:** 6 nodes, tanh · **Hidden 2:** 10 nodes, sigmoid · **Hidden 3:** 3 nodes, tanh · **Output:** 4 nodes, softmax · **Loss:** mean squared error · **Initialisation:** all weights and biases set to 0

(a) Count the trainable parameters (weights + biases) of the spec, layer by layer. [2]

- (b) Examine the specification and identify every flaw, inconsistency or improvement. For each one state the fact from the deck that makes it a flaw. [3] **justify**
- (c) A corrected spec is $12 \rightarrow 16 \text{ ReLU} \rightarrow 16 \text{ ReLU} \rightarrow 4 \text{ softmax}$ with categorical cross-entropy and He initialisation. Count its parameters layer by layer. [2]
- (d) By what factor is the corrected network larger? In the original spec, what is the largest possible factor $|\sigma'(z)|$ by which the error signal is multiplied when it passes back through one unit of Hidden 2, and what is that factor for an active ReLU unit? [1]

read a spec

parameter count

design flaws

🔗 WHAT THIS TESTS

Mid-sem MS-6(b) in this module's language: slide 52 (sigmoid "avoid for hidden layers, strong saturation"; tanh "avoid in very deep networks"), slide 50 (softmax pairs with CCE), slide 55 ("bad: zero initialisation"), slide 51 (10–100 units per layer), and slide 21's parameter formula.

▶ HOW TO ATTEMPT

- Four transitions in (a): $12 \rightarrow 6$, $6 \rightarrow 10$, $10 \rightarrow 3$, $3 \rightarrow 4$. Weights + biases each.
- For (b) go item by item through the spec: each hidden activation, the width of each layer, the loss, the initialisation.
- Slide 37 gives $\sigma'(z) = \sigma(1 - \sigma) \leq 0.25$; $\text{ReLU}'(z) = 1$ for $z > 0$.

Σ SOLUTION

STEP 1 Hidden 1: $12 \times 6 + 6 = 72 + 6 = 78$

(a)

STEP 2 Hidden 2: $6 \times 10 + 10 = 60 + 10 = 70$

(a)

STEP 3 Hidden 3: $10 \times 3 + 3 = 30 + 3 = 33$

(a)

STEP 4 Output: $3 \times 4 + 4 = 12 + 4 = 16$. Total $78 + 70 + 33 + 16 = 197$ parameters.

(a)

STEP 5 Flaw 1 — sigmoid in Hidden 2. Slide 52: "Sigmoid: avoid for hidden layers (strong saturation)"; slide 37: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ is at most 0.25, so every backward pass through that layer shrinks the error signal by at least 4×, and by far more once $|z|$ is large (vanishing gradient). Replace with ReLU (slide 52: default choice).

(b)

STEP 6 Flaw 2 — a 3-unit layer just before a 4-class softmax. Hidden 3 squeezes everything into 3 numbers, fewer than the 4 classes it must separate: a bottleneck that throws away information the output needs. It is also far below the deck's "10–100 units per layer" starting guideline (slide 51). Widen it (or drop it).

(b)

STEP 7 Flaw 3 — MSE with a softmax output. Slide 50 pairs multi-class softmax with categorical cross-entropy; slide 32's clean output error $\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$ holds only for the matched pairs (MSE + identity, BCE + sigmoid, CCE + softmax). MSE on probabilities gives small, saturating gradients for confident wrong answers; CCE penalises them heavily (M5N slide 70). Use CCE.

(b)

STEP 8 Flaw 4 — zero initialisation. Slide 55: "Bad: zero initialization". With all weights 0, every unit in a layer computes the same z , gets the same δ , and receives the same update, so the units stay identical forever and the layer acts like one unit; slide 60's checklist says "weights initialised properly (not all zeros!)". Use Xavier for tanh/sigmoid or He for ReLU; biases may be 0.

(b)

STEP 9 Minor: the widths $6 \rightarrow 10 \rightarrow 3$ first widen then collapse; tanh in Hidden 1 and 3 is acceptable for a small network (slide 52) but ReLU is the modern default. Not flaws by the deck: 4 softmax units for 4 classes and 12 input nodes for 12 features are correct.

STEP 10 Hidden 1: $12 \times 16 + 16 = 192 + 16 = 208$

STEP 11 Hidden 2: $16 \times 16 + 16 = 256 + 16 = 272$

STEP 12 Output: $16 \times 4 + 4 = 64 + 4 = 68$. Total $208 + 272 + 68 = 548$ parameters.

STEP 13 Factor $548/197 = 2.78$, about 2.8 times larger. Through one sigmoid unit the error is multiplied by $\sigma'(z) \leq 0.25$ (equality only at $z = 0$); through an active ReLU unit by exactly 1 (slide 13: "gradient is 1 for $z > 0$, flows unchanged").

(a) $78 + 70 + 33 + 16 = 197$

(b) sigmoid hidden layer ($\sigma' \leq 0.25$); 3-unit bottleneck before 4 classes; MSE with softmax $\rightarrow$ CCE; zero init $\rightarrow$ identical units

(c) $208 + 272 + 68 = 548$

(d) $\times 2.78$; 0.25 versus 1

✓ ANSWERS AT A GLANCE

(a) 197 (b) 4 flaws (c) 548 (d) 2.78; 0.25 vs 1

! WHERE MARKS ARE LOST

- Counting the input layer as a parameter layer (there is no 12-parameter "layer 0").
- Listing "sigmoid is bad" with no number: the mark is for the 0.25 bound.
- Calling the 4-unit softmax output a flaw: it matches the 4 classes.
- Saying zero init "makes learning slow": it makes all units of a layer identical, which no amount of training undoes.

Q5.7

Gradient flow through five sigmoid layers, and the same chain with ReLU

8 marks M5N slides 33–34; M5 13, 47 · Book 5, Ch. 6

A 5-layer network uses sigmoid activations in every layer. All weight matrices have spectral norm $\|W^{[l]}\| = 0.9$; the sigmoid derivative at every layer is at its maximum, $g'^{[l]} = 0.25$; and the gradient magnitude at the output is $|\delta^{[5]}| = 1$. Use the deck's propagation rule $|\delta^{[l-1]}| = |\delta^{[l]}| \cdot \|W^{[l]}\| \cdot |g'^{[l-1]}|$.

- Compute $|\delta^{[4]}|, |\delta^{[3]}|, |\delta^{[2]}|, |\delta^{[1]}|$. [2]
- What fraction (as a percentage) of the output gradient reaches layer 1? [1]
- Repeat (a) if every layer uses ReLU and all units are active ($g' = 1$), with the same $\|W\| = 0.9$. How many times larger is $|\delta^{[1]}|$ than in the sigmoid case? [2]
- With sigmoid, how many backward steps k are needed before $|\delta| < 10^{-3}$, and so how many layers must the network have for its first layer to receive less than 10^{-3} ? Give the same k for the ReLU chain. [2]
- Justify, using the bound on the sigmoid derivative, why slide 47 lists "ReLU activations" as the first solution to vanishing gradients. [1] **justify**

Ⓢ WHAT THIS TESTS

The Numerical deck's gradient-flow problem (slides 33–34) with a longer chain: the per-layer factor $\|W\| \cdot g'$, geometric shrinking, a logarithm to count layers, and the reason ReLU's slope of 1 keeps the signal alive.

▶ HOW TO ATTEMPT

- The factor per layer is $0.9 \times 0.25 = 0.225$. Multiply four times.
- For ReLU the factor is $0.9 \times 1 = 0.9$.
- For (d) solve $0.225^k < 10^{-3}$ with logarithms; round k up; check by multiplying.

Σ SOLUTION

STEP 1 Factor per layer: $\|W\| \cdot g' = 0.9 \times 0.25 = 0.225$.

(a)

STEP 2 $|\delta^{[4]}| = 1 \times 0.9 \times 0.25 = 0.225$

(a)

STEP 3 $|\delta^{[3]}| = 0.225 \times 0.225 = 0.0506$

(a)

STEP 4 $|\delta^{[2]}| = 0.0506 \times 0.225 = 0.0114$

(a)

STEP 5 $|\delta^{[1]}| = 0.0114 \times 0.225 = 0.00256$ (exactly $0.225^4 = 0.002563$)

(a)

STEP 6 $0.002563/1 = 0.256\%$: about a quarter of one percent of the output gradient reaches the first layer.

(b)

the deck's 4-layer example kept 0.8 %; one more layer and a slightly larger $\|W\|$ leave even less

STEP 7 Factor $0.9 \times 1 = 0.9$: $|\delta^{[4]}| = 0.9$, $|\delta^{[3]}| = 0.81$, $|\delta^{[2]}| = 0.729$, $|\delta^{[1]}| = 0.656$ ($= 0.9^4$).

(c)

STEP 8 Ratio $0.6561/0.002563 = 256 = 4^4$: each of the four sigmoid steps lost a factor 4 that ReLU keeps.

(c)

STEP 9 Sigmoid: $0.225^k < 10^{-3} \Rightarrow k > \frac{\ln 10^{-3}}{\ln 0.225} = \frac{-6.908}{-1.492} = 4.63$, so $k = 5$ backward steps.

(d)

Check: $0.225^4 = 0.00256 > 10^{-3}$, $0.225^5 = 0.000577 < 10^{-3}$. Five steps back from the output means layer $L - 5$: a network with $L = 6$ layers has its first layer below 10^{-3} .

STEP 10 ReLU: $0.9^k < 10^{-3} \Rightarrow k > \frac{-6.908}{\ln 0.9} = \frac{-6.908}{-0.1054} = 65.6$, so $k = 66$ steps ($0.9^{65} = 0.00106$,

(d)

$0.9^{66} = 0.000955$): a 67-layer network.

STEP 11 Key fact: $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$ everywhere (M5N slide 46), so each sigmoid layer multiplies the error by at most $0.25\|W\|$ and the product over L layers shrinks like 0.25^L at best; with $\|W\| < 1$ it vanishes faster still. $\text{ReLU}'(z) = 1$ for every active unit (slide 13: "flows unchanged"), so the only shrinking factor left is $\|W\|$, which good initialisation (He) keeps near 1. That is why ReLU makes very deep networks trainable and heads slide 47's list.

(e)

Q5.7: the error signal shrinks as it travels back (5 layers, $\|W\| = 0.9$)

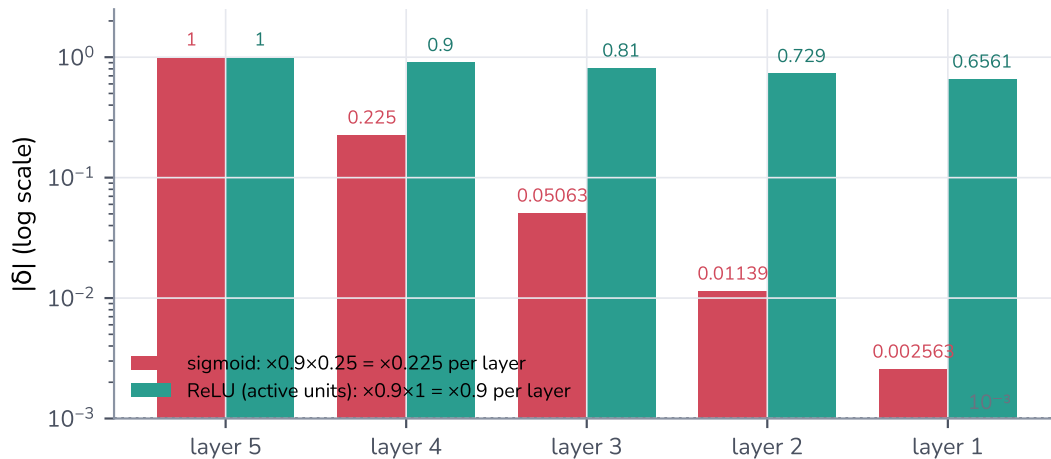


Figure 5.4 The error signal per layer on a log scale. The sigmoid bars fall by a factor 0.225 each step and reach 0.00256 at layer 1; the ReLU bars fall by only 0.9.

(a) 0.225, 0.0506, 0.0114, 0.00256

(b) 0.256%

(c) 0.9, 0.81, 0.729, 0.656; 256 times larger

(d) sigmoid $k = 5$ (a 6-layer network); ReLU $k = 66$

(e) $\sigma' \leq 0.25 \rightarrow \text{shrink} \leq 0.25^L$; $\text{ReLU}' = 1 \rightarrow \text{no shrink from the activation}$

✓ ANSWERS AT A GLANCE

(a) 0.225, 0.0506, 0.0114, 0.00256 (b) 0.256 % (c) 0.9 ... 0.656; $\times 256$ (d) 5 (6 layers); 66 (e) 0.25 vs 1

! WHERE MARKS ARE LOST

- Applying the factor five times for a 5-layer network: there are only four backward steps from layer 5 to layer 1.
- Using 0.25 alone as the factor and forgetting $\|W\| = 0.9$ (gives 0.0039 at layer 1).
- Rounding 4.63 down to 4 in (d): after 4 steps the signal is still 0.00256.
- In (e) saying "ReLU has no vanishing gradient" without the two numbers 0.25 and 1.

Q5.8

Sigmoid, tanh and ReLU with their derivatives at four pre-activations

8 marks M5N slides 44–56; M5 37 · Book 5, Ch. 2

Four hidden units of Nova's network have pre-activations $z = (1.5, -0.5, 0, 2.0)^T$. Use $e^{-1.5} = 0.223$, $e^{0.5} = 1.649$, $e^{-0.5} = 0.607$, $e^{1.5} = 4.482$, $e^2 = 7.389$, $e^{-2} = 0.135$.

- Compute $\sigma(z)$ and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ for each entry. [2]
- Compute $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ and $\tanh'(z) = 1 - \tanh^2(z)$ for each entry. [2]
- Compute $\text{ReLU}(z)$ and $\text{ReLU}'(z)$ for each entry, using the deck's rule $\text{ReLU}'(z) = 0$ for $z \leq 0$. [1]
- Which activation has the largest derivative at $z = 0$? By what factor is it larger than the sigmoid's? [1]

- (e) State the deck's two reasons for preferring tanh over sigmoid in a hidden layer. [1] **justify**
- (f) Under ReLU, which of the four units are dead for this input, and what does that do to the error signal δ that reaches their incoming weights? [1]

by hand activations derivatives

WHAT THIS TESTS

The Numerical deck's activation problem (slides 53–56) with fresh inputs, including the special point $z = 0$: $\sigma' = 0.25$, $\tanh' = 1$, $\text{ReLU}' = 0$. Also the tanh-versus-sigmoid facts of slides 50–52.

HOW TO ATTEMPT

- Sigmoid: $1/(1 + e^{-z})$; the derivative needs only the value.
- Tanh: use the given exponentials; $\tanh(-z) = -\tanh(z)$ saves work.
- Put all twelve values in one table; the $z = 0$ column answers (d).

SOLUTION

STEP 1 $\sigma(1.5) = \frac{1}{1+0.223} = 0.818$; $\sigma(-0.5) = \frac{1}{1+1.649} = 0.378$; $\sigma(0) = \frac{1}{1+1} = 0.5$; $\sigma(2.0) = \frac{1}{1+0.135} = 0.881$

STEP 2 $\sigma': 0.818(1 - 0.818) = 0.149$; $0.378(1 - 0.378) = 0.235$; $0.5(0.5) = 0.25$; $0.881(1 - 0.881) = 0.105$

STEP 3 $\tanh(1.5) = \frac{4.482-0.223}{4.482+0.223} = \frac{4.259}{4.705} = 0.905$; $\tanh(-0.5) = \frac{0.607-1.649}{0.607+1.649} = \frac{-1.042}{2.256} = -0.462$

STEP 4 $\tanh(0) = \frac{1-1}{1+1} = 0$; $\tanh(2.0) = \frac{7.389-0.135}{7.389+0.135} = \frac{7.254}{7.524} = 0.964$

STEP 5 $\tanh': 1 - 0.905^2 = 1 - 0.819 = 0.180$; $1 - (-0.462)^2 = 1 - 0.213 = 0.786$; $1 - 0 = 1$; $1 - 0.964^2 = 1 - 0.929 = 0.070$

STEP 6 $\text{ReLU}(z) = (\max(0, 1.5), \max(0, -0.5), \max(0, 0), \max(0, 2.0)) = (1.5, 0, 0, 2.0)$;
 $\text{ReLU}'(z) = (1, 0, 0, 1)$ since only 1.5 and 2.0 are > 0 .

z	$\sigma(z)$	$\sigma'(z)$	$\tanh(z)$	$\tanh'(z)$	$\text{ReLU}(z)$	$\text{ReLU}'(z)$
1.5	0.818	0.149	0.905	0.180	1.5	1
-0.5	0.378	0.235	-0.462	0.786	0	0
0	0.5	0.25	0	1	0	0
2.0	0.881	0.105	0.964	0.070	2.0	1

STEP 1 At $z = 0$: $\sigma' = 0.25$, $\tanh' = 1$, $\text{ReLU}' = 0$ (deck rule for $z \leq 0$). Tanh has the largest, $1/0.25 = 4$ times the sigmoid's (M5N slide 52: "4× larger than sigmoid").

STEP 2 Slide 50: (1) tanh is zero-centred — $\tanh(0) = 0$ and outputs lie in $(-1, 1)$, so the next layer's inputs are not all positive; (2) stronger gradients — $\max \tanh' = 1$ against $\max \sigma' = 0.25$, so the error signal shrinks less per layer and convergence is faster.

STEP 3 Units 2 and 3 ($z = -0.5$ and $z = 0$): $\text{ReLU}' = 0$, so $\delta = (\dots) \odot 0 = 0$ for them and their incoming weights and biases get no update from this input (slide 24: "dead neuron gradient = 0").

(a) $\sigma = (0.818, 0.378, 0.5, 0.881)$, $\sigma' = (0.149, 0.235, 0.25, 0.105)$

(b) $\tanh = (0.905, -0.462, 0, 0.964)$, $\tanh' = (0.180, 0.786, 1, 0.070)$

(c) $\text{ReLU} = (1.5, 0, 0, 2.0)$, $\text{ReLU}' = (1, 0, 0, 1)$

(d) tanh, 1 versus 0.25: 4 times

(e) zero-centred; max slope 1 versus 0.25

(f) units 2 and 3; $\delta = 0$, no update

✓ ANSWERS AT A GLANCE

(a) σ' : 0.149, 0.235, 0.25, 0.105 (b) $\tanh'$: 0.180, 0.786, 1, 0.070 (c) (1, 0, 0, 1) (d) tanh, $\times 4$ (e) zero-centred, 1 vs 0.25 (f) units 2, 3

! WHERE MARKS ARE LOST

- $\sigma(-0.5)$ computed with $e^{-0.5}$ in the denominator (gives 0.622, the value of $\sigma(+0.5)$).
- Writing $\tanh'(z) = 1 - \tanh(z)$ instead of $1 - \tanh^2(z)$.
- Giving $\text{ReLU}'(0) = 1$ or "undefined": the deck defines it as 0 for $z \leq 0$.
- In (e) saying "tanh is smoother": the two facts are zero-centred and the slope bound 1 versus 0.25.

Stable softmax, cross-entropy and its gradient; MSE, MAE and BCE derivatives

Q5.9

8 marks M5N slides 47–49, 57–61, 64–73 · Book 5, Ch. 4, 5

Nova's 4-way sound classifier (classes: name, other voice, music, noise) produces the logits $z = (1.0, 2.5, 0.5, -1.0)^\top$ for a clip whose true class is 2 ("other voice"), so $y = (0, 1, 0, 0)^\top$. Use $e^{-1.5} = 0.223$, $e^{-2} = 0.135$, $e^{-3.5} = 0.030$. Separately, a regression head predicts $\hat{y} = 4.2$ when the true value is $y = 5$, and a binary head predicts $\hat{y} = 0.8$ when $y = 1$.

- (a) Compute the softmax probabilities using the numerically stable form (subtract the maximum logit first). [2]
- (b) Compute the cross-entropy loss $L = -\sum_i y_i \ln \hat{y}_i$. [1]
- (c) Compute the gradient $\partial L / \partial z = \hat{\mathbf{y}} - \mathbf{y}$ and check that its entries sum to zero. [1]
- (d) For the regression head compute the MSE loss $\ell = \frac{1}{2}(\hat{y} - y)^2$ and its derivative $\partial \ell / \partial \hat{y}$, and the MAE loss $\ell = |\hat{y} - y|$ and its derivative. [2]
- (e) For the binary head compute the BCE loss and its derivative $\partial \ell / \partial \hat{y} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$. [1]
- (f) Justify, using the softmax property from the deck, why subtracting the maximum logit does not change the probabilities, and state what it prevents. [1] **justify**

by hand

softmax + CE

loss derivatives

🔗 WHAT THIS TESTS

The Numerical deck's softmax problem (slides 57–61) and its loss-derivative examples (slides 64–73) with fresh numbers: the stable shift, $-\ln$ of the true-class probability, the clean gradient $\hat{\mathbf{y}} - \mathbf{y}$, and the three scalar derivatives $\hat{y} - y$, $\text{sign}(\hat{y} - y)$, $(\hat{y} - y)/(\hat{y}(1 - \hat{y}))$.

▶ HOW TO ATTEMPT

- Subtract 2.5 from every logit; the largest becomes 0 and $e^0 = 1$. Sum the four exponentials, divide.
- Only the true class contributes to the loss: $-\ln \hat{y}_2$.
- The gradient is the probability vector with 1 subtracted at the true class.

- Use the deck's derivative table (M5N slide 79) for (d) and (e).

Σ SOLUTION

STEP 1 $\max z = 2.5$. $z - \max z = (1.0 - 2.5, 2.5 - 2.5, 0.5 - 2.5, -1.0 - 2.5) =$
(a) $(-1.5, 0, -2.0, -3.5)$

STEP 2 $e^{z - \max} = (e^{-1.5}, e^0, e^{-2}, e^{-3.5}) = (0.223, 1, 0.135, 0.030)$; $\text{sum} = 0.223 + 1 + 0.135 +$
(a) $0.030 = 1.388$

STEP 3 $\hat{y} = \frac{1}{1.388}(0.223, 1, 0.135, 0.030) = (0.161, 0.720, 0.097, 0.022)$; check $0.161 + 0.720 +$
(a) $0.097 + 0.022 = 1.000 \checkmark$

STEP 4 $L = -[0 \cdot \ln 0.161 + 1 \cdot \ln 0.720 + 0 + 0] = -\ln 0.720 = 0.328$
(b) the true class already has the highest probability, so the loss is small

STEP 5 $\frac{\partial L}{\partial z} = \hat{y} - y = (0.161 - 0, 0.720 - 1, 0.097 - 0, 0.022 - 0) =$
(c) $(0.161, -0.280, 0.097, 0.022)$; $\text{sum} = 0.161 - 0.280 + 0.097 + 0.022 = 0 \checkmark$ (probabilities sum to 1 and y sums to 1).

STEP 6 $\text{MSE: } \ell = \frac{1}{2}(4.2 - 5)^2 = \frac{1}{2}(-0.8)^2 = \frac{1}{2}(0.64) = 0.32$; $\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y = 4.2 - 5 = -0.8$ (negative:
(d) increase $\hat{y}$).

STEP 7 $\text{MAE: } \ell = |4.2 - 5| = 0.8$; $\frac{\partial \ell}{\partial \hat{y}} = \text{sign}(4.2 - 5) = \text{sign}(-0.8) = -1$.
(d) MAE's slope is always ± 1 ; MSE's grows with the error

STEP 8 $\text{BCE with } y = 1: \ell = -\ln \hat{y} = -\ln 0.8 = 0.223$; $\frac{\partial \ell}{\partial \hat{y}} = \frac{0.8 - 1}{0.8(1 - 0.8)} = \frac{-0.2}{0.16} = -1.25$.
(e)

STEP 9 Key fact (M5N slide 47): softmax is translation invariant, $\text{softmax}(z + c) = \text{softmax}(z)$,
(f) because $e^{z_j+c} / \sum_k e^{z_k+c} = e^c e^{z_j} / (e^c \sum_k e^{z_k})$ and e^c cancels. With $c = -\max z$ every exponent is ≤ 0 , so every $e^{z_j - \max}$ lies in $(0, 1]$: no overflow for large logits (slide 61 of M5: "use stable implementations, e.g. log-sum-exp for softmax"). Check: the unshifted form gives $e^{2.5} / (e^{1.0} + e^{2.5} + e^{0.5} + e^{-1.0}) = 12.18 / 16.92 = 0.720$, the same.

(a) $\hat{y} = (0.161, 0.720, 0.097, 0.022)$

(b) $L = -\ln 0.720 = 0.328$

(c) $(0.161, -0.280, 0.097, 0.022)$, **sum 0**

(d) **MSE 0.32, derivative -0.8; MAE 0.8, derivative -1**

(e) **BCE 0.223, derivative -1.25**

(f) **softmax($z + c$) = softmax(z); prevents overflow of e^z**

✓ ANSWERS AT A GLANCE

(a) 0.161, 0.720, **(b)** 0.328 **(c)** 0.161, -0.280, **(d)** 0.32 / -0.8; **(e)** 0.223 / **(f)** translation
 0.097, 0.022 0.097, 0.022 0.8 / -1 -1.25 invariance

! WHERE MARKS ARE LOST

- Subtracting the maximum and then forgetting that $e^0 = 1$ is one of the four terms.
- Using $\log_{10}$ instead of $\ln$ in (b) (gives 0.143).
- Writing the gradient as $y - \hat{y}$, or leaving $+0.720$ at the true class.

- Forgetting the $\frac{1}{2}$ in MSE (0.64) or dropping the sign of -0.8 ; giving the MAE derivative as -0.8 instead of -1 .

Design a network with the deck's

Q5.10 sizing rules, then count its multiply-adds

8 marks M5N slides 36–41; M5 21, 30 · Book 5, Ch. 7

A clinic wants a network that reads $d = 40$ blood-test features and sorts a patient into $c = 4$ risk groups. There are $M = 2,000$ labelled records. Use the Numerical deck's rules of thumb: $L \leq \log_2(M/(d \cdot c)) + 1$ and $\text{Parameters} \leq M/5$. ($\log_2 12.5 = 3.644$.)

- Compute the maximum number of layers L . [1]
- Compute the parameter budget, then count the parameters (weights + biases, layer by layer) of the candidates $[40, 16, 4]$, $[40, 8, 4]$ and $[40, 6, 4]$ and mark each against the budget. [3]
- Give the final recommendation the deck's procedure leads to, and its number of parameters per training sample. [1]
- Count the multiply-add operations of one forward pass (one example) for the network you chose, and for the deck's $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ network. Then give both counts for a mini-batch of $B = 128$. [2]
- The 784-network has 235,146 parameters. If it were trained on the deck's medium image set with $M = 5,000$, compute its parameters per sample and its budget, and state what the deck concludes for such a case. [1]

by hand

architecture design

operation count

WHAT THIS TESTS

The Numerical deck's three sizing examples (slides 36–41) on a new dataset: the depth bound, the $M/5$ budget, shrinking the hidden layer until it fits; then slide 30's complexity $O(B \sum_{\ell} n_{\ell-1} n_{\ell})$ – one multiply-add per weight.

HOW TO ATTEMPT

- $M/(dc)$ first, then $\log_2$, then $+1$, then round down.
- Each candidate has two transitions; count weights + biases for each and add.
- Multiply-adds per layer = (units in) $\times$ (units out): the weight count without biases. Multiply by B for a batch.

SOLUTION

STEP 1 $\frac{M}{d \cdot c} = \frac{2,000}{40 \times 4} = \frac{2,000}{160} = 12.5$; $L \leq \log_2 12.5 + 1 = 3.644 + 1 = 4.644 \Rightarrow L \leq 4$.

(a)

STEP 2 Budget: $M/5 = 2,000/5 = 400$ parameters.

(b)

STEP 3 $[40, 16, 4]$: $40 \times 16 + 16 = 656$; $16 \times 4 + 4 = 68$; total $656 + 68 = 724 > 400$ ✗

(b)

STEP 4 $[40, 8, 4]$: $40 \times 8 + 8 = 328$; $8 \times 4 + 4 = 36$; total $328 + 36 = 364 \leq 400$ ✓

(b)

STEP 5 $[40, 6, 4]$: $40 \times 6 + 6 = 246$; $6 \times 4 + 4 = 28$; total $246 + 28 = 274 \leq 400$ ✓

(b)

STEP 6 The deck shrinks the hidden layer until the count fits and stops at the first that does (slide 37: [20, 6, 3] "with regularisation"). Here that is [40, 8, 4]: $L = 2 \leq 4$, 364 parameters, $364/2,000 = 0.182$ parameters per sample. Recommendation: [40, 8, 4], ReLU hidden, softmax output, with regularisation (L2 / dropout / early stopping).
 (c) [40, 6, 4] also fits but has less capacity; the deck picks the largest that fits

STEP 7 [40, 8, 4]: $40 \times 8 + 8 \times 4 = 320 + 32 = 352$ multiply-adds per example.

(d)

STEP 8 $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$: $784 \times 256 + 256 \times 128 + 128 \times 10 = 200,704 + 32,768 + 1,280 = 234,752$ multiply-adds per example — the 235,146 parameters minus the $256 + 128 + 10 = 394$ biases, which are additions only.

(d)

STEP 9 For $B = 128$: $352 \times 128 = 45,056$ and $234,752 \times 128 = 30,048,256$, about 30 million multiply-adds per mini-batch (slide 30: $O(B \sum_{\ell} n_{\ell-1} n_{\ell})$).

(d)

STEP 10 $235,146/5,000 = 47.0$ parameters per sample; budget $5,000/5 = 1,000$, so the network is 235 times over budget. The deck's conclusion for the medium image set (slide 39): even [784, 16, 10] with 12,714 parameters is over budget, so "use regularisation techniques (dropout, L2) with larger architectures" rather than shrinking further.

(e)

(a) $L \leq 4.644$, so $L \leq 4$

(b) budget 400; 724 ✗, 364 ✓, 274 ✓

(c) [40, 8, 4] with regularisation; 0.182 parameters per sample

(d) 352 and 234,752 per example; 45,056 and 30,048,256 per batch of 128

(e) 47.0 per sample versus a budget of 1,000: needs regularisation (dropout, L2)

✓ ANSWERS AT A GLANCE

(a) 4 (b) 400; 724 / 364 / 274 (c) [40, 8, 4], 0.182 (d) 352; 234,752; $\times 128$ (e) 47 vs 1,000

! WHERE MARKS ARE LOST

- Taking $\log_2$ of M/d and forgetting c (gives $\log_2 50 + 1 = 6.6$).
- Leaving out the biases in (b): [40, 16, 4] would count 704 and still fail, but [40, 8, 4] would count 352, not 364.
- Recommending [40, 16, 4] because "more capacity is better": it breaks the $M/5$ rule the question sets.
- Counting the biases as multiply-adds in (d): 235,146 instead of 234,752.

Module 6: Convolutional Neural Networks

Ten exam-style questions on the CNN deck: shapes through a whole pipeline, parameters per layer, convolution and pooling by hand, multi-channel filters, a Keras CNN snippet, receptive fields, padding-stride-dilation arithmetic, the 1×1 bottleneck, an engineer's flawed spec, and ResNet with transfer learning.

On your slides: **Module 6, slides 18–141** · convolution $(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n)K(m, n)$ · output size $\lfloor (n + 2p - k)/s \rfloor + 1$ · parameters $(k \cdot k \cdot C_{in} + 1) C_{out}$ · pooling · receptive field · 1×1 convolution · FC and softmax · VGG, Inception, ResNet · transfer learning

Module 6 gives Nova a camera. A convolutional layer slides a small filter over the image and writes one number per position: a feature map. Every layer has a shape $H \times W \times C$, and the whole exam skill is to carry that shape through a pipeline with the size formula $\lfloor (n + 2p - k)/s \rfloor + 1$, to count parameters with $(k \cdot k \cdot C_{in} + 1) C_{out}$ for a convolution, 0 for pooling and $m(n + 1)$ for a dense layer, and to do a small convolution or pooling by hand. The deck ends with receptive fields, dilation, the 1×1 bottleneck, the famous architectures (VGG, Inception, ResNet) and the transfer-learning grid. Every question below is built like the real paper's Q1(a): a scenario with all numbers given, a numeric core to compute completely, then one or two short pointed parts with one specific expected answer each.

Q	Scenario	Numeric / pointed marks	Slides	Marks
6.1	Satellite crop-health CNN, $96 \times 96 \times 4$: shape after every layer, shape before Flatten, parameters per layer and total; which layer holds most	7 / 1	32, 59, 85–86, 94–97	8
6.2	Convolution by hand: a 5×5 image and a 3×3 diagonal detector; valid, same-padded and stride-2 outputs; what the peaks mean	7 / 1	37–47, 57–62	8
6.3	A 4×4 conv output $\rightarrow$ ReLU $\rightarrow$ max pool and average pool; shapes and parameters of a conv-pool chain; a small shift	7 / 1	74–76, 81–84, 96	8
6.4	Multi-channel convolution: $3 \times 3 \times 2$ input, $2 \times 2 \times 2$ filter, bias 0.5; parameters for 1 and 8 filters; a bigger image	7 / 1	67–73, 94	8
6.5	Keras MNIST CNN snippet: shapes, nature, parameters per layer, iterations per epoch, why softmax + CCE, interpret an output vector	6 / 2	32, 85–91, 94–97	8
6.6	Receptive field: three 3×3 vs one 7×7 ; parameters at 64 channels; dilation $r = 3$; conv-pool-conv chain; the two VGG reasons	7 / 1	50–52, 63–66, 114	8
6.7	Sizes across a pipeline from 224: 7×7 s2 p3, pool, 3×3 s1/s2, dilated 3×3 ; parameters; which ops halve	7 / 1	57–66, 94, 99	8
6.8	1×1 bottleneck at $28 \times 28 \times 256$ vs a direct 5×5 : parameters, reduction factor, multiply-adds; the two uses of 1×1	7 / 1	77–80, 102, 124	8

6.9	An engineer's CNN spec (sigmoid convs, no pooling, ReLU output, MSE): shapes, Flatten size, Dense parameters; the four flaws; the corrected counts	6 / 2	74, 81, 85–91, 99, 149	8
6.10	ResNet arithmetic: basic vs bottleneck block, $y = F(x) + x$ with numbers, the gradient highway over 5 blocks; transfer-learning strategy and trainable count	7 / 1	101–102, 129–134, 140–141	8

Shape before Flatten and total

Q6.1 parameters of a satellite crop-health CNN 8 marks Slides 32, 59, 85–86, 94–97 · Book 6, Ch. 3 and 5

An agri-tech firm designs a compact CNN to run on a small satellite processor for crop-health mapping. The input images are multi-spectral tiles of size $96 \times 96 \times 4$ (four bands). The architecture is: Conv2D(filters = 8, kernel = 5×5 , stride = 1, padding = 0) → MaxPool2D(pool = 2×2 , stride = 2) → Conv2D(filters = 16, kernel = 3×3 , stride = 2, padding = 1) → MaxPool2D(pool = 2×2 , stride = 2) → Flatten → FC(64) → FC(4). Every convolution and FC layer has a bias.

- Using the output-size formula $\lfloor (n + 2p - k) / s \rfloor + 1$, give the exact shape $H \times W \times C$ after every layer. [3]
- State the shape of the feature map immediately before the Flatten layer and the length of the flattened vector. [1]
- Compute the number of parameters (weights and biases) of each layer and the total for the whole network. [3]
- Which layer holds the most parameters, and what percentage of the total? Justify in terms of the FC parameter count $m(n + 1)$ after the Flatten. [1] **justify**

by hand

shapes through a pipeline

count parameters

WHAT THIS TESTS

The real paper's Q1(a) skill: carry $H \times W \times C$ through convolutions and pools with the floor in the size formula, then count $(k \cdot k \cdot C_{in} + 1) C_{out}$ per convolution, 0 per pool and $m(n + 1)$ per FC layer.

HOW TO ATTEMPT

- Do one layer per line: write n, k, p, s , substitute, take the floor, add 1. The channel count is the number of filters; pooling keeps it.
- Flatten multiplies the three numbers of the last shape.
- Parameters: convolutions depend on k, C_{in}, C_{out} only, never on $H \times W$; the first FC layer uses the flattened length as n .

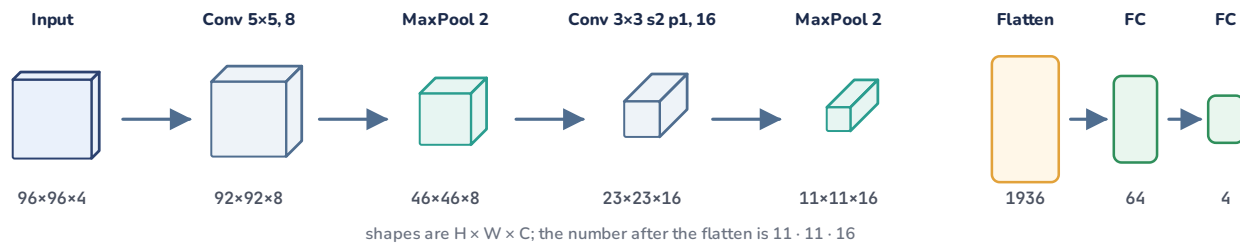


Figure 6.1 The satellite CNN as a chain of volumes. Notice that the two pools halve the height and width (with a floor) and never change the channel count; the flatten turns $11 \times 11 \times 16$ into one vector of 1936 numbers.

Σ SOLUTION

STEP 1 Conv1 ($k = 5, s = 1, p = 0$): $\lfloor (96 + 0 - 5)/1 \rfloor + 1 = 91 + 1 = 92$. Eight filters, so the shape is $92 \times 92 \times 8$.

(a) no padding: the map shrinks by $k - 1 = 4$

STEP 2 Pool1 ($k = 2, s = 2, p = 0$): $\lfloor (92 - 2)/2 \rfloor + 1 = 45 + 1 = 46$, channels unchanged: $46 \times 46 \times 8$.

STEP 3 Conv2 ($k = 3, s = 2, p = 1$): $\lfloor (46 + 2 - 3)/2 \rfloor + 1 = \lfloor 22.5 \rfloor + 1 = 22 + 1 = 23$. Sixteen filters: $23 \times 23 \times 16$.

(a) the floor matters here: $45/2$ is not an integer

STEP 4 Pool2 ($k = 2, s = 2$): $\lfloor (23 - 2)/2 \rfloor + 1 = \lfloor 10.5 \rfloor + 1 = 10 + 1 = 11$: $11 \times 11 \times 16$.

(a)

STEP 5 The map before Flatten is $11 \times 11 \times 16$. Flattened length = $11 \times 11 \times 16 = 1936$.

(b)

STEP 6 Conv1: $(5 \times 5 \times 4 + 1) \times 8 = (100 + 1) \times 8 = 101 \times 8 = 808$

(c)

STEP 7 Pool1: 0. Conv2: $(3 \times 3 \times 8 + 1) \times 16 = (72 + 1) \times 16 = 73 \times 16 = 1,168$. Pool2: 0.

(c)

STEP 8 FC(64): $64 \times (1936 + 1) = 64 \times 1937 = 123,968$

(c) $m(n + 1)$: 1936 weights per unit plus one bias, for 64 units

STEP 9 FC(4): $4 \times (64 + 1) = 4 \times 65 = 260$

(c)

STEP 10 Total = $808 + 0 + 1,168 + 0 + 123,968 + 260 = 126,204$ parameters.

(c)

STEP 11 FC(64) holds $123,968 / 126,204 = 98.2\%$ of all parameters. Key fact (slides 86, 88): an FC layer connects every one of the $n = 1936$ flattened values to every one of its $m = 64$ units, so its count $m(n + 1)$ grows with the whole spatial map, while a convolution's $(k \cdot k \cdot C_{in} + 1)C_{out}$ does not depend on $H \times W$ at all (parameter sharing, slide 94). That is why the deck lists "parameter heavy" as the first disadvantage of FC layers.

(a) $92 \times 92 \times 8 \rightarrow 46 \times 46 \times 8 \rightarrow 23 \times 23 \times 16 \rightarrow 11 \times 11 \times 16$

(b) $11 \times 11 \times 16$; flattened length 1936

(c) 808, 0, 1,168, 0, 123,968, 260; total 126,204

(d) FC(64), 98.2 %; $m(n + 1)$ with $n = 1936$ versus conv counts that ignore $H \times W$

✓ ANSWERS AT A GLANCE

(a) 92, 46, 23, 11 (channels 8, 8, 16, 16) (b) $11 \times 11 \times 16 = 1936$ (c) 126,204 (d) FC(64), 98.2 %

! WHERE MARKS ARE LOST

- Dropping the floor: $45/2 + 1 = 23.5$ or $21/2 + 1 = 11.5$ are not shapes; the answers are 23 and 11.
- Forgetting the +1 bias per filter: $800 + 1,152$ instead of $808 + 1,168$, and $123,904$ instead of $123,968$.
- Using $C_{in} = 3$ for the first convolution: the tiles have four bands, so $5 \times 5 \times 4$.
- Giving pooling layers parameters: they are fixed max operations (slide 96).

Q6.2

Convolution by hand: valid, same-padded and stride-2 outputs of a diagonal detector

8 marks Slides 37–47, 57–62 · Book 6, Ch. 4

Nova's camera gives a 5×5 binary image I , and the first layer uses one 3×3 filter K (no bias):

$$I = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \end{bmatrix}$$

Convolution is the cross-correlation of slide 37: place K on a window of I , multiply element by element, add.

- (a) With stride 1 and no padding, give the size of the output map and compute every value. Show the products of the first two windows, at positions (0,0) and (0,1), in full. [3]
- (b) With "same" padding, state p , the output size, and compute the top-left value and the centre value of the padded output. [2]
- (c) With stride 2 and no padding, give the output size and all its values. [2]
- (d) At which positions does the map in (a) reach its largest value, and what does a large positive value mean, in the words of slide 47? [1] **justify**

by hand

convolution

padding and stride

🎯 WHAT THIS TESTS

The deck's 6×6 sliding example (slides 39–46) with fresh numbers: window, multiply, add, move; then the size formula with $p = 1$ and with $s = 2$; and the meaning of a feature map (high value = the filter's pattern is present).

▶ HOW TO ATTEMPT

- Size first: $\lfloor (5 + 2p - 3)/s \rfloor + 1$. For (a) that is 3, for (b) 5, for (c) 2.
- Only the 1-cells of the window matter: add the kernel entries that sit under a 1.
- For (c) do not recompute: stride 2 picks every second window, so its values are the corners of the map in (a).

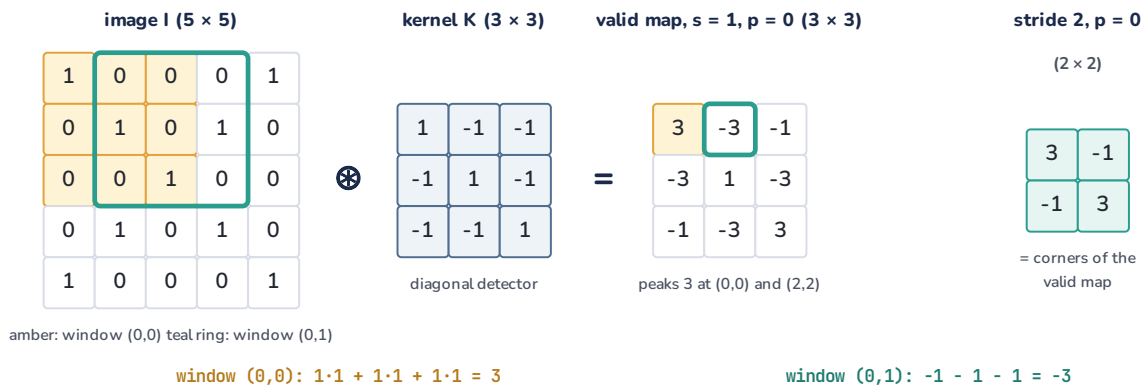


Figure 6.2 The image, the kernel and the valid 3 × 3 map. The amber window at (0,0) sits exactly on the main diagonal and scores 3; the teal window one step right scores -3. Notice that the stride-2 map is just the four corners of the stride-1 map.

SOLUTION

STEP 1 Size: $\lfloor (5 + 0 - 3)/1 \rfloor + 1 = 2 + 1 = 3$, so the output is 3 × 3.

(a)

STEP 2

(a) Window (0,0) is rows 0-2, columns 0-2 of I: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. Products with K: $1 \cdot 1 + 0 + 0 +$

$$0 + 1 \cdot 1 + 0 + 0 + 0 + 1 \cdot 1 = 3.$$

the three 1s sit under the three +1s of K

STEP 3

(a) Window (0,1) is rows 0-2, columns 1-3: $\begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$. Products: the 1s sit under $K(1,0) = -1$, $K(1,2) = -1$, $K(2,1) = -1$: $-1 - 1 - 1 = -3$.

STEP 4

(a) The other seven windows the same way: (0,2): $-1 + 1 - 1 = -1$; (1,0): -3 ; (1,1): $1 - 1 + 1 - 1 + 1 = 1$; (1,2): -3 ; (2,0): -1 ; (2,1): -3 ; (2,2): 3.

STEP 5

(a)

$$I * K = \begin{bmatrix} 3 & -3 & -1 \\ -3 & 1 & -3 \\ -1 & -3 & 3 \end{bmatrix}$$

STEP 6

(b) Same padding for $k = 3$, $s = 1$: $p = (k - 1)/2 = 1$ (slide 57). Size $\lfloor (5 + 2 - 3)/1 \rfloor + 1 = 4 + 1 = 5$: the output is 5 × 5, the same as the input.

STEP 7

(b) Top-left value: the window is the padded corner $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, so $1 \cdot K(1,1) + 1 \cdot K(2,2) =$

$$1 + 1 = 2.$$

the zero border contributes nothing; only two 1s are inside

STEP 8

(b) Centre value: the window is rows 1-3, columns 1-3 of I, the same as window (1,1) of part (a): 1.

STEP 9

(c) Stride 2, no padding: $\lfloor (5 - 3)/2 \rfloor + 1 = 1 + 1 = 2$, so 2 × 2. The windows start at rows/columns 0 and 2, so the values are the corners of the map in (a): $\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$.

STEP 10

(d) The maximum 3 is at (0,0) and (2,2): the two windows that contain a full main diagonal. Key fact (slide 47): a feature map shows where the filter's pattern was detected; "high values = strong pattern presence, low values = weak or absent pattern". Here K is a main-diagonal detector, so the two 3s mark the two diagonal strokes of the X, and the -3s mark windows that contain the anti-diagonal instead.

(a) 3×3 ; $\begin{bmatrix} 3 & -3 & -1 \\ -3 & 1 & -3 \\ -1 & -3 & 3 \end{bmatrix}$; windows (0,0) $\rightarrow$ 3, (0,1) $\rightarrow$ -3

(b) $p = 1$, output 5×5 ; top-left 2, centre 1

(c) 2×2 ; $\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$

(d) (0,0) and (2,2); a high value means the filter's pattern (a main diagonal) is present there

✓ ANSWERS AT A GLANCE

(a) 3, -3, -1 / -3, 1, -3 / -1, -3, 3 (b) $p = 1$, 5×5 ; 2, 1 (c) 3, -1 / -1, 3 (d) (0,0), (2,2): pattern present

! WHERE MARKS ARE LOST

- Flipping the kernel (true convolution) instead of cross-correlation: the deck and every framework use the kernel as written (slide 37).
- Moving the window by 1 in (c) and then dropping rows: stride 2 means the window starts at 0, 2, so there are only two positions per axis.
- Forgetting that the padded border is zeros: the top-left same-padded value is 2, not 3.
- Answering (d) with "the filter fired" and no position: the mark is for (0,0) and (2,2) plus the slide-47 meaning.

Conv $\rightarrow$ ReLU $\rightarrow$ max pool and average

Q6.3 pool, then the shapes and parameters of the chain

8 marks Slides 74–76, 81–84, 96 · Book 6, Ch. 5

One filter of a convolutional layer produced this 4×4 output map (before the activation):

$$Z = \begin{bmatrix} 2 & -1 & 0 & 3 \\ -4 & 1 & 2 & -2 \\ 0 & -3 & 4 & 2 \\ 5 & 3 & -2 & 0 \end{bmatrix}$$

The layer is followed by ReLU and then a pooling layer with a 2×2 window and stride 2.

- Write the map after ReLU. [1]
- Compute the output of max pooling ($k = 2, s = 2$) on the ReLU map, showing each window. [2]
- Compute the output of average pooling ($k = 2, s = 2$) on the ReLU map. [2]
- The full layer takes a $32 \times 32 \times 3$ image through 10 filters of size 3×3 (stride 1, padding 1) and ReLU, then this pooling. Give the output shape after the convolution and after the pool, and the number of trainable parameters of the convolution and of the pool. [2]
- The values 5 and 3 in the bottom row of Z swap places. Recompute the bottom-left max-pool and average-pool outputs. Which property from slide 74 does the result show? [1] **justify**

by hand

ReLU and pooling

shapes

WHAT THIS TESTS

The deck's Conv + ReLU + MaxPool chain (slide 84) and the two pooling examples (slides 75–76) with fresh numbers, then the two facts that pooling keeps the channel count and has zero parameters (slides 74, 96), and the reason pooling gives translation invariance to small shifts.

HOW TO ATTEMPT

- ReLU: replace every negative by 0, keep the rest.
- Cut the 4×4 into four non-overlapping 2×2 windows; take the max, then the mean, of each.
- For (d) use the size formula with $p = 1$ (same size) and then $\lfloor 32/2 \rfloor = 16$; a pool has no weights.

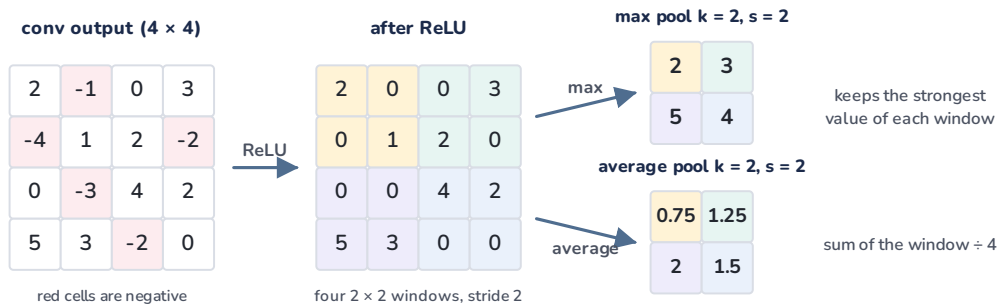


Figure 6.3 ReLU zeroes the red cells; then each coloured 2×2 window becomes one number. Notice the bottom-left window: its max is 5 whether the 5 sits at the left or the right.

SOLUTION

STEP 1 $\text{ReLU}(Z) = \max(0, Z)$:

(a)

$$A = \begin{bmatrix} 2 & 0 & 0 & 3 \\ 0 & 1 & 2 & 0 \\ 0 & 0 & 4 & 2 \\ 5 & 3 & 0 & 0 \end{bmatrix}$$

six negatives become 0; positives are unchanged

STEP 2 Windows of A : top-left $\{2, 0, 0, 1\}$, top-right $\{0, 3, 2, 0\}$, bottom-left $\{0, 0, 5, 3\}$, bottom-right $\{4, 2, 0, 0\}$.

(b)

STEP 3 Max of each: 2, 3, 5, 4.

(b)

$$\text{MaxPool}(A) = \begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix}$$

STEP 4 Sums: $2 + 0 + 0 + 1 = 3$; $0 + 3 + 2 + 0 = 5$; $0 + 0 + 5 + 3 = 8$; $4 + 2 + 0 + 0 = 6$. Divide each by $k^2 = 4$: 0.75, 1.25, 2, 1.5.

(c)

$$\text{AvgPool}(A) = \begin{bmatrix} 0.75 & 1.25 \\ 2 & 1.5 \end{bmatrix}$$

STEP 5 Convolution: $\lfloor (32 + 2 - 3)/1 \rfloor + 1 = 31 + 1 = 32$; 10 filters, so $32 \times 32 \times 10$. Pool: $\lfloor (32 - 2)/2 \rfloor + 1 = 15 + 1 = 16$, channels kept: $16 \times 16 \times 10$.

(d)

STEP 6 Convolution parameters: $(3 \times 3 \times 3 + 1) \times 10 = 28 \times 10 = 280$. Pool parameters: 0 (a fixed max or mean, slide 96).

(d)

the pool cuts the values from $32 \cdot 32 \cdot 10 = 10,240$ to 2,560, a quarter, and learns nothing

STEP 7 Bottom-left window after the swap: $\{0, 0, 3, 5\}$. Max = 5 (unchanged), average = $8/4 = 2$ (unchanged). Key fact (slide 74): pooling gives "translation invariance: make features less sensitive to small shifts". A feature that moves by one cell inside its window gives the same pooled output.

(e)

(a) 4 as above (six zeros)

(b) $\begin{bmatrix} 2 & 3 \\ 5 & 4 \end{bmatrix}$

(c) $\begin{bmatrix} 0.75 & 1.25 \\ 2 & 1.5 \end{bmatrix}$

(d) $32 \times 32 \times 10$ then $16 \times 16 \times 10$; 280 and 0 parameters

(e) still 5 and 2; translation invariance to small shifts

✓ ANSWERS AT A GLANCE

(a) negatives $\rightarrow$ 0 (b) 2, 3 / 5, 4 (c) 0.75, 1.25 / 2, 1.5 (d) $32^2 \times 10, 16^2 \times 10$; 280, 0 (e) 5, 2; small-shift invariance

! WHERE MARKS ARE LOST

- Pooling the raw map Z instead of the ReLU map: the average pool would give $-0.5, 0.75, 1.25, 1$.
- Sliding the pool window with stride 1 (a 3×3 output): $k = 2, s = 2$ gives non-overlapping windows and a 2×2 output.
- Giving the pool a channel count of 1 or parameters like $2 \times 2 = 4$: channels stay 10 and the count is 0.

Q6.4

Multi-channel convolution by hand, and parameters for one filter and for eight

8 marks Slides 67–73, 94 · Book 6, Ch. 4

A two-channel input of size $3 \times 3 \times 2$ is convolved with one filter of size $2 \times 2 \times 2$ with bias $b = 0.5$, stride 1, no padding:

$$I_1 = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{bmatrix}, \quad I_2 = \begin{bmatrix} 0 & 1 & 1 \\ 2 & 0 & 1 \\ 1 & 3 & 2 \end{bmatrix}, \quad K_1 = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}, \quad K_2 = \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix}$$

I_c is input channel c and K_c is the slice of the filter that sits on channel c .

- (a) Give the output shape. Compute the output at position (0,0), showing the channel-1 sum, the channel-2 sum, their total and the bias. [2]
- (b) Compute the remaining three output values the same way. [3]
- (c) How many trainable parameters does this one filter have? [1]
- (d) The layer uses 8 such filters. Give its output shape and its total parameter count. [1]
- (e) The same 8-filter layer is now applied to a $64 \times 64 \times 2$ input. Give the output shape and the parameter count, and justify the count in terms of parameter sharing (slide 94). [1] **justify**

by hand

channels

parameter sharing

🎯 WHAT THIS TESTS

Slide 69's two steps — convolve each channel with its own slice of the filter, then add across channels and add one bias — and the rule $(k \cdot k \cdot C_{in} + 1) C_{out}$ with the fact that it ignores the image size.

▶ HOW TO ATTEMPT

- Output size $\lfloor (3 - 2)/1 \rfloor + 1 = 2$, one channel per filter.

- At each position take the 2×2 window of I_1 with K_1 and of I_2 with K_2 ; add the two sums; add 0.5 once (not once per channel).
- Parameters per filter = weights across all channels + 1 bias.

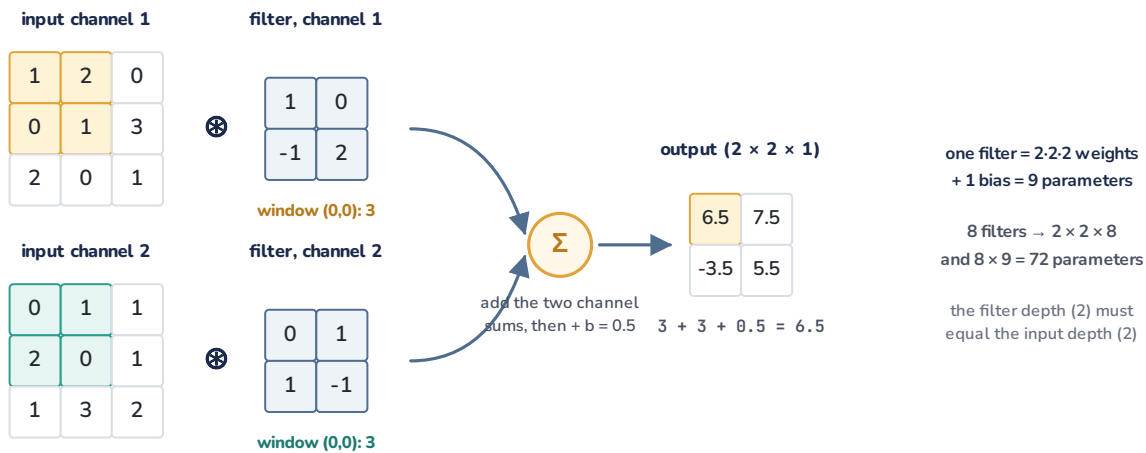


Figure 6.4 One filter is as deep as the input. Each channel gets its own 2×2 slice; the two channel sums are added, then the single bias. Notice the output has one channel however many input channels there are.

Σ SOLUTION

STEP 1 Shape: $\lfloor (3 + 0 - 2)/1 \rfloor + 1 = 2$ each way, one filter, so $2 \times 2 \times 1$.

(a)

STEP 2 Position (0,0), channel 1: window $\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ with $K_1: 1(1) + 2(0) + 0(-1) + 1(2) = 1 + 0 + 0 + 2 = 3$

(a)

STEP 3 Channel 2: window $\begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}$ with $K_2: 0(0) + 1(1) + 2(1) + 0(-1) = 0 + 1 + 2 + 0 = 3$

(a)

STEP 4 $O(0,0) = 3 + 3 + 0.5 = 6.5$

(a)

sum over channels, then one bias

STEP 5 (0,1): ch 1 $\begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}: 2 + 0 - 1 + 6 = 7$; ch 2 $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}: 0 + 1 + 0 - 1 = 0$; $O = 7 + 0 + 0.5 = 7.5$

(b)

STEP 6 (1,0): ch 1 $\begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}: 0 + 0 - 2 + 0 = -2$; ch 2 $\begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}: 0 + 0 + 1 - 3 = -2$; $O = -2 - 2 + 0.5 = -3.5$

(b)

STEP 7 (1,1): ch 1 $\begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}: 1 + 0 + 0 + 2 = 3$; ch 2 $\begin{bmatrix} 0 & 1 \\ 3 & 2 \end{bmatrix}: 0 + 1 + 3 - 2 = 2$; $O = 3 + 2 + 0.5 = 5.5$

(b)

STEP 8 $O = \begin{bmatrix} 6.5 & 7.5 \\ -3.5 & 5.5 \end{bmatrix}$

(b)

STEP 9 One filter: $k \times k \times C_{in} + 1 = 2 \times 2 \times 2 + 1 = 8 + 1 = 9$ parameters.

(c)

STEP 10 Eight filters: output $2 \times 2 \times 8$ (one channel per filter); parameters $(2 \times 2 \times 2 + 1) \times 8 = 9 \times 8 = 72$.

(d)

STEP 11 Output $\lfloor (64 - 2)/1 \rfloor + 1 = 63$, so $63 \times 63 \times 8$. Parameters: still 72. Key fact (slides 28, 94): the same filter is used at every position (parameter sharing), so the count $(k \cdot k \cdot C_{in} + 1)C_{out}$ depends on k, C_{in}, C_{out} only and "is independent of input spatial dimensions". A dense layer from $64 \cdot 64 \cdot 2 = 8192$ inputs to $63 \cdot 63 \cdot 8 = 31,752$ outputs would need $31,752 \times 8193 = 260,144,136$ parameters.

(e)

(a) $2 \times 2 \times 1; O(0,0) = 3 + 3 + 0.5 = 6.5$

$$(b) O = \begin{bmatrix} 6.5 & 7.5 \\ -3.5 & 5.5 \end{bmatrix}$$

(c) 9

(d) $2 \times 2 \times 8; 72$

(e) $63 \times 63 \times 8$; still 72 — parameter sharing, the count ignores $H \times W$

✓ ANSWERS AT A GLANCE

(a) 6.5 (b) 7.5, -3.5, 5.5 (c) 9 (d) $2 \times 2 \times 8, 72$ (e) $63 \times 63 \times 8, 72$

! WHERE MARKS ARE LOST

- Adding the bias once per channel (7.0 instead of 6.5): one filter has one bias.
- Giving a two-channel output: the channel sums are added, so one filter gives one map.
- Counting parameters as $2 \times 2 + 1 = 5$ per filter: the filter has depth 2, so 8 weights.
- Scaling the count with the image in (e): 72 stays 72.

A Keras CNN snippet: shapes,

Q6.5 parameters, iterations and the output vector **8 marks** Slides 32, 85–91, 94–97 · Book 6, Ch. 3 and 5

Consider the following Python code snippet, written to recognise handwritten digits from 28×28 grayscale images. The training set has 60,000 images.

```
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=64)
```

Keras uses padding "valid" (no padding) and stride 1 for these convolutions, and stride 2 for the pools.

- Draw the architecture and give the output shape of every layer, including the flattened length. [2]
- State the nature of the problem (binary / multi-class / regression), giving the two lines that show it. Why is softmax with categorical cross-entropy used rather than sigmoid with binary cross-entropy? Under what condition would you switch to sigmoid? [1] **justify**
- Calculate the number of trainable parameters (weights + biases) of each layer and the total. [3]
- How many weight updates happen in one epoch, and in the whole run? [1]
- For one test image the model outputs [0.02, 0.01, 0.62, 0.05, 0.03, 0.10, 0.04, 0.06, 0.03, 0.04]. Give the predicted class, its confidence, the runner-up class, and check that the vector is a valid softmax output. [1]

WHAT THIS TESTS

Reading a CNN definition the way the mid-sem reads a Dense one: the last layer and the loss name the task; every layer has a shape; convolution parameters are $(k \cdot k \cdot C_{in} + 1)C_{out}$; pools and Flatten have none; an epoch is $\lceil N/B \rceil$ updates; a softmax vector is read by its argmax.

HOW TO ATTEMPT

- Shapes: valid 3×3 takes 2 off each side; a 2×2 pool halves with a floor ($11 \rightarrow 5$).
- Parameters: first conv has $C_{in} = 1$; the first Dense uses the flattened length $5 \cdot 5 \cdot 64$.
- Updates per epoch: $60000/64$ rounded up; the last batch is smaller.

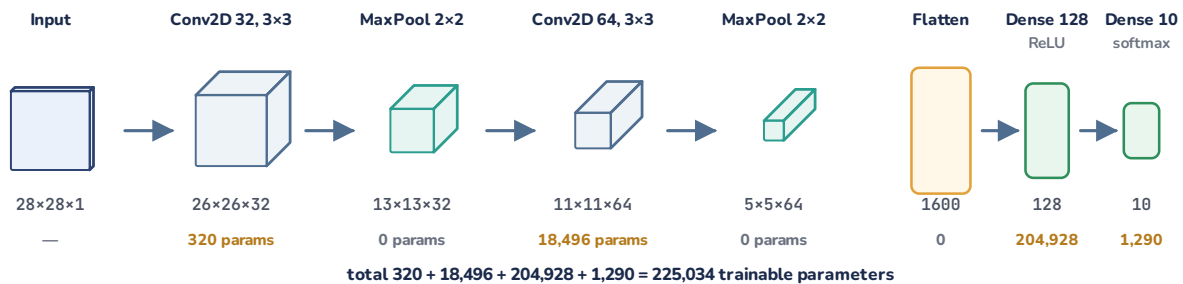


Figure 6.5 The snippet as a chain of shapes with the parameter count under each layer. Notice that the first Dense layer holds 91 % of all parameters, and the two pools and the Flatten hold none.

SOLUTION

STEP 1 Conv2D($32, 3 \times 3$): $\lfloor (28 - 3)/1 \rfloor + 1 = 26$, so $26 \times 26 \times 32$. MaxPool: $\lfloor (26 - 2)/2 \rfloor + 1 = 13$, so $13 \times 13 \times 32$.

STEP 2 Conv2D($64, 3 \times 3$): $13 - 3 + 1 = 11$, so $11 \times 11 \times 64$. MaxPool: $\lfloor (11 - 2)/2 \rfloor + 1 = \lfloor 4.5 \rfloor + 1 = 5$, so $5 \times 5 \times 64$.

the odd size 11 loses its last row and column

STEP 3 Flatten: $5 \times 5 \times 64 = 1600$. Dense(128): 128. Dense(10): 10 probabilities. (Figure 6.5.)

STEP 4 Multi-class classification with $K = 10$ classes: Dense(10, activation='softmax') and loss='categorical_crossentropy'. Key fact (slides 89–90): softmax turns the 10 logits into a probability distribution that sums to 1, which is right when each image belongs to exactly one of 10 mutually exclusive classes; cross-entropy then compares that distribution with the one-hot label. Sigmoid + BCE is the special case $K = 2$ with one output unit, or the choice when classes are not exclusive (multi-label), where every unit is a separate yes/no.

STEP 5 Conv2D(32): $(3 \times 3 \times 1 + 1) \times 32 = 10 \times 32 = 320$

STEP 6 MaxPool: 0. Conv2D(64): $(3 \times 3 \times 32 + 1) \times 64 = (288 + 1) \times 64 = 289 \times 64 = 18,496$

STEP 7 MaxPool: 0. Flatten: 0.

STEP 8 Dense(128): $128 \times (1600 + 1) = 128 \times 1601 = 204,928$

STEP 9 Dense(10): $10 \times (128 + 1) = 10 \times 129 = 1,290$

STEP 10 Total = $320 + 0 + 18,496 + 0 + 0 + 204,928 + 1,290 = 225,034$

STEP 11 Per epoch $60000/64 = 937.5$, so 938 mini-batches (937 of size 64 and one of size 32): 938 updates. Whole run $938 \times 10 = 9,380$ updates.

(d)

STEP 12 Argmax is index 2 (value 0.62): the predicted digit is 2 with confidence 62 %. Runner-up: index 5 (0.10). Check: the ten values are all in $[0, 1]$ and add to $0.02 + 0.01 + 0.62 + 0.05 + 0.03 + 0.10 + 0.04 + 0.06 + 0.03 + 0.04 = 1.00$, so it is a valid softmax output.

(e)

(a) $26 \times 26 \times 32 \rightarrow 13 \times 13 \times 32 \rightarrow 11 \times 11 \times 64 \rightarrow 5 \times 5 \times 64 \rightarrow 1600 \rightarrow 128 \rightarrow 10$

(b) multi-class (10); softmax gives one distribution over exclusive classes; switch to sigmoid for $K = 2$ or multi-label

(c) 320, 0, 18,496, 0, 0, 204,928, 1,290; total 225,034

(d) 938 per epoch; 9,380 in total

(e) class 2, 0.62; runner-up class 5 (0.10); sums to 1.00

✓ ANSWERS AT A GLANCE

(a) 26, 13, 11, 5; 1600 (b) multi-class (c) 225,034 (d) 938; 9,380 (e) 2 (0.62)

! WHERE MARKS ARE LOST

- Pooling 11 to 5.5 or 6: the floor gives 5, so the flattened length is 1600, not 2304.
- Using $C_{in} = 3$ for the first convolution (the input is grayscale, one channel) or forgetting the bias per filter (288 instead of 289).
- Answering (d) with "10 updates": an epoch has 938 mini-batch updates.
- Reading 0.62 as "62 % accuracy": it is the model's probability for class 2 on this one image.

Q6.6

Receptive field: three 3×3 against one 7×7 , dilation, a conv-pool-conv chain

8 marks Slides 50–52, 63–66, 114 · Book 6, Ch. 4

Asha compares two designs for a block that keeps 64 channels: design A stacks three 3×3 convolutions (stride 1); design B uses one 7×7 convolution (stride 1). Use the deck's formula $RF_l = RF_{l-1} + (K_l - 1) \prod_{i=1}^{l-1} S_i$ with $RF_0 = 1$.

- Compute the receptive field after each layer of design A, and of design B. [2]
- With $C_{in} = C_{out} = 64$ and biases excluded, compute the parameters of design A and of design B, and the ratio B/A. [2]
- Design C is one 3×3 convolution with dilation rate $r = 3$. Compute its effective kernel size $k_{\text{eff}} = k + (k - 1)(r - 1)$, its receptive field, and its number of weights per input channel per filter, compared with design B's. [1]
- A different chain is 3×3 conv ($s = 1$) $\rightarrow 2 \times 2$ max pool ($s = 2$) $\rightarrow 3 \times 3$ conv ($s = 1$). Compute the receptive field after each of the three layers. [2]
- Give the two reasons on slide 114 (VGG) why design A is preferred to design B. [1] **justify**

by hand

receptive field

dilation

🎯 WHAT THIS TESTS

Slide 52's growth formula (with the product of earlier strides when a pool is in the chain), the parameter formula at a fixed channel width, the dilation formula of slide 63, and VGG's two arguments for small filters.

▶ HOW TO ATTEMPT

- Stride 1 everywhere: each 3×3 layer adds $k - 1 = 2$ to the receptive field.
- Parameters without bias: $k^2 C_{in} C_{out}$ per layer.
- In (d) the pool counts as a layer with $K = 2$, $S = 2$; after it the product of strides is 2, so the next 3×3 adds $2 \times 2 = 4$.

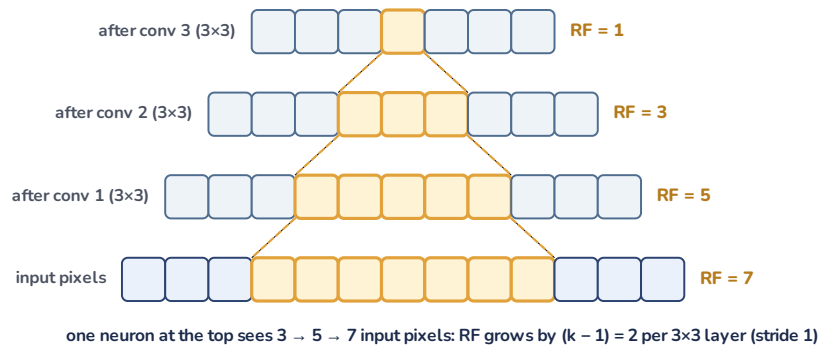


Figure 6.6 One neuron after three 3×3 layers, seen in one dimension. Notice how the amber region widens by 2 at each layer: 3, 5, 7 input pixels, the same 7 that a single 7×7 filter sees at once.

Σ SOLUTION

STEP 1 Design A, all strides 1 so the product is always 1: $RF_1 = 1 + (3 - 1) \times 1 = 3$; $RF_2 = 3 + 2 = 5$; $RF_3 = 5 + 2 = 7$.

STEP 2 Design B: $RF_1 = 1 + (7 - 1) \times 1 = 7$. Both designs see a 7×7 patch of the input.

STEP 3 Design A: one layer has $3 \times 3 \times 64 \times 64 = 36,864$ weights; three layers = $3 \times 36,864 = 110,592$.

STEP 4 Design B: $7 \times 7 \times 64 \times 64 = 49 \times 4096 = 200,704$. Ratio $200,704 / 110,592 = 1.815$: B needs about 1.8 times the weights for the same receptive field.
with biases: 110,784 vs 200,768, same ratio

STEP 5 Design C: $k_{\text{eff}} = 3 + (3 - 1)(3 - 1) = 3 + 4 = 7$, so its receptive field is 7, the same as B. Weights per input channel per filter: $3 \times 3 = 9$ against B's $7 \times 7 = 49$ (slide 65: "same parameter count as standard convolution"). The price is that C skips 40 of the 49 positions.

STEP 6 Layer 1 (conv, $K = 3$, $S = 1$): $RF_1 = 1 + 2 \times 1 = 3$.

STEP 7 Layer 2 (pool, $K = 2$, $S = 2$): $RF_2 = 3 + (2 - 1) \times 1 = 4$. The product of strides is now $1 \times 2 = 2$.

STEP 8 Layer 3 (conv, $K = 3$): $RF_3 = 4 + (3 - 1) \times 2 = 4 + 4 = 8$.
after a stride-2 layer every later kernel step covers 2 input pixels

STEP 9 Key facts (slide 114): (1) fewer parameters — "two 3×3 conv $\approx$ one 5×5 conv (fewer parameters)", here 110,592 against 200,704; (2) more non-linearities — three ReLUs instead of one, so the block can represent more complex functions. Slide 52 adds the same insight: stacking small kernels is more efficient than one large kernel with the same receptive field.

(a) A: 3, 5, 7; B: 7

(b) A: 110,592; B: 200,704; ratio 1.815

(c) $k_{\text{eff}} = 7$, RF 7, 9 weights against 49

(d) 3, 4, 8

(e) fewer parameters; more non-linearities (ReLU after each layer)

✓ ANSWERS AT A GLANCE

(a) 3, 5, 7 / 7 (b) 110,592 vs 200,704; $\times 1.815$ (c) 7; 9 vs 49 (d) 3, 4, 8 (e) params, non-linearity

! WHERE MARKS ARE LOST

- Multiplying receptive fields ($3 \times 3 \times 3 = 27$) instead of adding $k - 1$ per stride-1 layer.
- Forgetting the stride product in (d): $4 + 2 = 6$ is wrong; the pool doubles every later step, so 8.
- Counting the ratio as $49/9 = 5.4$: that is per layer; A has three layers, so 1.815.
- Giving one reason in (e); the deck gives two.

Q6.7

Padding, stride and dilation across a five-layer pipeline from 224×224

8 marks Slides 57–66, 94, 99 · Book 6, Ch. 4

The front of an image classifier processes a $224 \times 224 \times 3$ photo through five layers (biases everywhere):

Layer	Operation	k	s	p	r	Filters
1	convolution	7	2	3	1	64
2	max pool	3	2	1	—	—
3	convolution	3	1	1	1	64
4	convolution	3	2	1	1	128
5	dilated convolution	3	1	2	2	128

- (a) Compute the spatial size after each of the five layers with $\lfloor (n + 2p - k_{\text{eff}})/s \rfloor + 1$, where $k_{\text{eff}} = k + (k - 1)(r - 1)$. Write the full shape $H \times W \times C$ each time. [4]
- (b) Compute the parameters of layer 1 and of layer 4. [2]
- (c) Compute the number of values in the input volume and in the output of layer 5, and their ratio. [1]
- (d) Using the formula, state which layers halve the size and which keep it, and the condition on p , k_{eff} and s in each case. [1] **justify**

by hand

output-size formula

dilation

🎯 WHAT THIS TESTS

The one formula of slides 59–63 used five times with different k , s , p , r , including a pool and a dilated kernel; the parameter formula with the right C_{in} ; and the reading of the formula that says when a layer halves and when it keeps the size.

▶ HOW TO ATTEMPT

- For each layer write n , $2p$, k_{eff} , s ; compute the fraction; floor; add 1. Carry the result to the next line.
- Only layer 5 has $r \neq 1$: $k_{\text{eff}} = 3 + 2 \times 1 = 5$.
- Channels: a convolution's output channels = its filter count; the pool keeps 64.

Σ SOLUTION

STEP 1 Layer 1: $\lfloor (224 + 6 - 7)/2 \rfloor + 1 = \lfloor 223/2 \rfloor + 1 = 111 + 1 = 112$: $112 \times 112 \times 64$.

(a)

STEP 2 Layer 2: $\lfloor (112 + 2 - 3)/2 \rfloor + 1 = \lfloor 111/2 \rfloor + 1 = 55 + 1 = 56$: $56 \times 56 \times 64$.

(a)

a pool keeps the channel count

STEP 3 Layer 3: $\lfloor (56 + 2 - 3)/1 \rfloor + 1 = 55 + 1 = 56$: $56 \times 56 \times 64$.

(a)

STEP 4 Layer 4: $\lfloor (56 + 2 - 3)/2 \rfloor + 1 = \lfloor 55/2 \rfloor + 1 = 27 + 1 = 28$: $28 \times 28 \times 128$.

(a)

STEP 5 Layer 5: $k_{\text{eff}} = 3 + (3 - 1)(2 - 1) = 5$; $\lfloor (28 + 4 - 5)/1 \rfloor + 1 = 27 + 1 = 28$: $28 \times 28 \times 128$.

(a)

$p = 2$ is exactly $(k_{\text{eff}} - 1)/2$, so the dilated layer keeps the size (slide 66)

STEP 6 Layer 1: $(7 \times 7 \times 3 + 1) \times 64 = (147 + 1) \times 64 = 148 \times 64 = 9,472$

(b)

STEP 7 Layer 4: $(3 \times 3 \times 64 + 1) \times 128 = (576 + 1) \times 128 = 577 \times 128 = 73,856$

(b)

STEP 8 Input: $224 \times 224 \times 3 = 150,528$ values. Output of layer 5: $28 \times 28 \times 128 = 100,352$ values.

(c)

Ratio $150,528/100,352 = 1.5$: the map is 64 times smaller in area but has about 43 times more channels (Pattern 1, slide 99: reduce space, expand channels).

STEP 9 Key fact: the size is $\lfloor (n + 2p - k_{\text{eff}})/s \rfloor + 1$. When $2p = k_{\text{eff}} - 1$ and $s = 1$ the fraction is exactly $n - 1$ and the size stays n : layers 3 ($2p = 2 = 3 - 1$) and 5 ($2p = 4 = 5 - 1$). When $2p = k_{\text{eff}} - 1$ and $s = 2$ the fraction is $(n - 1)/2$, so the size becomes $\lfloor (n - 1)/2 \rfloor + 1 = n/2$ for even n : layers 1, 2 and 4 halve ($224 \rightarrow 112 \rightarrow 56$ and $56 \rightarrow 28$).

(d)

(a) 112, 56, 56, 28, 28; shapes $112^2 \times 64, 56^2 \times 64, 56^2 \times 64, 28^2 \times 128, 28^2 \times 128$

(b) 9,472 and 73,856

(c) 150,528 and 100,352; ratio 1.5

(d) halve: layers 1, 2, 4 ($s = 2, 2p = k_{\text{eff}} - 1$); keep: layers 3, 5 ($s = 1, 2p = k_{\text{eff}} - 1$)

✓ ANSWERS AT A GLANCE

(a) 112, 56, 56, 28, 28 (b) 9,472; 73,856 (c) $150,528 : 100,352 = 1.5$ (d) 1, 2, 4 halve; 3, 5 keep

! WHERE MARKS ARE LOST

- Using $k = 3$ instead of $k_{\text{eff}} = 5$ for the dilated layer: it would print 30 instead of 28.
- Forgetting to double p : $224 + 3 - 7$ gives 111, not 112.
- Counting layer 4 with $C_{\text{in}} = 128$ (147,584): the input to layer 4 has 64 channels.
- Giving the pool parameters or changing its channel count.

Q6.8

The 1×1 bottleneck: parameters and multiply-adds against a direct 5×5

8 marks Slides 77–80, 102, 124 · Book 6, Ch. 5 and 7

Inside an Inception-style module the feature map is $28 \times 28 \times 256$. Two ways to produce a $28 \times 28 \times 256$ output with 5×5 spatial context are compared (padding keeps the size; biases are excluded):

- **Direct:** one 5×5 convolution, $256 \rightarrow 256$.
 - **Bottleneck:** 1×1 convolution $256 \rightarrow 64$, then 5×5 convolution $64 \rightarrow 64$, then 1×1 convolution $64 \rightarrow 256$.
- Compute the parameters of each of the three bottleneck layers and their total. [3]
 - Compute the parameters of the direct 5×5 layer and the reduction factor direct/bottleneck. [2]
 - Each output position costs one multiply-add per weight. Compute the multiply-adds of both designs over the 28×28 map. [2]
 - Name the two uses of a 1×1 convolution on slide 79 that the bottleneck relies on, and say what the first 1×1 layer does to the channel count. [1] **justify**

count parameters

1x1 convolution

ops count

WHAT THIS TESTS

The bottleneck arithmetic of slides 80 and 102 with clean numbers: $k^2 C_{in} C_{out}$ three times against once, and the fact that the compute at a fixed map size is the parameter count times the number of positions (slide 124).

HOW TO ATTEMPT

- A 1×1 layer has $C_{in} \times C_{out}$ weights; a 5×5 layer has $25 C_{in} C_{out}$.
- Positions: $28 \times 28 = 784$. Multiply each parameter count by 784.
- The ratio is the same for parameters and for multiply-adds, because both designs run on the same 28×28 map.

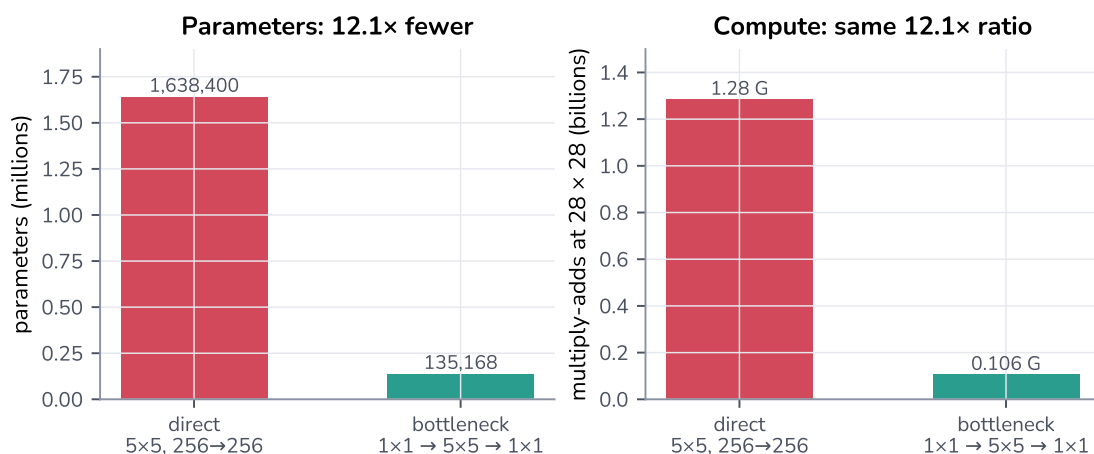


Figure 6.8 Parameters and multiply-adds for the two designs. Notice both bars shrink by the same factor 12.1: at a fixed map size, compute is just parameters $\times$ positions.

SOLUTION

STEP 1 $1 \times 1, 256 \rightarrow 64: 1 \times 1 \times 256 \times 64 = 16,384$

(a)

STEP 2 $5 \times 5, 64 \rightarrow 64: 5 \times 5 \times 64 \times 64 = 25 \times 4096 = 102,400$

(a)

STEP 3 $1 \times 1, 64 \rightarrow 256: 1 \times 1 \times 64 \times 256 = 16,384$

(a)

STEP 4 Bottleneck total = $16,384 + 102,400 + 16,384 = 135,168$

(a)

STEP 5 Direct: $5 \times 5 \times 256 \times 256 = 25 \times 65,536 = 1,638,400$

(b)

STEP 6 Reduction factor = $1,638,400 / 135,168 = 12.12$, so about 12 times fewer parameters.

(b)

slide 80 quotes about 11× for this bottleneck; the exact arithmetic here gives 12.1×

STEP 7 Positions: $28 \times 28 = 784$. Direct: $1,638,400 \times 784 = 1,284,505,600 \approx 1.28 \times 10^9$ multiply-adds.

(c)

STEP 8 Bottleneck: $135,168 \times 784 = 105,971,712 \approx 1.06 \times 10^8$. Same ratio 12.12.

(c)

STEP 9 Key facts (slide 79): (1) dimensionality reduction — a 1×1 convolution is a weighted sum across channels at each position, so the first layer takes 256 channels to 64 while keeping 28×28 unchanged, and the expensive 5×5 then runs on 64 channels instead of 256; (2) adding non-linearity — each 1×1 is followed by ReLU, so the block has three non-linearities instead of one. Slide 79 lists the third use, computational efficiency, which is (c).

(d)

(a) $16,384 + 102,400 + 16,384 = 135,168$

(b) 1,638,400; factor 12.12

(c) 1.28×10^9 vs 1.06×10^8 multiply-adds

(d) channel reduction ($256 \rightarrow 64$ at the same 28×28) and added non-linearity (ReLU after each 1×1)

✓ ANSWERS AT A GLANCE

(a) 135,168 (b) 1,638,400; ×12.12 (c) 1.28 G vs 106 M (d) reduce channels; non-linearity

! WHERE MARKS ARE LOST

- Treating a 1×1 layer as free: it has $C_{in} \times C_{out}$ weights, here 16,384 each.
- Computing the 5×5 middle layer with 256 channels: the bottleneck's whole point is that it runs at 64.
- Multiplying by 28 instead of $28 \times 28 = 784$ positions.
- Saying the 1×1 "reduces the image": it changes the channel count only; 28×28 is unchanged.

An engineer's CNN spec: shapes,

Q6.9

parameters, four flaws, corrected counts

8 marks Slides 74, 81, 85–91, 99, 149 · Book 6, Ch. 5 and 6

An engineer proposes the following CNN for a 10-class image classification task with $64 \times 64 \times 3$ inputs and one-hot labels:

Layer	Specification
Conv 1	16 filters, 3×3 , stride 1, no padding, sigmoid
Conv 2	32 filters, 3×3 , stride 1, no padding, sigmoid
Pooling	none
Flatten → Dense	10 units, ReLU
Loss	mean squared error

- (a) Compute the output shape of each layer and the size of the flattened vector. [2]
- (b) Compute the parameters of Conv 1, Conv 2 and the Dense layer, and the total. What percentage sits in the Dense layer? [2]
- (c) Examine the specification and identify its four flaws. For each, give the deck's fix and the fact that justifies it (the sigmoid derivative bound, the pooling slide, the softmax slide, the loss for one-hot labels). [2] **justify**
- (d) Correct the spec by using ReLU after each convolution, inserting a 2×2 max pool (stride 2) after each convolution, using padding 1 on Conv 2, and using softmax with categorical cross-entropy. Compute the new shapes, the flattened size, the Dense parameters and the new total. By what factor did the Dense parameters fall? [2]

count parameters

find the flaws

justify

🔗 WHAT THIS TESTS

MS-6(b) for a CNN: read a spec, compute what it implies (a 115,200-long flatten and a million-parameter Dense layer), and name each flaw with the deck's specific reason: $\sigma'(z) \leq 0.25$, pooling and Pattern 1, softmax for class probabilities, cross-entropy for one-hot targets.

▶ HOW TO ATTEMPT

- Valid 3×3 convolutions take 2 off each side: $64 \rightarrow 62 \rightarrow 60$.
- The Dense layer's count is $10 \times (\text{flatten} + 1)$; it dominates everything.
- For (d) redo the shapes with the pools (floor at $31 \rightarrow 15$), then the same Dense formula.

Σ SOLUTION

STEP 1 Conv 1: $\lfloor (64 - 3)/1 \rfloor + 1 = 62$; $62 \times 62 \times 16$. Conv 2: $62 - 3 + 1 = 60$; $60 \times 60 \times 32$.

(a)

STEP 2 Flatten: $60 \times 60 \times 32 = 115,200$. Dense: 10 outputs.

(a)

STEP 3 Conv 1: $(3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = 448$. Conv 2: $(3 \times 3 \times 16 + 1) \times 32 = 145 \times 32 = 4,640$.

(b)

STEP 4 Dense: $10 \times (115,200 + 1) = 1,152,010$. Total = $448 + 4,640 + 1,152,010 = 1,157,098$; the Dense layer holds 99.6%.

(b)

the two convolutions together are under 0.5 % of the network

STEP 5 Flaw 1, sigmoid in the hidden convolutions: $\sigma'(z) = \sigma(1 - \sigma) \leq 0.25$, so two sigmoid layers scale gradients by at most $0.25^2 = 0.0625$ and saturate for large $|z|$ (vanishing gradient, slide 149). Fix: ReLU, "the default choice for activation functions in CNNs, applied after each convolutional layer" (slide 81).

(c)

STEP 6 Flaw 2, no downsampling: the flatten is 115,200 long and the Dense layer alone has 1.15 M parameters, an overfitting risk (slide 88). Fix: a pooling layer after each convolution —

(c)

"spatial downsampling, translation invariance, computational efficiency, overfitting reduction" (slide 74), Pattern 1 "reduce spatial dimensions, increase channels" (slide 99).

STEP 7 Flaw 3, ReLU on the output of a classifier: ReLU outputs are unbounded and do not sum to 1, so they are not class probabilities. Fix: softmax, "typically used in the final layer for multi-class classification" (slide 89), which gives a valid distribution over the 10 classes.

STEP 8 Flaw 4, MSE with one-hot labels: MSE is the regression loss; for probabilities against a one-hot target the deck uses categorical cross-entropy (slides 89–91, Module 4), which penalises confident wrong answers and gives the clean gradient $\hat{y}_j - y_j$. Fix: softmax + CCE.

STEP 9 Corrected shapes: Conv 1 ($p = 0$) $62 \rightarrow \text{pool } \lfloor (62 - 2)/2 \rfloor + 1 = 31: 31 \times 31 \times 16$. Conv 2 ($p = 1$): $\lfloor (31 + 2 - 3)/1 \rfloor + 1 = 31: 31 \times 31 \times 32$. Pool: $\lfloor (31 - 2)/2 \rfloor + 1 = \lfloor 14.5 \rfloor + 1 = 15: 15 \times 15 \times 32$.

STEP 10 Flatten = $15 \times 15 \times 32 = 7,200$. Dense: $10 \times (7,200 + 1) = 72,010$. Convolution counts are unchanged (448 and 4,640): padding and pooling add no parameters.

STEP 11 New total = $448 + 4,640 + 72,010 = 77,098$. Dense parameters fell by $1,152,010/72,010 = 16.0$, a factor of 16 (two pools, each dividing the area by 4).

(a) $62 \times 62 \times 16, 60 \times 60 \times 32$; flatten 115,200

(b) 448, 4,640, 1,152,010; total 1,157,098; Dense 99.6 %

(c) sigmoid hidden $\rightarrow$ ReLU ($\sigma' \leq 0.25$); no pooling $\rightarrow 2 \times 2$ max pools; ReLU output $\rightarrow$ softmax; MSE $\rightarrow$ categorical cross-entropy

(d) $62 \rightarrow 31 \rightarrow 31 \rightarrow 15$; flatten 7,200; Dense 72,010; total 77,098; factor 16

✓ ANSWERS AT A GLANCE

(a) 62, 60; 115,200	(b) 1,157,098 (99.6 % Dense)	(c) 4 flaws, 4 fixes	(d) 7,200 $\rightarrow$ 72,010; total 77,098; $\times 16$
------------------------	---------------------------------	-------------------------	--

! WHERE MARKS ARE LOST

- Listing "sigmoid is old" without the bound 0.25: the mark is for the derivative bound and saturation.
- Calling MSE "wrong because it is slow": it is the wrong loss for probabilities against one-hot targets; cross-entropy is the deck's choice.
- Pooling 31 to 16 (rounding up): the floor gives 15 and a flatten of 7,200, not 8,192.
- Changing the convolution counts in (d): padding and pooling do not add parameters.

ResNet blocks, the residual sum, the gradient highway and transfer learning

Q6.10

8 marks Slides 101–102, 129–134, 140–141 · Book 6, Ch. 7 and 8

A team fine-tunes a ResNet for a 5-class plant-disease task with a small dataset of natural photos, similar to ImageNet. Biases are excluded in (a) unless stated; batch-norm parameters are ignored throughout.

- (a) A basic residual block has two 3×3 convolutions, $64 \rightarrow 64$. Compute its parameters without biases and with biases. [2]
- (b) A bottleneck block is 1×1 $256 \rightarrow 64$, 3×3 $64 \rightarrow 64$, 1×1 $64 \rightarrow 256$. Compute its parameters and compare with one direct 3×3 convolution $256 \rightarrow 256$ (ratio). [2]
- (c) In a tiny residual block $F(\mathbf{x}) = \mathbf{W}\mathbf{x}$ with $\mathbf{W} = \begin{bmatrix} 0.5 & 0.1 \\ -0.2 & 0.3 \end{bmatrix}$ and $\mathbf{x} = (2, 1)^\top$. Compute $F(\mathbf{x})$ and the block output $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$, and the output a plain block (no skip) would give. [1]
- (d) With the deck's gradient formula $\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \left(1 + \frac{\partial F}{\partial \mathbf{x}}\right)$, and $\frac{\partial F}{\partial \mathbf{x}} = 0.1$ in every block, compute the factor that multiplies the gradient per block for a plain net and for a residual net, and the factors after 5 blocks. [2]
- (e) From the guideline grid on slide 141, which transfer-learning strategy fits (small dataset, similar domain)? If only the final FC layer $512 \rightarrow 5$ is trainable, how many parameters are trained? [1] **justify**

count parameters

residual connections

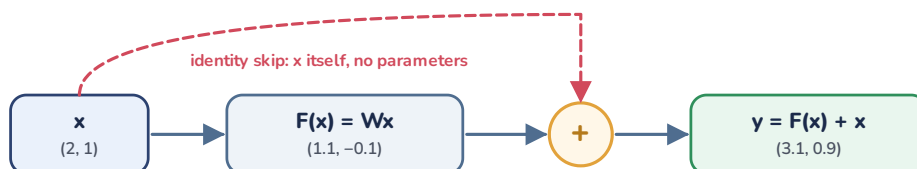
transfer learning

WHAT THIS TESTS

The block counts of slides 130–131 with exact arithmetic, the residual sum $H(x) = F(x) + x$ on real numbers, the "gradient highway" of slide 134 as a product of factors, and the (small, similar) cell of the transfer-learning grid with its trainable count.

HOW TO ATTEMPT

- $k^2 C_{in} C_{out}$ per convolution; add C_{out} per layer for biases.
- (c) is a 2×2 matrix-vector product, then add $\mathbf{x}$.
- (d): plain factor = $\partial F / \partial \mathbf{x}$; residual factor = $1 + \partial F / \partial \mathbf{x}$; raise both to the power 5.



backward: $\partial L / \partial \mathbf{x} = \partial L / \partial \mathbf{y} \cdot (1 + \partial F / \partial \mathbf{x})$ — the 1 comes from the dashed path

with $\partial F / \partial \mathbf{x} = 0.1$ per block: plain net $\times 0.1$ per block (10^{-5} over 5 blocks), residual $\times 1.1$ (1.61 over 5)

Figure 6.10 The residual block of part (c). Notice the dashed path: it carries $\mathbf{x}$ forward unchanged and carries the gradient backward with the factor 1 in $1 + \partial F / \partial \mathbf{x}$.

SOLUTION

STEP 1 One 3×3 convolution $64 \rightarrow 64$: $3 \times 3 \times 64 \times 64 = 9 \times 4096 = 36,864$. Two of them: $2 \times 36,864 = 73,728$.

STEP 2 With biases: each layer adds 64, so $73,728 + 128 = 73,856$.

(a) the identity skip adds no parameters

STEP 3 1×1 $256 \rightarrow 64$: $256 \times 64 = 16,384$; 3×3 $64 \rightarrow 64$: $36,864$; 1×1 $64 \rightarrow 256$: $16,384$. Total = $16,384 + 36,864 + 16,384 = 69,632$.

STEP 4 Direct 3×3 $256 \rightarrow 256$: $9 \times 256 \times 256 = 589,824$. Ratio $589,824 / 69,632 = 8.47$ (slide 131: "70K vs 590K, 8.4× reduction").

STEP 5 $F(\mathbf{x}) = \mathbf{W}\mathbf{x}$: $0.5(2) + 0.1(1) = 1.0 + 0.1 = 1.1$; $-0.2(2) + 0.3(1) = -0.4 + 0.3 = -0.1$. So $F(\mathbf{x}) = (1.1, -0.1)^\top$.

STEP 6 $\mathbf{y} = F(\mathbf{x}) + \mathbf{x} = (1.1 + 2, -0.1 + 1)^\top = (3.1, 0.9)^\top$. A plain block outputs only $F(\mathbf{x}) = (1.1, -0.1)^\top$.

(c) the block only has to learn the small change F ; the input passes through for free

STEP 7 Plain net: the factor per block is $\partial F / \partial x = 0.1$; after 5 blocks $0.1^5 = 10^{-5}$.

(d)

STEP 8 Residual net: $1 + 0.1 = 1.1$ per block; after 5 blocks $1.1^5 = 1.611$. Key fact (slide 134): the "1" gives the gradient a direct path, so it never shrinks to nothing however small $\partial F / \partial x$ is; even with $\partial F / \partial x = -0.9$ the factor is 0.1, not 0.

(d)

STEP 9 Key fact (slides 140–141): small dataset + similar domain is the feature-extraction cell — "freeze conv layers, train classifier only". Trainable parameters: FC $512 \rightarrow 5$ with bias = $5 \times (512 + 1) = 2,565$, about 0.02% of ResNet-18's 11.7 M (slide 133); all convolution weights stay frozen.

(e)

(a) 73,728 without biases; 73,856 with

(b) 69,632 vs 589,824; ratio 8.47

(c) $F(\mathbf{x}) = (1.1, -0.1)^\top$, $\mathbf{y} = (3.1, 0.9)^\top$; plain block $(1.1, -0.1)^\top$

(d) 0.1 vs 1.1 per block; 10^{-5} vs 1.611 after 5 blocks

(e) feature extraction: freeze conv layers, train the classifier; 2,565 parameters

✓ ANSWERS AT A GLANCE

(a) 73,728 / 73,856 (b) 69,632; $\times 8.47$ (c) (3.1, 0.9) (d) 10^{-5} vs 1.611 (e) freeze conv; 2,565

! WHERE MARKS ARE LOST

- Giving the skip connection parameters: an identity shortcut has none (a projection shortcut would, slide 132).
- Forgetting to add $\mathbf{x}$ in (c): $(1.1, -0.1)$ is the plain output, not the residual one.
- Writing the residual factor as $\partial F / \partial x$ alone: the mark is for the $1 +$ and its product over blocks.
- Choosing fine-tuning of all layers for a small dataset: the grid says freeze the convolutions; trainable count 2,565, not 11.7 M.